
DATA ANALYST

Série 2 — Volume I

Cahier d'exercices et d'applications

Exercices corrigés · Applications guidées · Projet
Compagnon du volume I : Fondations

À utiliser avec le livre : chaque chapitre renvoie ici
pour s'entraîner.

Table des matières

Mode d'emploi	1
Comment utiliser ce cahier	1
Numérotation et niveaux	1
Données et environnement	2
Lancer le code	3
Vérifier son installation	3
Mini-diagnostic de départ	4
Corrigés du mini-diagnostic	5
Chapitre 1 : Les essentiels de la statistique — exercices et applications	7
Applications	7
Application 1.1 — Résumer un panier (section 1.1)	7
Application 1.2 — Moyennes qui trompent (section 1.1.4)	9
Application 1.3 — Excel contre pandas (section 1.1.5)	10
Application 1.4 — Quelle loi pour quoi ? (section 1.2)	11
Application 1.5 — Simuler un échantillonnage (section 1.3.2)	12
Application 1.6 — Intervalles de confiance et biais (sections 1.3.3 à 1.3.6)	13
Application 1.7 — Corrélation, confusion et saison (section 1.4)	14
Application 1.8 — Mathématiques du quotidien (section 1.5)	15
Exercices	16
Exercice 1.1 ★ — Moyenne et médiane à la main (section 1.1.1)	16
Exercice 1.2 ★ — Écart-type d'un petit échantillon (section 1.1.2)	16
Exercice 1.3 ★★ — La règle de 1,5 écart interquartile (section 1.1.3)	16
Exercice 1.4 ★ — Moyenne des moyennes (section 1.1.4)	17
Exercice 1.5 ★★ — Un renversement de Simpson (section 1.1.4)	17
Exercice 1.6 ★ — Score z et normale (section 1.2.3)	17
Exercice 1.7 ★★ — Binomiale et Poisson à la main (section 1.2.2)	17
Exercice 1.8 ★ — Erreur type (section 1.3.2)	17
Exercice 1.9 ★★ — Intervalle de confiance d'une proportion (section 1.3.4)	17
Exercice 1.10 ★★ — Taille d'échantillon (section 1.3.5)	17
Exercice 1.11 ★ — Corrélation à la main (section 1.4.1)	17
Exercice 1.12 ★★★ — Concevoir une expérience (section 1.4.6)	18
Exercice 1.13 ★ — Pourcentages en cascade (section 1.5.1)	18
Exercice 1.14 ★★ — Marge, marque, TVA (section 1.5.4)	18
Corrigés	18
Corrigé 1.1	18
Corrigé 1.2	18
Corrigé 1.3	19
Corrigé 1.4	19

Corrigé 1.5	19
Corrigé 1.6	19
Corrigé 1.7	20
Corrigé 1.8	20
Corrigé 1.9	20
Corrigé 1.10	20
Corrigé 1.11	20
Corrigé 1.12	21
Corrigé 1.13	21
Corrigé 1.14	21
Chapitre 2 : Excel — exercices et applications	22
Applications	22
Application 2.1 — Explorer et contrôler le classeur (introduction, section 2.4) . . .	22
Application 2.2 — Agréger sous conditions (section 2.1.4)	23
Application 2.3 — Enrichir par recherche : le coût d'achat et la marge (section 2.1.6)	24
Application 2.4 — Nettoyer du texte et des dates (sections 2.1.7 et 2.1.8)	25
Application 2.5 — Recouper un tableau croisé dynamique (section 2.2)	26
Application 2.6 — Rejouer une chaîne Power Query sur plusieurs fichiers (section 2.3)	26
Application 2.7 — Auditer un classeur abîmé (section 2.4)	27
Application 2.8 — Trianguler : formules, SQL et pandas (sections 2.5 et 2.6)	28
Exercices	29
Exercice 2.1 ★ — Que devient une formule recopiée ? (section 2.1.2)	29
Exercice 2.2 ★ — SOMME.SI.ENS à la main (section 2.1.4)	29
Exercice 2.3 ★★ — Trois tailles de ligne (section 2.1.5)	29
Exercice 2.4 ★★ — Un barème de remise par palier (section 2.1.6)	29
Exercice 2.5 ★ — Dates (section 2.1.8)	30
Exercice 2.6 ★★ — Un tableau croisé à la main (section 2.2)	30
Exercice 2.7 ★★ — Moyenne des ratios ou ratio des sommes ? (section 2.2.4)	30
Exercice 2.8 ★★ — L'ordre des étapes compte (section 2.3)	30
Exercice 2.9 ★★ — Dépivoter et repivoter (section 2.3.4)	30
Exercice 2.10 ★★ — Remettre un tableau en forme (section 2.4.2)	30
Exercice 2.11 ★★ — Cinq contrôles (section 2.4.4)	30
Exercice 2.12 ★★ — Le panier moyen par canal, dans cinq outils (sections 2.5 et 2.6)	30
Corrigés	31
Corrigé 2.1	31
Corrigé 2.2	31
Corrigé 2.3	31
Corrigé 2.4	32
Corrigé 2.5	32
Corrigé 2.6	33
Corrigé 2.7	33
Corrigé 2.8	33
Corrigé 2.9	34
Corrigé 2.10	34
Corrigé 2.11	34
Corrigé 2.12	35
Chapitre 3 : SQL — exercices et applications	36
Applications	36

Application 3.1 — Découvrir la base (section 3.1)	36
Application 3.2 — Les ventes par canal et par année (sections 3.1 et 3.2)	38
Application 3.3 — Les meilleurs clients (sections 3.2 et 3.4)	38
Application 3.4 — Panier moyen et double comptage (section 3.2.6)	39
Application 3.5 — Les retours : combien, d'où, pourquoi (sections 3.2 et 3.4)	40
Application 3.6 — Évolution mensuelle, cumul et comparaison à l'an dernier (section 3.3)	41
Application 3.7 — Fidélité et cohortes (sections 3.3 et 3.4)	42
Application 3.8 — Une segmentation RFM en SQL (sections 3.3 et 3.4)	43
Application 3.9 — Index et plan d'exécution (section 3.5)	43
Exercices	44
Exercice 3.1 ★ — Les produits à forte marge (section 3.1.2)	44
Exercice 3.2 ★ — Une condition composée (section 3.1.3)	44
Exercice 3.3 ★★ — Les heures de pointe (section 3.1.5)	44
Exercice 3.4 ★★ — Pourquoi 0 ? (section 3.1.9)	44
Exercice 3.5 ★ — Le chiffre d'affaires par ville (section 3.2.2)	45
Exercice 3.6 ★★ — Les inscrits sans commande (section 3.2.3)	45
Exercice 3.7 ★★ — Un chiffre d'affaires trop beau (section 3.2.6)	45
Exercice 3.8 ★★ — Les clients perdus, par canal d'acquisition (section 3.2.7)	45
Exercice 3.9 ★★ — Les trois meilleurs produits par catégorie (section 3.3.3)	45
Exercice 3.10 ★★ — Le délai de la deuxième commande (section 3.3.4)	45
Exercice 3.11 ★★ — La meilleure semaine (section 3.3.5)	45
Exercice 3.12 ★★ — Un organigramme (section 3.4.5)	45
Exercice 3.13 ★★ — Changer de dialecte (section 3.6)	46
Exercice 3.14 ★★ — Sous-requête ou fenêtre ? (sections 3.5.1 et 3.5.2)	46
Corrigés	46
Corrigé 3.1	46
Corrigé 3.2	46
Corrigé 3.3	47
Corrigé 3.4	47
Corrigé 3.5	47
Corrigé 3.6	48
Corrigé 3.7	48
Corrigé 3.8	48
Corrigé 3.9	49
Corrigé 3.10	50
Corrigé 3.11	50
Corrigé 3.12	50
Corrigé 3.13	51
Corrigé 3.14	51
Chapitre 4 : Python et R pour l'analyse — exercices et applications	53
Applications	53
Application 4.1 — Premier contact avec les commandes (sections 4.1.2 à 4.1.4)	53
Application 4.2 — Les retours (sections 4.1.5 à 4.1.7)	54
Application 4.3 — Le même travail en R (section 4.2)	55
Application 4.4 — Remettre en état l'export de caisse (section 4.3.1)	55
Application 4.5 — Jointures et marge par produit (sections 4.3.2 et 4.3.3)	57
Application 4.6 — Restructurer : le chiffre d'affaires de chaque canal, mois par mois (section 4.3.4)	57
Application 4.7 — Le tableau du lundi, en fonction (sections 4.3.5 et 4.3.6)	58
Application 4.8 — Un notebook reproductible (section 4.4)	59

Application 4.9 — Un graphique avec ggplot2 (section 4.5)	60
Application 4.10 — polars et NumPy (section 4.6)	62
Exercices	62
Exercice 4.1 ★ — Types et lecture (section 4.1.3)	62
Exercice 4.2 ★ — .loc contre .iloc (section 4.1.4)	62
Exercice 4.3 ★ — Filtres combinés (section 4.1.5)	63
Exercice 4.4 ★★ — Colonnes calculées (section 4.1.6)	63
Exercice 4.5 ★ — Manquants et comptages (section 4.1.7)	63
Exercice 4.6 ★ — Traduire en tidyverse (section 4.2)	63
Exercice 4.7 ★★ — Panier moyen par canal et par année (section 4.3.2)	63
Exercice 4.8 ★★ — Taux de retour par mode de livraison (section 4.3.3)	63
Exercice 4.9 ★★ — Large et long (section 4.3.4)	63
Exercice 4.10 ★★ — Moyenne glissante (section 4.3.5)	63
Exercice 4.11 ★★ — L'état caché (section 4.4)	64
Exercice 4.12 ★★ — Année du calendrier contre année ISO (sections 4.3.5 et 4.3.6)	64
Exercice 4.13 ★★ — polars (section 4.6)	64
Corrigés	64
Corrigé 4.1	64
Corrigé 4.2	64
Corrigé 4.3	65
Corrigé 4.4	65
Corrigé 4.5	65
Corrigé 4.6	65
Corrigé 4.7	66
Corrigé 4.8	66
Corrigé 4.9	67
Corrigé 4.10	67
Corrigé 4.11	67
Corrigé 4.12	68
Corrigé 4.13	68
Chapitre 5 : Types de données, collecte et conception d'enquêtes — exercices et applications	69
Applications	69
Application 5.1 — La fiche d'identité d'une table (sections 5.1 et 5.2)	69
Application 5.2 — Lire avec les bons types (section 5.1.3)	70
Application 5.3 — Le piège du grain (section 5.1.4)	71
Application 5.4 — Nettoyer l'enquête et calculer un NPS (sections 5.3.5 et 5.3.6)	72
Application 5.5 — Répondants contre invités : pondérer (section 5.3.3 et 5.3.4)	73
Application 5.6 — Plans de sondage et taille d'échantillon (section 5.4)	74
Application 5.7 — Une API paginée et limitée en débit (section 5.5.2)	74
Application 5.8 — Moissonner un catalogue poliment (section 5.5.3)	75
Exercices	76
Exercice 5.1 ★ — Classer des variables (section 5.1.1)	76
Exercice 5.2 ★ — Quel résumé pour quelle variable ? (section 5.1.2)	76
Exercice 5.3 ★ — Lire sans rien perdre (section 5.1.3)	76
Exercice 5.4 ★★ — Combien de commandes par client ? (section 5.1.4)	77
Exercice 5.5 ★ — Recouper deux sources (section 5.2.2)	77
Exercice 5.6 ★★ — Rendre un fichier plus anonyme (section 5.2.3)	77
Exercice 5.7 ★ — Pondérer à la main puis par code (section 5.3.4)	77
Exercice 5.8 ★★ — Les bornes sans hypothèse, par canal (section 5.3.4)	77
Exercice 5.9 ★★ — Un NPS à la main, puis sur le fichier (section 5.3.6)	77

Exercice 5.10 ★★ — Combien de répondants pour voir une formulation ? (section 5.4.1)	77
Exercice 5.11 ★ — Dimensionner une enquête (section 5.4.3)	78
Exercice 5.12 ★★★ — Un client robuste pour une collecte hebdomadaire (section 5.5)	78
Corrigés	78
Corrigé 5.1	78
Corrigé 5.2	78
Corrigé 5.3	79
Corrigé 5.4	79
Corrigé 5.5	79
Corrigé 5.6	80
Corrigé 5.7	80
Corrigé 5.8	80
Corrigé 5.9	81
Corrigé 5.10	81
Corrigé 5.11	81
Corrigé 5.12	82
Projet du volume et auto-évaluation — exercices et applications	83
Projet du volume	83
P.1 Le cahier des charges	83
P.2 Étape 1 : comprendre les données	84
P.3 Étape 2 : lire le fichier brut de la caisse	84
P.4 Étape 3 : réconcilier la caisse et la base	85
P.5 Étape 4 : la synthèse du dernier trimestre, par trois outils	87
P.6 Étape 5 : les indicateurs de la synthèse	88
P.7 Étape 6 : le classeur de synthèse	89
P.8 Étape 7 : le graphique	90
P.9 Étape 8 : le message à la gérante	90
P.10 Les limites de l'étude	91
P.11 Variante : l'enquête de satisfaction	92
Auto-évaluation	92
Les essentiels de la statistique (chapitre 1)	92
Excel (chapitre 2)	93
SQL (chapitre 3)	93
Python et R (chapitre 4)	93
Types de données, collecte et enquêtes (chapitre 5)	94
Corrigés des questions	94
Votre grille d'auto-évaluation	98

Mode d'emploi

« On comprend en lisant, on retient en calculant. »

Ce cahier est le **compagnon du livre** du volume I (*Fondations*). Le livre explique les idées ; le cahier les fait travailler. Il contient, chapitre par chapitre, des **applications guidées**, des **exercices** et leurs **corrigés**, puis le **projet du volume** (une première analyse, du fichier brut au tableau de synthèse) et l'auto-évaluation.

Comment utiliser ce cahier

1. **Lisez d'abord la section du livre.** Chaque section qui a un prolongement ici se termine par une ligne de ce genre : > ■ **Pour s'entraîner.** Cahier, chapitre 1 : application 1.1, exercices 1.1 à 1.4.
2. **Essayez avant de regarder le corrigé.** Les énoncés sont regroupés dans la partie *Exercices*, les corrigés dans la partie *Corrigés* du même chapitre : on peut chercher sans voir la réponse.
3. **Calculez d'abord à la main.** Presque chaque notion de ce volume se vérifie sur quatre ou cinq lignes d'un tableau : faites-le avec un crayon avant d'écrire du code. Le code confirme ; il ne remplace pas la compréhension.
4. **Faites les applications dans l'ordre.** Chacune raconte une petite étude de la boutique : une question de la gérante, des étapes, du code, puis une lecture des résultats.
5. **Vérifiez par un second chemin.** Quand c'est possible, obtenez le même chiffre avec un autre outil (tableur, SQL, Python, R). C'est le meilleur détecteur d'erreurs.
6. **Dites ce que le chiffre ne dit pas.** Un résultat sans période, sans source, sans limite n'est pas un résultat.

Numérotation et niveaux

Chaque chapitre du cahier correspond au chapitre du livre de même numéro.

Élément	Numérotation	Exemple
Application	Application N.k	Application 2.1 : première application du chapitre 2
Exercice	Exercice N.k	Exercice 2.3 : troisième exercice du chapitre 2
Corrigé	Corrigé N.k	Corrigé 2.3 : correction de l'exercice 2.3

La difficulté des exercices est indiquée par des étoiles :

Niveau	Signification
★	application directe d'une idée ou d'un geste vu dans la section
★★	demande de combiner deux idées, ou un petit raisonnement
★★★	demande de la réflexion, un choix de méthode ou une petite expérience

Un exercice cite la section du livre qu'il exerce, par exemple « (section 1.2) ».

Données et environnement

Les données sont celles du livre, dans le dossier `donnees/`. Elles sont toutes **simulées**, avec des graines fixes (script `build/donnees_a1.py`) : vos résultats seront identiques à ceux du livre. Le catalogue complet, avec les colonnes de chaque fichier, figure dans la section « Carte du volume, données et environnement » du livre ; en voici l'essentiel.

Fichier	Contenu	Chapitres
<code>clients.csv</code> , <code>produits.csv</code>	6 000 clients ; 120 produits en six catégories	tous
<code>commandes.csv</code> , <code>lignes_commande.csv</code> , <code>retours.csv</code>	commandes 2023-2025, leurs lignes, les retours	1 à 4
<code>jours_exploitation.csv</code>	un jour par ligne : commandes, chiffre d'affaires, météo, promotion, publicité	1, 2, 4
<code>enquete_satisfaction.csv</code>	enquête de satisfaction (958 réponses)	1, 5, projet
<code>ventes_2025.xlsx</code>	classeur Excel de 2025 (feuilles Lignes, Produits, Clients)	2
<code>export_caisse_brut.csv</code>	export de caisse désordonné (une semaine)	2, 4, projet
<code>boutique.db</code>	base SQLite avec les six premières tables	3, 4

Les chiffres sont fictifs. La boutique, ses clients et ses ventes sont inventés. Ne tirez de ces données aucune conclusion sur le monde réel : elles servent à apprendre une méthode.

Chaque chapitre du cahier est **autonome** : il commence par ses imports et recharge ses données. Les applications sont dimensionnées pour s'exécuter en **quelques secondes à quelques dizaines de secondes** sur un ordinateur ordinaire, sans réseau.

Lancer le code

Placez-vous dans le dossier du volume (celui qui contient `donnees/`), car les chemins sont relatifs (`donnees/commandes.csv`). Vous pouvez travailler dans un notebook Jupyter, dans un éditeur avec la commande `python`, dans RStudio pour R, ou dans un outil de bases de données pour SQL.

Les chemins. Si Python répond `FileNotFoundError`, c'est presque toujours que vous n'êtes pas dans le bon dossier : vérifiez avec `import os; print(os.getcwd())`.

Les graines. Les simulations du cahier fixent leur graine (`np.random.default_rng(42)`, par exemple). Si vous la changez, vos chiffres changeront un peu : c'est une excellente façon de **voir l'incertitude** d'une estimation, mais ne comparez pas alors vos résultats à ceux du corrigé à la décimale.

Excel. Les exercices du chapitre 2 se font dans votre tableur. Si vous n'avez pas Excel, LibreOffice Calc (gratuit) lit le même fichier `ventes_2025.xlsx` et accepte presque toutes les formules du livre ; quand un menu ou une fonction diffère, la documentation de votre version fait foi.

Vérifier son installation

Quatre courts blocs vérifient que les bibliothèques sont installées et que les données se chargent, en Python, en SQL puis en R. Les numéros de version peuvent différer des nôtres, à condition que tout s'exécute.

1. Les bibliothèques Python.

```
import sys
from importlib import import_module
from importlib.metadata import version
print("python", sys.version.split()[0])
for p in ["numpy", "pandas", "scipy", "statsmodels", "matplotlib", "openpyxl", "polars", "duckdb"]:
    import_module(p)
    print(f"{p:12s}", version(p))
```

```
python 3.13.3
numpy      2.5.3
pandas     3.0.6
scipy      1.18.1
statsmodels 0.15.0
matplotlib 3.11.2
openpyxl   3.1.5
polars     2.0.0
duckdb     1.5.6
```

2. Les données et la base. Nous chargeons les commandes, ouvrons la base SQLite et comparons les deux comptes.

```
import sqlite3
import pandas as pd

commandes = pd.read_csv("donnees/commandes.csv")
```

```
con = sqlite3.connect("donnees/boutique.db")
print("fichier CSV :", len(commandes), "commandes,", commandes["id_client"].nunique(), "clients
↳ différents")
```

fichier CSV : 36395 commandes, 4806 clients différents

3. Une requête SQL sur la même connexion con :

```
SELECT canal, COUNT(*) AS commandes
FROM commandes
GROUP BY canal
ORDER BY commandes DESC;
```

canal	commandes
Boutique	16975
Site	15463
Réseaux	3957

4. Le même calcul en R, pour vérifier que R et le tidyverse sont là :

```
suppressPackageStartupMessages({library(readr); library(dplyr)})
commandes <- read_csv("donnees/commandes.csv", show_col_types = FALSE)
print(count(commandes, canal, sort = TRUE))
```

```
# A tibble: 3 × 2
  canal      n
  <chr> <int>
1 Boutique 16975
2 Site     15463
3 Réseaux  3957
```

Vous devez lire une version pour chaque bibliothèque, **36 395 commandes** pour **4 806 clients différents** (sur les 6 000 inscrits : tous n'ont pas commandé), et **le même tableau** par canal en SQL et en R : la boutique en tête avec 16 975 commandes, puis le Site avec 15 463 et les réseaux avec 3 957. Si les trois comptes concordent, votre installation est prête.

Mini-diagnostic de départ

Huit questions pour vérifier que les gestes de base sont en place. Répondez par écrit, puis comparez avec les corrigés plus bas ; chaque corrigé indique le chapitre à lire en cas d'hésitation. Il n'y a pas de note : l'objectif est de savoir **où revenir** avant de commencer.

1. Un article à 40 € est soldé de 20 %. Une carte de fidélité accorde ensuite 10 % de remise **sur le prix déjà soldé**. Quel est le prix final, et quelle est la remise totale en pourcentage du prix de départ ?
2. Cinq jours de commandes : 12, 15, 23, 11 et 19. Calculez la moyenne et la médiane. Le sixième jour, jour de promotion, compte 60 commandes : que deviennent la moyenne et la médiane ?
3. La boutique a reçu 100 commandes au panier moyen de 80 € et 300 commandes au panier moyen de 120 €. Quel est le panier moyen de l'ensemble ?
4. Dans commandes.csv, combien de commandes du canal Site passées en **2025** portent un code promo ? Quelle part cela représente-t-il des commandes du Site de 2025 ?
5. Trois tableaux. **Produits** : P1 « Bougie » (Décoration, 10 €), P2 « Plaid » (Maison, 30 €), P3 « Cahier » (Papeterie, 5 €). **Lignes** : (L1, P1, quantité 2), (L2, P2, 1), (L3, P1, 1), (L4, P3, 3). Quel est le chiffre d'affaires de chaque catégorie ? De quelle opération avez-vous eu besoin

?

6. Le tableau ci-dessous donne les commandes de 2025 selon le canal et le mode de livraison. Quelle part des commandes du Site est livrée à domicile ?

mode_livraison	Domicile	Point relais	Retrait magasin	Total
canal				
Boutique	0	0	5442	5442
Réseaux	776	551	99	1426
Site	3374	2272	432	6078
Total	4150	2823	5973	12946

7. Le chiffre d'affaires de la boutique est de 1 138 932 € en 2023 et de 1 324 764 € en 2025. Quelle est la croissance sur les deux ans ? Quel est le taux de croissance **annuel moyen** ?
8. L'enquête de satisfaction a été envoyée à 3 875 clients ; 958 ont répondu, et 55 % des répondants donnent une note de 4 ou 5. La gérante en conclut : « 55 % de nos clients sont satisfaits ». Cette conclusion est-elle justifiée ?

Corrigés du mini-diagnostic

1. La remise de 20 % ramène le prix à $40 \times 0,80 = 32$ €. Les 10 % s'appliquent à 32 € : $32 \times 0,90 = 28,80$ €. La remise totale est $(40 - 28,80)/40 = 28$ %, et **non 30 %** : les pourcentages de remises successives ne s'additionnent pas, ils se **multiplient** ($0,80 \times 0,90 = 0,72$). À relire : section 1.5 (pourcentages).

2. Moyenne : $(12 + 15 + 23 + 11 + 19)/5 = 80/5 = 16$; médiane : on trie (11, 12, 15, 19, 23), la valeur du milieu est **15**. Avec 60 commandes le sixième jour : la moyenne devient $140/6 \approx 23,3$, la médiane (11, 12, 15, 19, 23, 60 : moyenne des deux valeurs du milieu) $(15 + 19)/2 = 17$. La moyenne a bondi de 46 %, la médiane de 13 % : la **médiane résiste aux valeurs extrêmes**. À relire : section 1.1.

3. Il ne faut pas faire $(80 + 120)/2 = 100$: les deux canaux n'ont pas le même poids. La **moyenne pondérée** donne $(100 \times 80 + 300 \times 120)/400 = 44\,000/400 = 110$ €. À relire : section 1.5 (moyennes pondérées).

4. On filtre sur l'année, le canal et la présence d'un code promo. Les valeurs vides de code_promo signifient « pas de promotion » :

```
site25 = c25[c25["canal"] == "Site"]
avec_code = site25["code_promo"].notna().sum()
print(avec_code, "sur", len(site25), "soit", round(100 * avec_code / len(site25), 1), "%")
```

960 sur 6078 soit 15.8 %

960 commandes sur 6 078, soit 15,8 %. Le même calcul en SQL serait `SELECT COUNT(code_promo) FROM commandes WHERE canal = 'Site' AND date_commande >= '2025-01-01'` (le comptage d'une colonne ignore les valeurs vides). À relire : sections 3.1 et 4.3.

5. Il faut **joindre** les lignes aux produits (par id_produit), calculer le montant de chaque ligne (quantité $\times$ prix), puis **agréger** par catégorie : L1 et L3 donnent $2 \times 10 + 1 \times 10 = 30$ € pour la Décoration ; L2 donne $1 \times 30 = 30$ € pour la Maison ; L4 donne $3 \times 5 = 15$ € pour la Papeterie. L'opération clé est la **jointure** (sans elle, on ne connaît pas la catégorie des lignes). À relire : section 3.2.

6. Les commandes du Site en 2025 sont au nombre de 6 078 (colonne Total). Le tableau ci-dessus donne le nombre de commandes livrées à domicile ; la part s'obtient en divisant cette cellule par le total de la ligne Site :

```
t = pd.crosstab(c25["canal"], c25["mode_livraison"])
print((t.loc["Site"] / t.loc["Site"].sum()).round(3).to_dict())
```

```
{'Domicile': 0.555, 'Point relais': 0.374, 'Retrait magasin': 0.071}
```

La part des commandes du Site livrée à domicile est de **55,5 %** environ, contre 37,4 % en point relais et 7,1 % en retrait en magasin. Notez qu'il faut diviser par le total de la **ligne** (le canal), pas par le total général ; ce sont deux proportions différentes. À relire : sections 2.2 (tableaux croisés) et 1.1.

7. La croissance sur deux ans est $1\,324\,764 / 1\,138\,932 - 1 \approx 16,3 \%$. Le taux annuel moyen n'est **pas** la moitié de 16,3 % : chaque année s'applique au résultat de la précédente. On cherche g tel que $(1 + g)^2 = 1,163$, soit $g = \sqrt{1,163} - 1 \approx 7,85 \%$ par an (la moitié, 8,2 %, surestimerait). À relire : section 1.5 (taux de croissance).

```
ca23, ca25 = 1_138_932, 1_324_764
print(round(100 * (ca25 / ca23 - 1), 1), "% en deux ans |", round(100 * ((ca25 / ca23) ** 0.5 - 1),
↪ 2), "% par an")
```

```
16.3 % en deux ans | 7.85 % par an
```

8. **Non, pas sans précaution.** Deux raisons. D'abord, seulement **24,7 %** des clients invités ont répondu (958 sur 3 875) : on ne sait rien des trois quarts restants. Ensuite, ceux qui répondent **ne ressemblent pas** à ceux qui ne répondent pas ; par exemple, les clients avec carte de fidélité y sont surreprésentés (35,6 % des invités, 40,1 % des répondants identifiés) :

```
enq = lire("enquete_satisfaction")
clients = lire("clients")
inv = c25.drop_duplicates("id_client", keep="last").merge(clients[["id_client", "fidelite"]],
↪ on="id_client")
rep = enq.merge(clients[["id_client", "fidelite"]], on="id_client")
print("cartes parmi les invités :", round(100 * inv["fidelite"].mean(), 1), "% | parmi les répondants
↪ identifiés :", round(100 * rep["fidelite"].mean(), 1), "%")
print("réponses :", len(enq), "| invitations :", len(inv), "| notes 4 ou 5 :", round(100 *
↪ (enq["satisfaction_globale"] >= 4).mean(), 1), "%")
```

```
cartes parmi les invités : 35.6 % | parmi les répondants identifiés : 40.1 %
réponses : 958 | invitations : 3875 | notes 4 ou 5 : 55.2 %
```

Les 55 % décrivent **les répondants**, pas la clientèle ; on peut tout au plus dire « parmi les clients qui ont répondu, 55 % ont donné 4 ou 5 », et chercher comment corriger le biais de réponse. À relire : sections 1.3 (échantillonnage), 5.3 et 5.4 (enquêtes et biais).

À retenir. Si vous avez répondu juste aux questions 1 à 3, les calculs de base (pourcentages, moyennes) sont en place. Les questions 4 à 6 testent la lecture et la manipulation de tableaux (filtrer, joindre, croiser) : les chapitres 2 à 4 s'en chargent. Les questions 7 et 8 testent le **raisonnement** (croissance, biais) : le chapitre 1 et le chapitre 5 les approfondissent.

Chapitre 1 : Les essentiels de la statistique — exercices et applications

Ce chapitre du cahier accompagne le chapitre 1 du livre. Il contient **huit applications guidées** (de petites études que vous refaites sur les données de la boutique) et **quatorze exercices** de difficulté croissante (★ calcul à la main, ★★ calcul et interprétation, ★★★ réflexion de méthode), tous **corrigés**. Les applications se font devant l'ordinateur, étape par étape ; les exercices se font d'abord **sur papier**. Chaque exercice renvoie à la section du livre qui donne les outils.

Une seule cellule charge les bibliothèques et les données. Elle est reprise telle quelle au début de chaque application : si vous travaillez sur un notebook, exécutez-la une fois.

```
import sys, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
from scipy import stats
import outils_ch01 as O

d = O.charger()                                # commandes (avec leur panier), lignes, clients, retours, jours
cmd, lig, jours, cli = d.commandes, d.lignes, d.jours, d.clients
panier = cmd["panier"]
print(len(cmd), "commandes ;", len(lig), "lignes ;", len(jours), "jours ;", len(cli), "clients")

36395 commandes ; 83905 lignes ; 1096 jours ; 6000 clients
```

Applications

Application 1.1 — Résumer un panier (section 1.1)

Objectif. Décrire les paniers de la boutique **par canal**, repérer les valeurs inhabituelles, et refaire à la machine le calcul à la main du livre sur neuf commandes.

Étape 1 — Un tableau de résumés. On calcule, pour chaque canal, l'effectif, la moyenne, la médiane et l'écart-type.

```
r = cmd.groupby("canal")["panier"].agg(["count", "mean", "median", "std"]).round(2)
r["cv"] = (r["std"] / r["mean"]).round(2)
print(r)
```

	count	mean	median	std	cv
canal					
Boutique	16975	100.90	79.80	80.87	0.80
Réseaux	3957	100.52	79.70	81.48	0.81
Site	15463	99.77	79.76	80.39	0.81

Étape 2 — Les valeurs inhabituelles. On applique la règle de Tukey (1,5 écart interquartile) à chaque canal, puis on regarde les cinq plus grosses commandes.

```
def bornes(x):
    q1, q3 = x.quantile([0.25, 0.75])
    return q3 + 1.5 * (q3 - q1)
haut = cmd.groupby("canal")["panier"].apply(bornes).round(1)
n_ab = cmd.assign(h=cmd["canal"].map(haut)).query("panier > h").groupby("canal").size()
print(pd.DataFrame({"borne haute": haut, "valeurs au-delà": n_ab}))
nl = lig.groupby("id_commande").size()
print(cmd.nlargest(5, "panier")[["id_commande", "canal", "panier"]].assign(lignes=lambda t:
    ↪ t["id_commande"].map(nl)).to_string(index=False))
```

	borne haute	valeurs au-delà	
canal			
Boutique	276.9		635
Réseaux	277.0		147
Site	270.7		610
id_commande	canal	panier	lignes
27558	Réseaux	987.90	5
24157	Boutique	787.05	5
33982	Boutique	783.96	6
30123	Réseaux	772.53	6
31685	Boutique	752.96	6

Étape 3 — Concentration du chiffre d'affaires. Quelle part du chiffre d'affaires représentent les 10 % de commandes les plus importantes ?

```
tri = panier.sort_values(ascending=False).values
part = tri[: int(0.1 * len(tri))].sum() / tri.sum()
print("part des 10 % de commandes les plus grosses :", round(100 * part, 1), "%")
```

part des 10 % de commandes les plus grosses : 27.9 %

Étape 4 — Le calcul à la main, vérifié. Reprenez les neuf commandes du 15 avril 2025 en boutique et retrouvez les résultats du livre (moyenne 102,78 ; médiane 66,75 ; écart-type 85,83).

```
x9 = cmd[(cmd["date_commande"] == "2025-04-15") & (cmd["canal"] ==
    ↪ "Boutique")]["panier"].head(9).round(2)
print(sorted(x9), "| moyenne", round(x9.mean(), 2), "| médiane", x9.median(), "| écart-type",
    ↪ round(x9.std(), 2))
```

```
[9.69, 22.56, 48.74, 50.27, 66.75, 128.25, 132.37, 207.56, 258.83] | moyenne 102.78 | médiane 66.75 |
    ↪ écart-type 85.83
```

Lecture. Les trois canaux se ressemblent beaucoup (même moyenne, même médiane, même dispersion, coefficient de variation voisin de 0,80) : le canal ne différencie pas les paniers. Au total, 1 392 commandes dépassent la borne de Tukey de leur canal ; les cinq plus grosses comptent au moins 5 lignes chacune : elles sont **légitimes** (de grosses commandes), pas des erreurs. Les 10 % de commandes les plus importantes pèsent 28 % du chiffre d'affaires.

À vous. Refaites l'étape 2 avec la règle de 3 écarts-types autour de la moyenne au lieu de 1,5 EIQ : combien de commandes sont signalées, et pourquoi la règle est-elle moins adaptée ici ? *Piste : la moyenne et l'écart-type sont eux-mêmes tirés par les grosses commandes ; la règle de Tukey s'appuie sur les quartiles, qui ne le sont pas.*

Application 1.2 — Moyennes qui trompent (section 1.1.4)

Objectif. Constater que la moyenne de moyennes diffère de la moyenne globale, et fabriquer un paradoxe de Simpson.

Étape 1 — Taux de retour : moyenne simple ou taux global ? Calculez le taux de retour par canal, puis par catégorie, et comparez la moyenne simple des taux au taux global.

```
res = {}
for cle in ("canal", "categorie"):
    g = lig.groupby(cle)["retournee"].agg(["sum", "count"])
    g["taux_%"] = (100 * g["sum"] / g["count"]).round(2)
    res[cle] = g["taux_%"].mean()
    print(g["taux_%"].to_dict(), "| moyenne simple :", round(g["taux_%"].mean(), 2), "| global :",
          ↪ round(100 * lig["retournee"].mean(), 2))
```

```
{'Boutique': 3.08, 'Réseaux': 6.67, 'Site': 8.98} | moyenne simple : 6.24 | global : 5.96
{'Bien-être': 6.08, 'Cuisine': 6.15, 'Décoration': 6.04, 'Jardin': 6.16, 'Maison': 5.86, 'Papeterie':
↪ 5.43} | moyenne simple : 5.95 | global : 5.96
```

Étape 2 — Moyenne des paniers moyens mensuels. Faites la moyenne des 36 paniers moyens mensuels, puis comparez au panier moyen global.

```
mens = cmd.groupby(["annee", "mois"])["panier"].agg(["mean", "size"])
print("moyenne des moyennes mensuelles :", round(mens["mean"].mean(), 2), "| moyenne pondérée :",
      ↪ round((mens["mean"] * mens["size"]).sum() / mens["size"].sum(), 2), "| global :",
      ↪ round(panier.mean(), 2))
```

```
moyenne des moyennes mensuelles : 100.99 | moyenne pondérée : 100.38 | global : 100.38
```

Étape 3 — Fabriquer un paradoxe de Simpson. Dans le fichier réel, on ne trouve pas de renversement. On le fabrique : deux offres P et Q, deux groupes de clients, des envois répartis différemment.

```
env = pd.DataFrame({"offre": ["P", "P", "Q", "Q"], "groupe": ["facile", "difficile"] * 2, "envois":
↪ [300, 100, 100, 300], "achats": [180, 20, 65, 75]})
env["taux"] = env["achats"] / env["envois"]
print(env)
tot = env.groupby("offre")["envois", "achats"].sum(); tot["taux"] = tot["achats"] / tot["envois"]
print(tot)
```

	offre	groupe	envois	achats	taux
0	P	facile	300	180	0.60
1	P	difficile	100	20	0.20
2	Q	facile	100	65	0.65
3	Q	difficile	300	75	0.25
		envois	achats	taux	
	offre				
	P	400	200	0.50	
	Q	400	140	0.35	

Lecture. Sur les retours par catégorie, la moyenne simple (5,95 %) et le taux global (5,96 %) sont voisins parce que les catégories ont des poids et des taux proches ; par canal, ils diffèrent (6,24 % contre 5,96 %), parce que les canaux ont des poids très inégaux et des taux très différents. La moyenne simple des paniers mensuels s'écarte légèrement de la moyenne pondérée, qui retrouve exactement le panier moyen global. Enfin, l'offre Q convertit mieux dans les **deux** groupes (65 % contre 60 %, 25 % contre 20 %) mais moins bien **au total** (35 % contre 50 %), parce qu'elle a été envoyée surtout au groupe difficile.

À vous. Choisissez d'autres effectifs pour qu'un renversement apparaisse avec **trois** groupes. *Piste : un renversement exige que la répartition des envois entre groupes diffère fortement d'une offre à l'autre,*

et que les groupes aient des taux de base très différents.

Application 1.3 — Excel contre pandas (section 1.1.5)

Objectif. Faire calculer les mêmes résumés par un classeur Excel (recalculé par LibreOffice Calc) et par pandas, et comprendre les écarts de convention.

Étape 1 — Écrire un classeur avec des formules. On y met les paniers de 2025, une formule par statistique (noms anglais du fichier ; Excel en français les affiche traduits).

```
import tempfile, os
X = 0.X
p25 = cmd[cmd["annee"] == 2025]["panier"].round(2)
n = len(p25)
formules = {"moyenne": f"AVERAGE(A2:A{n+1})", "médiane": f"MEDIAN(A2:A{n+1})", "Q1":
    ↪ f"QUARTILE.INC(A2:A{n+1},1)",
            "Q3": f"QUARTILE.INC(A2:A{n+1},3)", "écart-type": f"STDEV.S(A2:A{n+1})", "asymétrie":
    ↪ f"SKEW(A2:A{n+1})", "aplatissement": f"KURT(A2:A{n+1})"}
dossier = tempfile.mkdtemp(dir=os.environ.get("TMPDIR"))
lignes_xl = [["panier", None, "statistique", "formule"]] + [[float(v)] for v in p25]
for i, (k, f) in enumerate(formules.items()):
    lignes_xl[i + 1] += [None, k, f]
chemin = X.classeur(os.path.join(dossier, "paniers2025.xlsx"), {"Feuil1": lignes_xl})
```

Étape 2 — Faire recalculer et comparer. LibreOffice ouvre le classeur, recalculé toutes les formules, et l'on lit les résultats.

```
val = X.valeurs(X.recalculer(chemin, dossier))
excel = {k: val[i + 1][3] for i, k in enumerate(formules)}
pandas_ = {"moyenne": p25.mean(), "médiane": p25.median(), "Q1": p25.quantile(.25), "Q3":
    ↪ p25.quantile(.75),
            "écart-type": p25.std(), "asymétrie": p25.skew(), "aplatissement": p25.kurt()}
cmp_ = pd.DataFrame({"Excel (LibreOffice)": excel, "pandas": pandas_})
cmp_["écart"] = (cmp_["Excel (LibreOffice)"] - cmp_["pandas"]).abs()
print(cmp_.round(4))
```

	Excel (LibreOffice)	pandas	écart
moyenne	102.3300	102.3300	0.0
médiane	81.6150	81.6150	0.0
Q1	44.0900	44.0900	0.0
Q3	137.7200	137.7200	0.0
écart-type	82.3264	82.3264	0.0
asymétrie	1.9478	1.9478	0.0
aplatissement	7.0136	7.0136	0.0

Étape 3 — Les conventions qui diffèrent. QUARTILE.EXC et STDEV.P ne donnent pas les mêmes valeurs que QUARTILE.INC et STDEV.S. Voyons-le sur nos neuf commandes.

```
x9 = np.array([9.69, 207.56, 132.37, 258.83, 66.75, 22.56, 50.27, 48.74, 128.25])
f9 = ["=QUARTILE.INC(A1:A9,1)", "=QUARTILE.EXC(A1:A9,1)", "=STDEV.S(A1:A9)", "=STDEV.P(A1:A9)"]
c9 = X.classeur(os.path.join(dossier, "neuf.xlsx"), {"F": [[float(v)] + (f9[0] if i == 0 else []) for
    ↪ i, v in enumerate(x9)]})
v9 = X.valeurs(X.recalculer(c9, dossier))[0][1:5]
print(dict(zip(["Q1 inclusif", "Q1 exclusif", "écart-type n-1", "écart-type n"], [round(float(v), 2)
    ↪ for v in v9])))
import shutil; shutil.rmtree(dossier, ignore_errors=True)
```

{'Q1 inclusif': 48.74, 'Q1 exclusif': 35.65, 'écart-type n-1': 85.83, 'écart-type n': 80.92}

Lecture. Les deux outils concordent à mieux que 10^{-6} (écart maximal $2,8e-13$) pour les sept statistiques : les formules d'Excel et celles de pandas sont les **mêmes**. Les écarts viennent des **conventions** : sur neuf valeurs, le premier quartile « exclusif » (35,65) diffère du quartile « inclusif »

(48,74), et l'écart-type avec n (80,92) est plus petit que celui avec $n - 1$ (85,83).

À vous. Ajoutez une formule TRIMMEAN à 10 % et comparez avec `scipy.stats.trim_mean(p25, 0.05)`. Piste : `TRIMMEAN(...; 0, 1)` retire 10 % **au total**, donc 5 % de chaque côté.

Application 1.4 — Quelle loi pour quoi ? (section 1.2)

Objectif. Confronter trois lois aux données : binomiale (retours), Poisson (commandes par heure), log-normale (paniers).

Étape 1 — Binomiale : les retours en Boutique. Pour 30 lignes vendues en Boutique, combien de retours ? Comparez la loi binomiale à 20 000 tirages.

```
from scipy.stats import binom
p_b = lig[lig["canal"] == "Boutique"]["retournee"].mean()
rng = np.random.default_rng(11)
bout = lig[lig["canal"] == "Boutique"]["retournee"].values
tir = np.array([rng.choice(bout, 30, replace=False).sum() for _ in range(20000)])
print("p :", round(p_b, 4), "| P(≥ 3) théorique :", round(binom.sf(2, 30, p_b), 4), "| observé :",
      ↪ round((tir >= 3).mean(), 4))
```

p : 0.0308 | P(≥ 3) théorique : 0.0638 | observé : 0.0633

Étape 2 — Poisson : les commandes de la Boutique par heure. On compte les commandes de la Boutique par heure d'arrivée, pour les samedis de décembre 2025, et l'on compare la variance et la moyenne.

```
h = cmd[(cmd["canal"] == "Boutique") & (cmd["date_commande"].dt.dayofweek == 5) &
      ↪ (cmd["date_commande"].dt.month == 12) & (cmd["annee"] == 2025)].copy()
h["heure_h"] = h["heure"].str[:2].astype(int)
cpt = h.groupby(["date_commande", "heure_h"]).size().unstack(fill_value=0).stack() # une valeur par
      ↪ (samedi, heure)
cpt = cpt[cpt.index.get_level_values(1).isin(range(11, 19))] # heures pleines
print("moyenne :", round(cpt.mean(), 2), "| variance :", round(cpt.var(), 2), "| variance/moyenne :",
      ↪ round(cpt.var() / cpt.mean(), 2))
```

moyenne : 3.22 | variance : 2.95 | variance/moyenne : 0.92

Étape 3 — Log-normale : les paniers. On ajuste une loi log-normale (moyenne et écart-type du logarithme) et l'on compare la part de paniers supérieurs à 300 € prévue et observée.

```
mu, sg = np.log(panier).mean(), np.log(panier).std()
prevu = 1 - stats.norm.cdf((np.log(300) - mu) / sg)
print("part prévue de paniers > 300 € :", round(prevu * 100, 2), "% | observée :", round((panier >
      ↪ 300).mean() * 100, 2), "%")
```

part prévue de paniers > 300 € : 5.54 % | observée : 2.76 %

Lecture. La binomiale prévoit 6,38 % de lots de 30 lignes avec au moins 3 retours ; on en observe 6,33 %. Pour les arrivées horaires (32 couples samedi-heure), le rapport variance/moyenne vaut 0,92 : proche de 1, mais sur peu de données ; l'écart reste de l'ordre de l'erreur d'estimation. Enfin, la log-normale prévoit 5,5 % de paniers supérieurs à 300 € contre 2,8 % observés : la log-normale **surestime d'un facteur deux** la part des gros paniers : sa queue haute est trop épaisse, comme le montrait le diagramme quantile-quantile du livre (les points se placent sous la droite à droite).

À vous. Refaites l'étape 2 sur **tous** les jours de l'année (pas seulement les samedis de décembre) : que devient le rapport variance/moyenne, et pourquoi ?

Application 1.5 — Simuler un échantillonnage (section 1.3.2)

Objectif. Voir l'erreur type, le théorème central limite et la correction de population finie à l'œuvre.

Étape 1 — L'erreur type selon la taille. Pour $n = 25, 100, 400$, tirez 1 000 échantillons de paniers de 2025 et comparez l'écart-type observé des moyennes à $\sigma/\sqrt{n}$.

```
pop = cmd[cmd["annee"] == 2025]["panier"].values
rng = np.random.default_rng(21)
lignes_t = []
for n in (25, 100, 400):
    mm = [rng.choice(pop, n, replace=False).mean() for _ in range(1000)]
    lignes_t.append((n, round(np.std(mm), 2), round(pop.std() / np.sqrt(n), 2)))
print(pd.DataFrame(lignes_t, columns=["n", "erreur type observée", "σ/√n"]))
```

	n	erreur type observée	σ/√n
0	25	16.39	16.46
1	100	7.84	8.23
2	400	4.11	4.12

Étape 2 — La forme de la distribution des moyennes. Mesurez l'asymétrie des moyennes de n paniers pour $n = 2, 10, 50, 250$.

```
for n in (2, 10, 50, 250):
    mm = np.array([rng.choice(pop, n, replace=False).mean() for _ in range(2000)])
    print(n, "asymétrie des moyennes :", round(stats.skew(mm), 2))
```

```
2 asymétrie des moyennes : 1.38
10 asymétrie des moyennes : 0.65
50 asymétrie des moyennes : 0.36
250 asymétrie des moyennes : 0.12
```

Étape 3 — Population finie. Avec $n = 5\,000$ sur une population de 12 946, comparez l'erreur type observée à $\sigma/\sqrt{n}$ et à la formule corrigée.

```
N, n = len(pop), 5000
mm = [rng.choice(pop, n, replace=False).mean() for _ in range(500)]
print("observée :", round(np.std(mm), 2), "| σ/√n :", round(pop.std() / np.sqrt(n), 2), "| corrigée",
      ":", round(pop.std() / np.sqrt(n) * np.sqrt((N - n) / (N - 1)), 2))
```

```
observée : 0.88 | σ/√n : 1.16 | corrigée : 0.91
```

Étape 4 — La moyenne contre la médiane. Quelle est l'erreur type de la médiane d'un échantillon de 100 paniers, comparée à celle de la moyenne ?

```
med = [np.median(rng.choice(pop, 100, replace=False)) for _ in range(1000)]
moy = [rng.choice(pop, 100, replace=False).mean() for _ in range(1000)]
print("erreur type de la médiane :", round(np.std(med), 2), "| de la moyenne :", round(np.std(moy),
      ":", 2))
```

```
erreur type de la médiane : 8.1 | de la moyenne : 8.0
```

Lecture. L'erreur type observée colle à $\sigma/\sqrt{n}$ pour les trois tailles (étape 1). L'asymétrie des moyennes s'efface avec n : le théorème central limite est à l'œuvre (étape 2). Quand l'échantillon est une grande fraction de la population, l'erreur observée (0,88) est plus proche de la formule corrigée (0,91) que de $\sigma/\sqrt{n}$ (1,16). Enfin, la médiane est à peine **moins précise** que la moyenne pour estimer un centre sur ces données (8,10 contre 8,00) : la robustesse aux valeurs extrêmes coûte ici très peu.

À vous. Répétez l'étape 1 avec la population des **clients** (dépendance annuelle par client) au lieu des

commandes. Piste : la distribution est encore plus asymétrique ; il faut davantage d'observations pour que la moyenne soit proche d'une normale.

Application 1.6 — Intervalles de confiance et biais (sections 1.3.3 à 1.3.6)

Objectif. Mesurer la couverture réelle d'intervalles de confiance, comparer deux intervalles de proportion, dimensionner une enquête, puis corriger un biais de sélection.

Étape 1 — Couverture : t contre 1,96, sur 30 observations. On construit 2 000 intervalles à 95 % avec des échantillons de 30 paniers, avec le coefficient de Student puis avec 1,96.

```
pop = cmd[cmd["annee"] == 2025]["panier"].values
mu = pop.mean()
rng = np.random.default_rng(31)
cov_t = cov_z = 0
for _ in range(2000):
    e = rng.choice(pop, 30, replace=False); m, se = e.mean(), e.std(ddof=1) / np.sqrt(30)
    cov_t += abs(m - mu) <= stats.t.ppf(0.975, 29) * se
    cov_z += abs(m - mu) <= 1.96 * se
print("couverture avec t :", cov_t / 2000, "| avec 1,96 :", cov_z / 2000)
```

couverture avec t : 0.925 | avec 1,96 : 0.915

Étape 2 — Wald contre Wilson pour un petit taux. Le taux de retour de la Boutique est d'environ 3 %. Avec des échantillons de 50 lignes, quelle est la couverture des deux intervalles ?

```
bout = lig[lig["canal"] == "Boutique"]["retournee"].values
p_v = bout.mean(); cw = cwi = 0
for _ in range(2000):
    k = rng.choice(bout, 50, replace=False).sum()
    w, wi = 0.ic_proportion(k, 50), 0.ic_proportion(k, 50, methode="wilson")
    cw += w[0] <= p_v <= w[1]; cwi += wi[0] <= p_v <= wi[1]
print("couverture de Wald :", cw / 2000, "| de Wilson :", cwi / 2000)
```

couverture de Wald : 0.785 | de Wilson : 0.931

Étape 3 — Dimensionner une enquête. Combien de lignes du Site faut-il pour connaître le taux de retour à ± 1 point ? Et à $\pm 0,5$ point ?

```
p_s = lig[lig["canal"] == "Site"]["retournee"].mean()
for e in (0.01, 0.005):
    print("marge ±", e * 100, "points :", int(np.ceil(1.96 ** 2 * p_s * (1 - p_s) / e ** 2)), "lignes")
```

marge ± 1.0 points : 3141 lignes
marge ± 0.5 points : 12563 lignes

Étape 4 — Corriger un biais de sélection. On tire 300 commandes (méthode « croisés à la caisse ») et l'on compte les commandes de leur client ; on corrige ensuite en pondérant chaque observation par l'inverse de la probabilité de l'avoir tirée, proportionnelle au nombre de commandes du client.

```
cpt = cmd.groupby("id_client").size() # commandes par client (clients ayant
↳ commandé)
ech = rng.choice(cmd["id_client"].values, 300) # on croise des commandes
brut = cpt.loc[ech].mean()
corrige = len(ech) / (1 / cpt.loc[ech]).sum() # moyenne pondérée par 1/n_i : moyenne
↳ harmonique
print("brut :", round(brut, 2), "| corrigé :", round(corrige, 2), "| vérité (clients ayant commandé)
↳ :", round(cpt.mean(), 2))
```

```
brut : 16.33 | corrigé : 7.61 | vérité (clients ayant commandé) : 7.57
```

Lecture. Avec 30 observations, l'intervalle de Student couvre 92,5 % des fois, contre 91,5 % pour 1,96 : le coefficient de Student corrige, un peu, le petit échantillon (et l'asymétrie des paniers empêche d'atteindre 95 %). Pour un taux de 3 % mesuré sur 50 lignes, l'intervalle de Wald couvre 78,5 % des fois seulement, celui de Wilson 93,1 % : Wilson est nettement meilleur. Pour connaître le taux de retour du Site à ± 1 point, il faut 3 141 lignes, et 12 563 à $\pm 0,5$ point (quatre fois plus). Enfin, la pondération par l'inverse du nombre de commandes ramène l'estimation à 7,6, tout près de la vérité des clients ayant commandé (7,6) alors que l'estimation brute valait 16,3 ; mais elle ne peut rien pour les clients **qui n'ont jamais commandé** : une fois exclus du tirage, ils ne reviennent pas.

À vous. Refaites l'étape 4 avec 3 000 commandes : le biais brut change-t-il ? *Piste : non, c'est l'intérêt de l'exemple : un biais de sélection ne diminue pas avec la taille de l'échantillon.*

Application 1.7 — Corrélation, confusion et saison (section 1.4)

Objectif. Refaire l'enquête sur la publicité et les ventes : corrélation brute, corrélation à mois égal, effet ajusté, comparaison avec la vérité programmée.

Étape 1 — La corrélation brute. Pearson et Spearman entre la dépense publicitaire du jour et le chiffre d'affaires du jour.

```
j = jours.copy()
print("Pearson :", round(j["depense_pub"].corr(j["chiffre_affaires"]), 3), "| Spearman :",
      ↪ round(j["depense_pub"].corr(j["chiffre_affaires"], method="spearman"), 3))
```

```
Pearson : 0.415 | Spearman : 0.286
```

Étape 2 — À mois égal. On retire la moyenne de chaque mois, puis on corrèle les écarts.

```
for c in ("depense_pub", "chiffre_affaires"):
    j[c + "_ecart"] = j[c] - j.groupby("mois")[c].transform("mean")
print("corrélation à mois égal :", round(j["depense_pub_ecart"].corr(j["chiffre_affaires_ecart"]),
      ↪ 3))
```

```
corrélation à mois égal : 0.049
```

Étape 3 — Effet ajusté. Une régression du logarithme du nombre de commandes sur la dépense hebdomadaire, avec ou sans neutralisation du calendrier.

```
import statsmodels.formula.api as smf
j["pub7k"] = j["depense_pub"].rolling(7, min_periods=1).sum() / 1000
j["pluvieux"] = (j["pluie_mm"] > 1).astype(int); j["t"] = np.arange(len(j)) / 365.25; j["lc"] =
    ↪ np.log(j["nb_commandes"])
m0 = smf.ols("lc ~ pub7k", data=j).fit()
m1 = smf.ols("lc ~ pub7k + C(mois) + C(jour_sem) + promo_active + t + pluvieux", data=j).fit()
for nom, m in (("naïf", m0), ("ajusté", m1)):
    lo, hi = m.conf_int().loc["pub7k"]
    print(nom, ": +", round((np.exp(m.params["pub7k"]) - 1) * 100, 1), "% (", round((np.exp(lo) - 1) *
    ↪ 100, 1), "% à", round((np.exp(hi) - 1) * 100, 1), "%)")
print("promotion : +", round((np.exp(m1.params["promo_active"]) - 1) * 100, 1), "% | tendance
    ↪ annuelle : +", round((np.exp(m1.params["t"]) - 1) * 100, 1), "%")
```

```
naïf : + 31.9 % ( 28.2 à 35.7 )
ajusté : + 0.7 % ( -3.7 à 5.4 )
promotion : + 19.2 % | tendance annuelle : + 6.5 %
```

Étape 4 — Des corrélations fortuites. Avec 50 variables aléatoires de 30 observations, combien

de paires dépassent 0,36 en valeur absolue ?

```
rng = np.random.default_rng(5)
rr = np.corrcoef(rng.normal(size=(30, 50)).T)[np.triu_indices(50, 1)]
print("paires :", len(rr), "| |r| > 0,36 :", int((abs(rr) > 0.36).sum()), "| attendu sous
↳ l'indépendance : environ", round(0.05 * len(rr)))
```

```
paires : 1225 | |r| > 0,36 : 55 | attendu sous l'indépendance : environ 61
```

Lecture. La corrélation brute (0,42) tombe à 0,05 une fois comparés des jours du **même mois**. L'effet « naïf » de 1 000 € de dépense hebdomadaire est de +32 % des commandes, l'effet ajusté de +0,7 % (vérité programmée : +1,5 %, mais l'intervalle est trop large pour conclure). L'effet de la promotion, lui, est retrouvé : +19 % (vérité : +18 %). Enfin, sur 1 225 paires de variables sans lien, 55 dépassent 0,36.

À vous. Estimez l'effet de la **pluie** sur les commandes du canal Boutique seul (comptez les commandes de la Boutique par jour). *Piste : la vérité programmée est -8 % pour la Boutique et +5 % pour le Site ; sur le total les deux effets se compensent presque.*

Application 1.8 — Mathématiques du quotidien (section 1.5)

Objectif. Calculer des croissances, décomposer prix et volume par catégorie, mesurer les marges, et chiffrer l'effet d'une règle d'arrondi.

Étape 1 — Croissance par canal. Chiffre d'affaires annuel de chaque canal, taux annuel et TCAM 2023-2025.

```
ca = lig.groupby(["canal", "annee"])["montant"].sum().unstack()
tab = pd.DataFrame({"2024/2023 %": (ca[2024] / ca[2023] - 1) * 100, "2025/2024 %": (ca[2025] /
↳ ca[2024] - 1) * 100,
                    "TCAM %": ((ca[2025] / ca[2023]) ** 0.5 - 1) * 100}).round(2)
print(tab)
```

	2024/2023 %	2025/2024 %	TCAM %
canal			
Boutique	-5.98	0.51	-2.79
Réseaux	4.81	13.42	9.03
Site	18.96	22.92	20.92

Étape 2 — Prix et volume par catégorie (2025 contre 2024). Pour chaque catégorie : croissance des unités, du chiffre d'affaires moyen par unité, et du chiffre d'affaires.

```
g = lig[lig["annee"].isin([2024, 2025]).groupby(["categorie", "annee"]).agg(ca=("montant", "sum"),
↳ u=("quantite", "sum")).unstack()
dec = pd.DataFrame({"volume %": (g["u"][2025] / g["u"][2024] - 1) * 100,
                    "prix-mix %": ((g["ca"][2025] / g["u"][2025]) / (g["ca"][2024] / g["u"][2024]) -
↳ 1) * 100,
                    "CA %": (g["ca"][2025] / g["ca"][2024] - 1) * 100}).round(1)
print(dec)
```

	volume %	prix-mix %	CA %
categorie			
Bien-être	6.2	3.1	9.5
Cuisine	4.2	3.1	7.4
Décoration	7.3	2.5	10.1
Jardin	9.7	4.2	14.3
Maison	10.1	3.0	13.4
Papeterie	6.6	3.0	9.8

Étape 3 — Marque par canal. Taux de marque 2025 (CA HT = montant / 1,2) de chaque canal : moyenne pondérée et moyenne simple des catégories.

```
l25 = lig[lig["annee"] == 2025].assign(ca_ht=lambda t: t["montant"] / 1.2, cout=lambda t:
↳ t["quantite"] * t["cout_achat"])
par = l25.groupby("canal")[["ca_ht", "cout"]].sum()
print(((par["ca_ht"] - par["cout"]) / par["ca_ht"] * 100).round(2).to_dict())
```

```
{'Boutique': 38.0, 'Réseaux': 38.11, 'Site': 37.88}
```

Étape 4 — L'effet d'une règle d'arrondi. La TVA incluse dans chaque ligne de 2025 est montant – montant/1,2. Comparez la somme des TVA **arrondies ligne par ligne** à la TVA calculée sur le total.

```
l25["tva"] = l25["montant"] - l25["montant"] / 1.2
ligne = l25["tva"].round(2).sum() # arrondi (au pair) ligne par ligne
total = round(l25["tva"].sum(), 2)
print("TVA arrondie ligne par ligne :", round(ligne, 2), "| TVA sur le total :", total, "| écart :",
↳ round(ligne - total, 2), "€ sur", len(l25), "lignes")
```

```
TVA arrondie ligne par ligne : 220785.54 | TVA sur le total : 220793.95 | écart : -8.41 € sur 29827
↳ lignes
```

Lecture. Le Site croît de 20,9 % par an, la Boutique de -2,8 % (elle recule) : le déplacement vers le Site se lit dans les TCAM. Par catégorie, la croissance en volume va de 4,2 % à 10,1 % et l'effet prix-mix de 2,5 % à 4,2 % : la hausse de 3 % du catalogue est partout, mais le mix et les remises la modulent. Les taux de marque des canaux ne diffèrent presque pas (37,9 à 38,1 %). Enfin, la règle d'arrondi ligne par ligne produit un écart de -8,41 € sur 29 827 lignes : faible en valeur relative, mais **à documenter**.

À vous. Refaites l'étape 4 en arrondissant **par commande** (somme des montants de chaque commande, puis TVA arrondie). *Piste : trois règles, trois totaux ; aucune n'est « fausse », mais il faut en choisir une et la noter.*

Exercices

Exercice 1.1 ★ — Moyenne et médiane à la main (section 1.1.1)

Sept paniers (en €) : 12 ; 45 ; 38 ; 60 ; 22 ; 51 ; 340. Calculez la moyenne et la médiane. Que deviennent-elles si l'on retire 340 ? Quel résumé annonceriez-vous pour décrire une commande « typique » ?

Exercice 1.2 ★ — Écart-type d'un petit échantillon (section 1.1.2)

Cinq délais de livraison (jours) : 2 ; 3 ; 3 ; 4 ; 8. Calculez la moyenne, la variance (avec $n - 1$), l'écart-type et le coefficient de variation.

Exercice 1.3 ★★ — La règle de 1,5 écart interquartile (section 1.1.3)

Onze paniers (en €) : 18 ; 22 ; 25 ; 27 ; 30 ; 31 ; 35 ; 38 ; 42 ; 47 ; 190. (a) Calculez Q_1 , Q_3 et l'EIQ (méthode inclusive). (b) Quelles valeurs la règle de Tukey signale-t-elle ? (c) Que feriez-vous de cette valeur avant de calculer la moyenne ?

Exercice 1.4 ★ — Moyenne des moyennes (section 1.1.4)

Une enquête de satisfaction a eu trois groupes de répondants : 400 clients de la Boutique (85 % de satisfaits), 100 clients des Réseaux (60 %), 500 clients du Site (70 %). Calculez le taux de satisfaits de l'ensemble, puis la moyenne simple des trois taux. Expliquez l'écart.

Exercice 1.5 ★★ — Un renversement de Simpson (section 1.1.4)

Deux offres sont envoyées à deux groupes de clients. Offre P : 300 envois au groupe « facile » (180 achats) et 100 au groupe « difficile » (20 achats). Offre Q : 100 envois au groupe facile (65 achats) et 300 au groupe difficile (75 achats). Calculez les taux par groupe puis au total. Quelle offre est la meilleure ? Quel conseil donneriez-vous ?

Exercice 1.6 ★ — Score z et normale (section 1.2.3)

L'âge des clients a une moyenne de 43,3 ans et un écart-type de 13,7 ans. Calculez le score z d'une cliente de 25 ans et d'un client de 65 ans, puis la part approximative des clients entre 30 et 56 ans si l'âge était exactement normal.

Exercice 1.7 ★★ — Binomiale et Poisson à la main (section 1.2.2)

- (a) En Boutique, le taux de retour est de 3 %. Sur un lot de 30 lignes, quelle est la probabilité qu'aucune ne soit retournée ? Qu'au moins deux le soient ? (b) La Boutique reçoit en moyenne 4 commandes par heure creuse. Quelle est la probabilité de n'en recevoir **aucune** pendant une heure ? D'en recevoir 8 ou plus ?

Exercice 1.8 ★ — Erreur type (section 1.3.2)

L'écart-type des paniers est de 80 €. Quelle est l'erreur type de la moyenne pour $n = 64$? Pour $n = 256$? Quelle taille d'échantillon donne une erreur type de 2,5 € ?

Exercice 1.9 ★★ — Intervalle de confiance d'une proportion (section 1.3.4)

Sur 400 avis, 120 déclarent être « très satisfaits ». Donnez le taux observé et l'intervalle de confiance à 95 % de Wald. Pouvez-vous affirmer que le vrai taux dépasse 25 % ? Que dit l'intervalle de Wilson ?

Exercice 1.10 ★★ — Taille d'échantillon (section 1.3.5)

- (a) Combien de paniers faut-il pour estimer le panier moyen à ± 4 € près, avec un écart-type supposé de 80 € ? (b) Combien de répondants pour estimer une proportion proche de 30 % à ± 3 points ? (c) Si la population ne compte que 2 000 clients, de combien cet effectif se réduit-il (formule corrigée $n' = n / (1 + (n - 1) / N)$) ?

Exercice 1.11 ★ — Corrélation à la main (section 1.4.1)

Cinq jours : dépense publicitaire (centaines d'euros) $x = 1; 2; 3; 4; 5$ et nombre de commandes $y = 2; 4; 5; 4; 5$. Calculez r à la main (tableau des écarts). Que dire de r^2 ?

Exercice 1.12 ★★★ — Concevoir une expérience (section 1.4.6)

La gérante constate que les clients qui **ouvrent** les e-mails de la boutique achètent plus que ceux qui ne les ouvrent pas, et conclut : « les e-mails font vendre ». (a) Donnez trois explications **non causales** de cette corrélation. (b) Décrivez une expérience aléatoire qui permettrait de trancher. (c) Si le taux d'achat de référence est de 10 % et que vous voulez détecter une hausse de 2 points, la formule indicative $n = 2(1,96 + 0,84)^2 \bar{p}(1 - \bar{p})/\delta^2$ (avec $\bar{p}$ proche de 0,11 et $\delta = 0,02$) donne combien de clients **par groupe** ?

Exercice 1.13 ★ — Pourcentages en cascade (section 1.5.1)

- (a) Un prix de 80 € augmente de 25 %, puis baisse de 20 %. Quel est le prix final, quelle est la variation totale ? (b) Un prix TTC est de 96 € avec une TVA de 20 % : quel est le prix HT ? (c) La part du Site passe de 35 % à 45 % des commandes : exprimez l'évolution en points et en pourcentage.

Exercice 1.14 ★★ — Marge, marque, TVA (section 1.5.4)

Un produit est acheté 18 € HT. (a) On veut un taux de **marque** de 35 % : quel prix HT, quel prix TTC (TVA à 20 %) ? (b) Quel est alors le taux de **marge** ? (c) Si l'on avait appliqué à tort 35 % au coût d'achat, quel taux de marque aurait-on obtenu ?

Corrigés**Corrigé 1.1**

Total = 568, donc moyenne = $568/7 \approx 81,1$ €. Rangés : 12 ; 22 ; 38 ; 45 ; 51 ; 60 ; 340, la médiane est la quatrième valeur : **45 €**. Sans 340 : moyenne $228/6 = 38$ €, médiane $(38 + 45)/2 = 41,5$ €. La moyenne chute de 81,1 à 38 €, la médiane de 45 à 41,5 € : la médiane est robuste. Pour une commande « typique », on annonce la **médiane** (45 €), en précisant qu'une commande atypique de 340 € tire la moyenne.

```
v = np.array([12, 45, 38, 60, 22, 51, 340]); w = np.delete(v, 6)
print(round(v.mean(), 1), np.median(v), round(w.mean(), 1), np.median(w))
```

```
81.1 45.0 38.0 41.5
```

Corrigé 1.2

Moyenne = $20/5 = 4$ jours. Écarts : -2 ; -1 ; -1 ; 0 ; 4, carrés : 4 ; 1 ; 1 ; 0 ; 16, somme 22. Variance = $22/4 = 5,5$, écart-type $\sqrt{5,5} \approx 2,35$ jours, coefficient de variation $2,35/4 \approx 0,59$. La valeur 8 contribue à elle seule à $16/22$ de la somme : un seul retard pèse lourd dans la dispersion.

```
v = np.array([2, 3, 3, 4, 8]); print(v.mean(), v.var(ddof=1), round(v.std(ddof=1), 3),
↳ round(v.std(ddof=1) / v.mean(), 2))
```

```
4.0 5.5 2.345 0.59
```

Corrigé 1.3

- (a) Avec $n = 11$, Q_1 est à la position $1 + 0,25 \times 10 = 3,5$: entre 25 et 27, soit **26** ; Q_3 à la position 8,5 : entre 38 et 42, soit **40**. $EQ = 14$. (b) Bornes : $26 - 21 = 5$ et $40 + 21 = 61$. La valeur **190** dépasse 61 : elle est signalée ; les autres sont dans l'intervalle. (c) On vérifie d'abord si c'est une erreur (virgule décalée : 19,0 ?), un client professionnel, ou une grosse commande légitime ; on **ne la supprime pas** sans raison, on annonce la médiane en plus de la moyenne, ou l'on calcule la moyenne avec et sans cette valeur.

```
v = np.array([18, 22, 25, 27, 30, 31, 35, 38, 42, 47, 190]); q1, q3 = np.quantile(v, [.25, .75]);
↪ print(q1, q3, q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1))
```

```
26.0 40.0 5.0 61.0
```

Corrigé 1.4

Satisfaits : $0,85 \times 400 + 0,60 \times 100 + 0,70 \times 500 = 340 + 60 + 350 = 750$ sur 1 000, soit **75 %**. Moyenne simple : $(85 + 60 + 70)/3 \approx 71,7 \%$. L'écart (3,3 points) vient de ce que la moyenne simple donne au groupe de 100 clients (le plus faible taux) le même poids qu'aux autres. La bonne moyenne est la moyenne **pondérée par les effectifs**.

```
print((0.85 * 400 + 0.6 * 100 + 0.7 * 500) / 1000, np.mean([0.85, 0.6, 0.7]))
```

```
0.75 0.7166666666666667
```

Corrigé 1.5

Offre P : groupe facile $180/300 = 60 \%$, difficile $20/100 = 20 \%$, total $200/400 = 50 \%$. Offre Q : facile $65/100 = 65 \%$, difficile $75/300 = 25 \%$, total $140/400 = 35 \%$. Q est meilleure **dans chaque groupe** ($65 > 60$ et $25 > 20$) mais P l'est **au total** ($50 > 35$) : c'est le paradoxe de Simpson, dû à la composition différente des envois (P a visé surtout le groupe facile, Q surtout le groupe difficile). Conseil : comparer les offres **à groupe égal** (ou envoyer les deux offres, au hasard, à la même population), et conclure que Q est la meilleure offre.

```
print(180 / 300, 20 / 100, 200 / 400, 65 / 100, 75 / 300, 140 / 400)
```

```
0.6 0.2 0.5 0.65 0.25 0.35
```

Corrigé 1.6

$z_{25} = (25 - 43,3)/13,7 \approx -1,34$; $z_{65} = (65 - 43,3)/13,7 \approx +1,58$. Entre 30 et 56 ans : z de $-0,97$ à $+0,93$; la part vaut $\Phi(0,93) - \Phi(-0,97) \approx 0,824 - 0,166 = 0,658$, soit environ **66 %** (proche des 68 % de la règle à ± 1 écart-type, puisque 30 et 56 ans sont à peu près à ± 1 écart-type).

```
print(round((25 - 43.3) / 13.7, 2), round((65 - 43.3) / 13.7, 2), round(stats.norm.cdf((56 - 43.3) /
↪ 13.7) - stats.norm.cdf((30 - 43.3) / 13.7), 3))
```

```
-1.34 1.58 0.657
```

Corrigé 1.7

- (a) $P(K = 0) = 0,97^{30} \approx 0,401$. $P(K \geq 2) = 1 - P(0) - P(1)$, avec $P(1) = 30 \times 0,03 \times 0,97^{29} \approx 0,372$: donc $1 - 0,401 - 0,372 \approx 0,227$. (b) $P(N = 0) = e^{-4} \approx 0,0183$; $P(N \geq 8) = 1 - P(N \leq 7) \approx 0,051$ (le calcul exige la somme des huit premiers termes ; voir le code).

```
from scipy.stats import binom, poisson
print(round(binom.pmf(0, 30, 0.03), 3), round(binom.sf(1, 30, 0.03), 3), round(poisson.pmf(0, 4), 4),
      ↪ round(poisson.sf(7, 4), 3))
```

```
0.401 0.227 0.0183 0.051
```

Corrigé 1.8

Erreur type = $\sigma/\sqrt{n}$: $80/8 = 10$ € pour $n = 64$, $80/16 = 5$ € pour $n = 256$ (quadrupler n divise par 2). Pour 2,5 € : $\sqrt{n} = 80/2,5 = 32$, donc $n = 1024$.

```
print(80 / np.sqrt(64), 80 / np.sqrt(256), (80 / 2.5) ** 2)
```

```
10.0 5.0 1024.0
```

Corrigé 1.9

$\hat{p} = 120/400 = 30\%$. Erreur type $\sqrt{0,3 \times 0,7/400} \approx 0,0229$; marge $1,96 \times 0,0229 \approx 0,045$; intervalle de Wald [25,5 % ; 34,5 %]. Il exclut (de peu) 25 % : on peut dire que le taux dépasse 25 % avec une confiance de 95 %, mais « de peu ». L'intervalle de Wilson, [25,7 % ; 34,7 %], est un peu décalé vers le centre et conduit à la même conclusion.

```
print(np.round(0.ic_proportion(120, 400), 4), np.round(0.ic_proportion(120, 400, methode="wilson"),
      ↪ 4))
```

```
[0.2551 0.3449] [0.2572 0.3466]
```

Corrigé 1.10

- (a) $n = (1,96 \times 80/4)^2 = 39,2^2 \approx 1536,6$, soit **1 537** paniers (on arrondit à l'entier supérieur). (b) $n = 1,96^2 \times 0,3 \times 0,7/0,03^2 \approx 896,4$, soit **897** répondants. (c) $n' = 897/(1 + 896/2000) = 897/1,448 \approx 619$: la population finie réduit l'effectif nécessaire d'environ 30 %.

```
print(int(np.ceil((1.96 * 80 / 4) ** 2)), int(np.ceil(1.96 ** 2 * 0.21 / 0.03 ** 2)), round(897 / (1 +
      ↪ 896 / 2000)))
```

```
1537 897 619
```

Corrigé 1.11

$\bar{x} = 3$, $\bar{y} = 4$. Écarts x : -2 ; -1 ; 0 ; 1 ; 2 ; écarts y : -2 ; 0 ; 1 ; 0 ; 1. Produits : 4 ; 0 ; 0 ; 0 ; 2, somme 6. $\sum (x - \bar{x})^2 = 10$, $\sum (y - \bar{y})^2 = 6$. $r = 6/\sqrt{10 \times 6} = 6/7,746 \approx 0,775$ et $r^2 \approx 0,6$: environ 60 % de la variation des commandes est « partagée » avec celle de la dépense le long d'une droite. Avec **cinq** points, ce chiffre est très instable.

```
x = np.array([1, 2, 3, 4, 5]); y = np.array([2, 4, 5, 4, 5]); print(round(np.corrcoef(x, y)[0, 1], 3),
      ↪ round(np.corrcoef(x, y)[0, 1] ** 2, 2))
```

0.775 0.6

Corrigé 1.12

- (a) Explications non causales : (1) les clients qui ouvrent les e-mails sont déjà **plus engagés** (ils aiment la boutique, ils achetaient déjà beaucoup) : confusion ; (2) un client qui vient d'acheter **ouvre** davantage les e-mails de suivi (causalité inverse) ; (3) ouvrir est plus fréquent chez les clients avec carte et adresse valide, qui achètent plus (confusion par le profil).
 (b) Tirer **au hasard** la moitié des clients (parmi ceux qui ont consenti) qui recevront l'e-mail, l'autre moitié ne le recevant pas, et comparer le taux d'achat sur une période fixée à l'avance.
 (c) $n = 2 \times 2,8^2 \times 0,11 \times 0,89 / 0,02^2 \approx 3\,838$ clients par groupe.

```
print(round(2 * (1.96 + 0.84) ** 2 * 0.11 * 0.89 / 0.02 ** 2))
```

3838

Corrigé 1.13

- (a) $80 \times 1,25 = 100$ puis $100 \times 0,80 = 80$: le prix final est **80 €**, la variation totale **0 %** (+25 % puis -20 % se compensent exactement : $1,25 \times 0,8 = 1$). (b) Prix HT = $96 / 1,2 = 80$ €. (c) En **points** : +10 points. En **pourcentage** : $(45 - 35) / 35 \approx +28,6$ %.

```
print(80 * 1.25 * 0.8, 96 / 1.2, 45 - 35, round((45 / 35 - 1) * 100, 1))
```

80.0 80.0 10 28.6

Corrigé 1.14

- (a) Prix HT = $18 / (1 - 0,35) \approx 27,69$ €, prix TTC = $27,69 \times 1,2 \approx 33,23$ €. (b) Marge = $27,69 - 18 = 9,69$ €, taux de marge = $9,69 / 18 \approx 53,8$ % (on retrouve $0,35 / 0,65 = 0,538$). (c) Avec +35 % sur le coût, prix HT = $18 \times 1,35 = 24,30$ €, marge 6,30 €, taux de marque $6,30 / 24,30 \approx 25,9$ %, bien moins que les 35 % voulus.

```
ht = 18 / (1 - 0.35); print(round(ht, 2), round(ht * 1.2, 2), round((ht - 18) / 18 * 100, 1),  
↪ round((18 * 1.35 - 18) / (18 * 1.35) * 100, 1))
```

27.69 33.23 53.8 25.9

Chapitre 2 : Excel — exercices et applications

Ce chapitre du cahier accompagne le chapitre 2 du livre. Il contient **huit applications guidées** (de petites études sur le classeur de la gérante, à refaire pas à pas) et **douze exercices** de difficulté croissante (★ à la main, ★★ avec un peu de code, ★★★ à construire), tous **corrigés** à la fin. Rappel : Excel n'est pas installé sur la machine qui produit le livre ; **toutes les formules sont calculées avec LibreOffice** par 0.évaluer (formules écrites avec les **noms anglais** du fichier .xlsx), puis **recoupées avec pandas**. Pour les voir comme dans votre Excel, X.en_fr(formule) les convertit en français. Les tableaux croisés dynamiques, Power Query, DAX, VBA, Google Sheets et Looker Studio ne sont pas exécutables ici : nous recalculons les résultats attendus en pandas ou en SQL.

```
import os, sys, sqlite3, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import outils_xl as X
import outils_ch02 as O

L, P, C = O.charger_2025()                                # lignes de 2025, catalogue, clients (identiques aux
↳ feuilles de ventes_2025.xlsx)
n = len(L) + 1                                             # dernière ligne de données de la feuille Lignes
R = lambda c: f"Lignes!{c}2:{c}{n}"                      # plage d'une colonne de la feuille Lignes
print(len(L), "lignes de ventes en 2025 ;", len(P), "produits ;", len(C), "clients")
```

29827 lignes de ventes en 2025 ; 120 produits ; 6000 clients

Applications

Application 2.1 — Explorer et contrôler le classeur (introduction, section 2.4)

Objectif. Ouvrir un classeur inconnu comme le ferait un analyste : formes, types, valeurs manquantes, contrôles de base, recoupement avec la base de données.

Étape 1 — ce que contient le fichier. On lit les trois feuilles avec pandas et l'on regarde les types.

```
x = pd.read_excel("donnees/ventes_2025.xlsx", sheet_name=None)
print({k: v.shape for k, v in x.items()})
print(dict(x["Lignes"].dtypes.astype(str)))
print("valeurs manquantes :", x["Lignes"].isna().sum()[lambda s: s > 0].to_dict())
```

```
{'Lignes': (29827, 13), 'Produits': (120, 6), 'Clients': (6000, 5)}
{'id_ligne': 'int64', 'id_commande': 'int64', 'date_commande': 'datetime64[us]', 'id_client':
↳ 'int64', 'canal': 'str', 'code_promo': 'str', 'id_produit': 'int64', 'nom_produit': 'str',
↳ 'categorie': 'str', 'quantite': 'int64', 'prix_unitaire': 'float64', 'remise_pct': 'int64',
↳ 'montant': 'float64'}
valeurs manquantes : {'code_promo': 25221}
```

Étape 2 — des contrôles de cohérence. Les identifiants sont-ils uniques ? Les montants positifs ? La période complète ? Le montant d'une ligne est-il bien $\text{prix} \times \text{quantité} \times (1 - \text{remise})$?

```
lg = x["Lignes"]
print("identifiants de ligne uniques :", lg["id_ligne"].is_unique, "| montants ≤ 0 :",
↳ int((lg["montant"] <= 0).sum()))
print("période :", lg["date_commande"].min().date(), "→", lg["date_commande"].max().date())
ecart = (lg["quantite"] * lg["prix_unitaire"] * (1 - lg["remise_pct"] / 100) - lg["montant"]).abs()
print("écart maximal sur une ligne :", round(float(ecart.max()), 4), "€ (arrondi au centime) | lignes
↳ avec un écart > 1 centime :", int((ecart > 0.0101).sum()))
```

```
identifiants de ligne uniques : True | montants ≤ 0 : 0
période : 2025-01-01 → 2025-12-31
écart maximal sur une ligne : 0.005 € (arrondi au centime) | lignes avec un écart > 1 centime : 0
```

Étape 3 — recoupement avec la base. Le classeur est un extrait de la base boutique.db ; leurs totaux doivent être identiques.

```
con = sqlite3.connect("donnees/boutique.db")
tot_bdd = con.execute("SELECT ROUND(SUM(l.montant), 2), COUNT(*) FROM lignes_commande l JOIN
↳ commandes c ON c.id_commande = l.id_commande WHERE c.date_commande >= '2025-01-01').fetchone()
print("base :", tot_bdd, "| classeur :", (round(float(lg["montant"].sum()), 2), len(lg)))
```

```
base : (1324763.72, 29827) | classeur : (1324763.72, 29827)
```

À vous. Ajoutez un contrôle sur la colonne canal : quelles valeurs contient-elle, et combien de lignes par valeur ? Rédigez en trois lignes le « README » du classeur.

Application 2.2 — Agréger sous conditions (section 2.1.4)

Objectif. Calculer par formules quelques indicateurs de la gérante et vérifier chacun avec pandas.

Étape 1 — les formules (noms anglais), évaluées par LibreOffice.

```
F = {"CA Site": f'=SUMIFS({R("M")},{R("E")},"Site")',
    "lignes remises": f'=COUNTIFS({R("L")},>0)',
    "CA Décoration en décembre":
    ↳ f'=SUMIFS({R("M")},{R("I")},"Décoration",{R("C")},>=&DATE(2025,12,1))',
    "montant moyen d'une ligne": f'=AVERAGE({R('M')})'}
res = O.evaluer(F, {"Lignes": L})
for k, f in F.items():
    print(f"{k:28s} → {O.fr(res[k]):>12s}")
print("exemple de formule :", X.en_fr(F["CA Site"]))
```

```
CA Site                → 617 715,45
lignes remises         → 4 606
CA Décoration en décembre → 60 891,04
montant moyen d'une ligne → 44,41
exemple de formule : =SOMME.SI.ENS(Lignes!M2:M29828;Lignes!E2:E29828;"Site")
```

Étape 2 — le recoupement avec pandas.

```
d = L[L["categorie"] == "Décoration"]
attendu = {"CA Site": L.loc[L["canal"] == "Site", "montant"].sum(), "lignes remises":
↳ (L["remise_pct"] > 0).sum(),
```

```

"CA Décoration en décembre": d.loc[d["date_commande"] >= "2025-12-01", "montant"].sum(),
↳ "montant moyen d'une ligne": L["montant"].mean()
ecarts = {k: abs(float(res[k]) - float(v)) for k, v in attendu.items()}
print("écart maximal :", round(max(ecarts.values()), 6))

```

```
écart maximal : 0.0
```

À vous. Ajoutez la formule du nombre de lignes du canal Réseaux vendues en juillet, puis celle du montant moyen d'une ligne remise. Vérifiez par pandas.

Application 2.3 — Enrichir par recherche : le coût d'achat et la marge (section 2.1.6)

Objectif. Ajouter à chaque ligne le coût d'achat du produit par une recherche, puis calculer la marge par catégorie.

Étape 1 — une colonne de recherche (29 827 formules XL00KUP) et la marge par catégorie.

```

cats = sorted(L["categorie"].unique())
F = {f"marge {c}": f'=SUMPRODUCT(({R("I")}={c}))*({R("M")}-{R("J")})*Lignes!N2:N{n})' for c in cats}
F["marge totale"] = f'=SUM({R("M")})-SUMPRODUCT({R("J")},Lignes!N2:N{n})'
res = 0.evaluer(F, {"Lignes": L, "Produits": P}, colonnes={"Lignes": {"cout_u":
↳ "=XL00KUP(G{r},Produits!A:A,Produits!E:E)"}})
print(pd.Series({k: round(v, 2) for k, v in res.items()}).to_string())

```

marge Bien-être	53943.61
marge Cuisine	111442.65
marge Décoration	128773.48
marge Jardin	171875.43
marge Maison	146928.68
marge Papeterie	26847.03
marge totale	639810.88

Étape 2 — recouplement par fusion pandas.

```

M = L.merge(P[["id_produit", "cout_achat"]], on="id_produit")
M["marge"] = M["montant"] - M["quantite"] * M["cout_achat"]
att = M.groupby("categorie")["marge"].sum()
print("écart maximal par catégorie :", round(max(abs(res[f"marge {c}"] - att[c]) for c in cats), 6),
↳ "| écart sur le total :", round(abs(res["marge totale"] - M["marge"].sum()), 6))

```

```
écart maximal par catégorie : 0.0 | écart sur le total : 0.0
```

Étape 3 — le piège de la correspondance approchée. Une table de produits non triée et une recherche avec VRAI (ou sans quatrième argument) :

```

extra = {"Ref": [{"id", "nom"}, [7, "Moule mat"], [3, "Bol design"], [12, "Poêle mat"], [5, "Cadre
↳ design"]]}
r = 0.evaluer({"exacte": "=VL00KUP(5,Ref!A2:B5,2,FALSE)", "approchée":
↳ "=VL00KUP(5,Ref!A2:B5,2,TRUE)"}, extra=extra)
print(r)

```

```
{'exacte': 'Cadre design', 'approchée': '#N/A'}
```

À vous. Calculez le **taux de marge** (marge / chiffre d'affaires) de chaque catégorie par formules, puis comparez à celui du TCD de la section 2.2.4 (ratio de sommes).

Application 2.4 — Nettoyer du texte et des dates (sections 2.1.7 et 2.1.8)

Objectif. Nettoyer les premières lignes de l'export de caisse avec des formules, et comparer avec pandas.

Étape 1 — lire l'export en texte (sans interpréter) et extraire douze lignes de vente.

```
brut = pd.read_csv("donnees/export_caisse_brut.csv", sep=";", encoding="cp1252", header=None,
↳ dtype=str, skip_blank_lines=False)
ech = brut.iloc[4:16, [0, 1, 3]].reset_index(drop=True)
ech.columns = ["ticket", "date", "article"]
print(ech.head(4).to_string())
```

	ticket	date	article
0	T33133	03/11/2025	BOÎTE RUSTIQUE
1	T33137	03/11/2025	Tapis nordique
2	T33137	03/11/2025	Statuette rustique
3	T33137	03/11/2025	Étagère compact

Étape 2 — les formules, ligne par ligne. Numéro de ticket, date, article en « majuscule initiale » (la fonction NOMPROPRE mettrait une majuscule à chaque mot : nous utiliserons plutôt MAJUSCULE (GAUCHE(...)) et MINUSCULE (STXT(...)).

```
lignes = [{"ticket", "date", "article"}] + ech.values.tolist()
F = {}
for i in range(len(ech)):
    r = i + 2
    F[f"num{i}"] = f"=VALUE(RIGHT(Export!A{r},LEN(Export!A{r})-1))"
    F[f"date{i}"] = f"=DATEVALUE(Export!B{r})"
    F[f"art{i}"] = f"=UPPER(LEFT(TRIM(Export!C{r}),1))&LOWER(MID(TRIM(Export!C{r}),2,100))"
res = 0.evaluer(F, extra={"Export": lignes})
nums = [int(res[f"num{i}"]) for i in range(len(ech))]; arts = [res[f"art{i}"] for i in
↳ range(len(ech))]
dates = [pd.Timestamp("1899-12-30") + pd.Timedelta(days=int(res[f"date{i}"])) for i in
↳ range(len(ech))]
print(nums[:4], arts[:4], [d.strftime("%d/%m/%Y") for d in dates[:3]])
```

```
[33133, 33137, 33137, 33137] ['Boîte rustique', 'Tapis nordique', 'Statuette rustique', 'Étagère
↳ compact'] ['03/11/2025', '03/11/2025', '03/11/2025']
```

Étape 3 — recouplement.

```
att_num = ech["ticket"].str[1:].astype(int).tolist()
att_art = ech["article"].str.strip().str.capitalize().tolist()
att_dat = pd.to_datetime(ech["date"], format="%d/%m/%Y").tolist()
print("numéros identiques :", nums == att_num, "| articles identiques :", arts == att_art, "| dates
↳ identiques :", dates == att_dat)
```

```
numéros identiques : True | articles identiques : True | dates identiques : True
```

À vous. Que donnerait NOMPROPRE sur bien-être ? Et sur BOÎTE RUSTIQUE ? Dans quel cas la différence compte-t-elle (indice : une jointure avec le catalogue) ?

Application 2.5 — Recouper un tableau croisé dynamique (section 2.2)

Objectif. Construire le tableau croisé « catégorie × trimestre » comme le ferait Excel (avec pandas), puis le recouper avec des SOMME.SI.ENS.

Étape 1 — le tableau attendu.

```
pt = L.assign(trimestre=L["date_commande"].dt.quarter).pivot_table(index="categorie",
    ↪ columns="trimestre", values="montant", aggfunc="sum", margins=True, margins_name="Total")
print(pt.round(2).to_string())
```

trimestre	1	2	3	4	Total
categorie					
Bien-être	27442.93	26028.04	20767.18	43272.69	117510.84
Cuisine	53688.76	52530.23	42320.21	84773.38	233312.58
Décoration	55146.61	51061.80	43924.15	108609.77	258742.33
Jardin	29892.76	102809.91	143416.32	77835.65	353954.64
Maison	73190.94	65747.55	52308.41	113367.10	304614.00
Papeterie	12246.69	12605.15	11835.70	19941.79	56629.33
Total	251608.69	310782.68	314571.97	447800.38	1324763.72

Étape 2 — 24 formules SOMME.SI.ENS (6 catégories × 4 trimestres), avec des bornes de dates.

```
bornes = {1: (1, 4), 2: (4, 7), 3: (7, 10), 4: (10, 13)}
F = {}
for c in cats:
    for q, (m1, m2) in bornes.items():
        fin = "DATE(2026,1,1)" if m2 == 13 else f"DATE(2025,{m2},1)"
        F[(c, q)] =
    ↪ f'=SUMIFS({R("M")},{R("I")},{c},{R("C")}, ">="&DATE(2025,{m1},1),{R("C")}, "<"&{fin})'
res = 0.evaluer({f"{c}|{q}": f for (c, q), f in F.items()}), {"Lignes": L})
```

Étape 3 — comparer, et lire les proportions.

```
grille = pd.DataFrame({q: [res[f"{c}|{q}"] for c in cats] for q in bornes}, index=cats)
print("écart maximal :", round(float((grille - pt.loc[cats, [1, 2, 3, 4]]).abs().max()).max(), 6))
print("part de chaque trimestre dans l'année (%) :", (pt.loc["Total", [1, 2, 3, 4]] / pt.loc["Total",
    ↪ "Total"] * 100).round(1).to_dict())
```

écart maximal : 0.0

part de chaque trimestre dans l'année (%) : {1: 19.0, 2: 23.5, 3: 23.7, 4: 33.8}

À vous. Quelle catégorie est la plus saisonnière ? Proposez une mesure (rapport du meilleur trimestre au moins bon) et calculez-la.

Application 2.6 — Rejouer une chaîne Power Query sur plusieurs fichiers (section 2.3)

Objectif. Écrire une fonction qui reproduit les étapes de la requête de la section 2.3, l'appliquer à l'export de caisse, puis « empiler le dossier » avec un second fichier.

Étape 1 — la fonction de nettoyage (une étape de Power Query par ligne).

```
def nettoyer(chemin):
    b = pd.read_csv(chemin, sep=";", encoding="cp1252", header=None, dtype=str,
    ↪ skip_blank_lines=False)
    total = float(b.iloc[-1, 7].replace(",", ".")) # ligne de total, gardée
    ↪ pour le contrôle
    t = b.iloc[3:].reset_index(drop=True); t.columns = t.iloc[0]; t = t.iloc[1:]
    t = t[t["N° ticket"] != "N° ticket"]; t = t[t["Qté"].notna()].copy()
    for c in ["Prix unitaire", "Montant"]:
        t[c] = t[c].str.replace(",", ".").astype(float)
```

```

t["Qté"] = t["Qté"].astype(int); t["Date"] = pd.to_datetime(t["Date"], format="%d/%m/%Y")
t["Article"] = t["Article"].str.strip().str.capitalize(); t["Catégorie"] =
↳ t["Catégorie"].str.strip().str.capitalize()
t["Montant corrigé"] = t["Montant"].fillna((t["Qté"] * t["Prix unitaire"]).round(2))
return t.reset_index(drop=True), total

s1, total1 = nettoyer("donnees/export_caisse_brut.csv")
print(len(s1), "lignes | total de contrôle", 0.fr(total1), "| somme corrigée", 0.fr(s1["Montant
↳ corrigé"].sum()))

```

280 lignes | total de contrôle 11 561,47 | somme corrigée 11 564,09

Étape 2 — un second fichier. Nous fabriquons un « fichier de la semaine suivante » en décalant les dates de sept jours (ce n'est qu'une simulation).

```

import tempfile, shutil
txt1 = open("donnees/export_caisse_brut.csv", encoding="cp1252").read()
dossier = tempfile.mkdtemp(dir=os.environ.get("TMPDIR"))
open(f"{dossier}/semaine_1.csv", "w", encoding="cp1252").write(txt1)
txt2 = txt1
for jj in range(3, 10):
    # 03/11 → 10/11, ..., 09/11 → 16/11 (aucune collision)
    txt2 = txt2.replace(f";{jj:02d}/11/2025;", f";{jj + 7:02d}/11/2025;")
open(f"{dossier}/semaine_2.csv", "w", encoding="cp1252").write(txt2)

```

Étape 3 — empiler le dossier et contrôler.

```

parts = [nettoyer(f"{dossier}/{f}") for f in sorted(os.listdir(dossier))]
tout = pd.concat([p[0] for p in parts], ignore_index=True)
print(len(tout), "lignes | du", tout["Date"].min().date(), "au", tout["Date"].max().date(), "| total
↳ de contrôle cumulé", 0.fr(sum(p[1] for p in parts)))
shutil.rmtree(dossier)

```

560 lignes | du 2025-11-03 au 2025-11-16 | total de contrôle cumulé 23 122,94

À vous. Que se passe-t-il si un fichier du dossier a une colonne de plus ? Quelle étape échoue, et pourquoi est-ce « un bon signe » ?

Application 2.7 — Auditer un classeur abîmé (section 2.4)

Objectif. Injecter des défauts dans les données et vérifier qu'une feuille de contrôles les détecte.

Étape 1 — abîmer une copie (40 doublons, 25 canaux mal écrits, 10 montants vides, 5 montants négatifs).

```

ab = pd.concat([L, L.sample(40, random_state=1)], ignore_index=True)
ab.loc[ab.sample(25, random_state=2).index, "canal"] = "site "
ab.loc[ab.sample(10, random_state=3).index, "montant"] = np.nan
idx_neg = ab[ab["montant"].notna()].sample(5, random_state=4).index
ab.loc[idx_neg, "montant"] = -ab.loc[idx_neg, "montant"]
na = len(ab) + 1
print(len(ab), "lignes dans la copie abîmée")

```

29867 lignes dans la copie abîmée

Étape 2 — la feuille de contrôles.

```

r = lambda c: f"Ab!{c}2:{c}{na}"
F = {"lignes": f"=ROWS({r('A')})", "doublons d'identifiant":
↳ f"=ROWS({r('A')})-COUNTA(UNIQUE({r('A')}))",
    "canaux non reconnus": f"=ROWS({r('E')})-COUNTIFS({r('E')}, 'Site')-COUNTIFS({r('E')},
↳ ',Boutique')-COUNTIFS({r('E')}, 'Réseaux')",

```

```

    "montants vides": f"=COUNTBLANK({r('M')})", "montants négatifs": f"=COUNTIFS({r('M')}, "<0")",
    ↪ "total": f"=SUM({r('M')})"
res = 0.evaluer(F, {"Ab": ab})
for k, v in res.items():
    print(f"{k:24s} {0.fr(v)}")

```

```

lignes                29 867
doublons d'identifiant 40
canaux non reconnus   25
montants vides        10
montants négatifs     5
total                 1 326 154,16

```

Étape 3 — recouper avec pandas.

```

print("pandas : doublons", int(ab["id_ligne"].duplicated().sum()), "| canaux non reconnus",
    ↪ int((~ab["canal"].isin(["Site", "Boutique", "Réseaux"])).sum()),
    "| vides", int(ab["montant"].isna().sum()), "| négatifs", int((ab["montant"] < 0).sum()))

```

```
pandas : doublons 40 | canaux non reconnus 25 | vides 10 | négatifs 5
```

À vous. Quels défauts un contrôle de total seul n'aurait-il pas détectés ? Ajoutez un contrôle sur les dates hors période.

Application 2.8 — Trianguler : formules, SQL et pandas (sections 2.5 et 2.6)

Objectif. Calculer le chiffre d'affaires du Site par mois avec trois outils indépendants, et vérifier qu'ils s'accordent ; écrire l'équivalent d'une mesure DAX et d'une requête QUERY.

Étape 1 — douze formules SOMME.SI.ENS.

```

F = {}
for m in range(1, 13):
    fin = "DATE(2026,1,1)" if m == 12 else f"DATE(2025,{m + 1},1)"
    F[m] = f"=SUMIFS({R('M')},{R('E')}, 'Site', {R('C')}, '>='&DATE(2025,{m},1), {R('C')}, '<'&{fin})"
res = 0.evaluer(F, {"Lignes": L})
excel = pd.Series({m: res[m] for m in F})

```

Étape 2 — SQL et pandas.

```

con = sqlite3.connect("donnees/boutique.db")
sql = pd.read_sql("""SELECT CAST(strftime('%m', c.date_commande) AS INTEGER) AS mois, SUM(l.montant)
    ↪ AS ca FROM lignes_commande l
        JOIN commandes c ON c.id_commande = l.id_commande WHERE c.canal = 'Site' AND
    ↪ c.date_commande >= '2025-01-01' GROUP BY mois""", con).set_index("mois")["ca"]
pdp = L[L["canal"] == "Site"].groupby(L["date_commande"].dt.month)["montant"].sum()
print("écart Excel-SQL :", round(float((excel - sql).abs().max()), 6), "| écart Excel-pandas :",
    ↪ round(float((excel - pdp).abs().max()), 6))
print("meilleur mois :", int(excel.idxmax()), "→", 0.fr(excel.max()), "€ | total :",
    ↪ 0.fr(excel.sum()), "€")

```

```

écart Excel-SQL : 0.0 | écart Excel-pandas : 0.0
meilleur mois : 12 → 90 048,47 € | total : 617 715,45 €

```

Étape 3 — une mesure DAX en pandas. CA N-1 et Croissance pour le Site, 2025 contre 2024 : pandas sur toutes les années.

```

cmd = pd.read_csv("donnees/commandes.csv", parse_dates=["date_commande"]); lig =
    ↪ pd.read_csv("donnees/lignes_commande.csv")
tt = lig.merge(cmd[["id_commande", "date_commande", "canal"]], on="id_commande"); tt = tt[tt["canal"]
    ↪ == "Site"]

```

```
ca_an = tt.groupby(tt["date_commande"].dt.year)["montant"].sum()
print({int(k): 0.fr(v) for k, v in ca_an.items()}, "| croissance du Site 2025/2024 :",
      ↪ 0.fr((ca_an[2025] / ca_an[2024] - 1) * 100), "%")
```

```
{2023: '422\u202f440,34', 2024: '502\u202f531', 2025: '617\u202f715,45'} | croissance du Site
↪ 2025/2024 : 22,92 %
```

À vous. Écrivez la formule QUERY de Google Sheets qui donnerait le CA du Site par mois, et la requête SQL équivalente.

Exercices

Exercice 2.1 ★ — Que devient une formule recopiée ? (section 2.1.2)

La cellule D5 contient $=B2*C\$1$. Écrivez le contenu de la cellule obtenue en recopiant D5 (a) vers F8, (b) vers H5, (c) vers D9. Vérifiez avec une petite fonction qui applique la règle de décalage.

Exercice 2.2 ★ — SOMME.SI.ENS à la main (section 2.1.4)

Voici huit lignes de ventes :

Ligne	Catégorie	Canal	Montant
1	Cuisine	Boutique	40
2	Cuisine	Site	25
3	Jardin	Boutique	80
4	Jardin	Site	60
5	Cuisine	Boutique	10
6	Jardin	Site	30
7	Cuisine	Site	15
8	Jardin	Boutique	20

Sans ordinateur, calculez : (a) la somme des montants du Jardin en Boutique ; (b) le nombre de lignes du Site avec un montant d'au moins 30 ; (c) le montant moyen de la Cuisine. Écrivez les trois formules, puis vérifiez avec 0.evaluer.

Exercice 2.3 ★★ — Trois tailles de ligne (section 2.1.5)

On range une ligne de vente de 2025 en « petit » (moins de 40 €), « moyen » (de 40 € inclus à 100 € exclus) ou « gros » (100 € et plus). Écrivez les 9 formules NB.SI.ENS qui comptent les lignes de chaque taille pour chaque canal, puis recoupez avec pd.crosstab et pd.cut. Quel canal a la plus grande part de gros ?

Exercice 2.4 ★★ — Un barème de remise par palier (section 2.1.6)

Une remise de fidélité dépend de la quantité : 0 % pour 1 article, 3 % pour 2, 5 % pour 3 ou 4, 8 % à partir de 5 (barème 1 → 0 ; 2 → 0,03 ; 3 → 0,05 ; 5 → 0,08). (a) Appliquez-le à chaque ligne de 2025 par RECHERCHEV en correspondance approchée et calculez la **remise moyenne** ; recoupez avec pandas. (b) Que se passe-t-il si le barème n'est plus trié (5, 1, 3, 2) ? Testez la quantité 3.

Exercice 2.5 ★ — Dates (section 2.1.8)

Calculez par formules, puis par Python : (a) le jour de la semaine du 25/12/2025 ; (b) le dernier jour de février 2024 ; (c) le nombre de jours ouvrés de décembre 2025 ; (d) l'âge en années entières, au 31/12/2025, d'une personne née le 12/05/1990.

Exercice 2.6 ★★ — Un tableau croisé à la main (section 2.2)

Avec les huit lignes de l'exercice 2.2 : construisez à la main le tableau croisé « catégorie en lignes, canal en colonnes, somme du montant », avec les totaux ; puis le même tableau en % **du total de la ligne**. Vérifiez avec `pivot_table`.

Exercice 2.7 ★★ — Moyenne des ratios ou ratio des sommes ? (section 2.2.4)

Trois lignes de vente ont pour (montant, marge) : (10 ; 5), (100 ; 30), (200 ; 60). Calculez le taux de marge **moyen des lignes** et le taux de marge **global** (somme des marges sur somme des montants). Lequel donnerait un champ calculé de tableau croisé ? Faites ensuite le même calcul par canal sur les ventes de 2025.

Exercice 2.8 ★★ — L'ordre des étapes compte (section 2.3)

Dans la requête de la section 2.3, on change le type de Qté en entier **après** avoir retiré les en-têtes répétés et la ligne de total. Montrez ce qui se passe si l'on change le type **avant**. Quelle erreur obtient-on, et pourquoi est-ce « un bon signe » ?

Exercice 2.9 ★★★ — Dépivoter et repivoter (section 2.3.4)

Construisez le tableau **large** du chiffre d'affaires 2025 par canal (en lignes) et par mois (en colonnes). Dépivotez-le en un tableau **long**, puis repivotez-le. Vérifiez que vous retrouvez le tableau de départ et que le total est conservé.

Exercice 2.10 ★★ — Remettre un tableau en forme (section 2.4.2)

Le tableau ci-dessous (construit dans le corrigé) est présenté « pour être lu » : un titre, un blanc, des mois en lignes, des catégories en colonnes, des lignes de sous-total par trimestre. Écrivez le code qui le transforme en tableau ordonné (une ligne par mois et catégorie, sans total), puis vérifiez que le total général est conservé.

Exercice 2.11 ★★★ — Cinq contrôles (section 2.4.4)

Sur un extrait de 2 000 lignes de la feuille Lignes, on a injecté : 7 identifiants clients qui n'existent pas dans la feuille Clients, 4 dates de 2024, 3 lignes en double et 5 canaux mal écrits. Écrivez **cinq contrôles** sous forme de formules (clients inconnus, dates hors période, doublons, canaux hors liste, total comparé à une valeur de référence), appliquez-les et construisez le tableau « défaut injecté → contrôle qui le détecte ».

Exercice 2.12 ★★★ — Le panier moyen par canal, dans cinq outils (sections 2.5 et 2.6)

Le **panier moyen** d'un canal est son chiffre d'affaires divisé par son nombre de commandes **distinctes**. Calculez-le pour chaque canal en 2025 : par formules (`SOMME.SI.ENS` et `NB-VAL(UNIQUE(FILTRE(...)))`), en SQL (SQLite), en pandas. Écrivez (sans l'exécuter) la mesure DAX et la formule QUERY de Google Sheets équivalentes.

Corrigés

Corrigé 2.1

Règle : une référence **relative** se décale du même nombre de lignes et de colonnes que la formule ; la partie précédée de \$ ne bouge pas. (a) D5 → F8 : deux colonnes et trois lignes plus loin : \$B2 devient \$B5 (colonne figée, ligne relative +3), C\$1 devient E\$1 (colonne relative +2, ligne figée) : **=\$B5*E\$1**. (b) D5 → H5 : quatre colonnes plus loin, même ligne : **=\$B2*G\$1**. (c) D5 → D9 : quatre lignes plus bas : **=\$B6*C\$1**.

```
import re
def decaler(formule, dl, dc):
    num = lambda s: sum((ord(ch) - 64) * 26 ** i for i, ch in enumerate(reversed(s)))
    nom = lambda k: (nom((k - 1) // 26) if k > 26 else "") + chr(65 + (k - 1) % 26)
    def f(m):
        ca, c, la, l = m.groups()
        return f"{ca}{c if ca else nom(num(c) + dc)}{la}{l if la else int(l) + dl}"
    return re.sub(r"(\?)([A-Z]+)(\?)(\d+)", f, formule)
print(decaler("=$B2*C$1", 3, 2), decaler("=$B2*C$1", 0, 4), decaler("=$B2*C$1", 4, 0))
```

=\$B5*E\$1 \$=B2*G\$1 \$=B6*C\$1

Corrigé 2.2

(a) Jardin en Boutique : $80 + 20 = 100$. (b) Lignes du Site avec un montant ≥ 30 : les montants du Site sont 25, 60, 30, 15 ; deux sont ≥ 30 : **2**. (c) Montant moyen de la Cuisine : $(40 + 25 + 10 + 15) / 4 = 22,5$. Formules (avec les colonnes A à D et les lignes 2 à 9) : **=SOMME.SI.ENS(D2:D9;B2:B9;"Jardin";C2:C9;"Boutique")**, **=NB.SI.ENS(C2:C9;"Site";D2:D9;">=30")**, **=MOYENNE.SI.ENS(D2:D9;B2:B9;"Cuisine")**.

```
lg8 = [{"ligne", "catégorie", "canal", "montant"}, [1, "Cuisine", "Boutique", 40], [2, "Cuisine",
↳ "Site", 25], [3, "Jardin", "Boutique", 80], [4, "Jardin", "Site", 60],
      [5, "Cuisine", "Boutique", 10], [6, "Jardin", "Site", 30], [7, "Cuisine", "Site", 15], [8,
↳ "Jardin", "Boutique", 20]]
F = {"a": '=SUMIFS(H!D2:D9,H!B2:B9,"Jardin",H!C2:C9,"Boutique")', "b":
↳ '=COUNTIFS(H!C2:C9,"Site",H!D2:D9,">=30")', "c": '=AVERAGEIFS(H!D2:D9,H!B2:B9,"Cuisine")'}
print(0.evaluer(F, extra={"H": lg8}))
```

{'a': 100, 'b': 2, 'c': 22.5}

Corrigé 2.3

```
canaux = ["Boutique", "Site", "Réseaux"]; tailles = ["petit", "moyen", "gros"]
F = {}
for c in canaux:
    F[f"petit|{c}"] = f'=COUNTIFS({R("E")},{c},{R("M")}, "<40")'
    F[f"moyen|{c}"] = f'=COUNTIFS({R("E")},{c},{R("M")}, ">=40",{R("M")}, "<100")'
    F[f"gros|{c}"] = f'=COUNTIFS({R("E")},{c},{R("M")}, ">=100")'
res = 0.evaluer(F, {"Lignes": L})
tab = pd.DataFrame({c: [int(res[f"{t}|{c}"]) for t in tailles] for c in canaux}, index=tailles)
att = pd.crosstab(pd.cut(L["montant"], [-np.inf, 40, 100, np.inf], right=False, labels=tailles),
↳ L["canal"])[canaux]
print(tab.to_string()); print("identique à pandas :", bool((tab.values == att.values).all()))
print("part de gros (%) :", (tab.loc["gros"] / tab.sum() * 100).round(1).to_dict())
```

	Boutique	Site	Réseaux
petit	7425	8278	1938
moyen	4122	4488	1094

```

gros      1064 1162      256
identique à pandas : True
part de gros (%) : {'Boutique': 8.4, 'Site': 8.3, 'Réseaux': 7.8}

```

Les neuf formules et `pd.crosstab` donnent le même tableau. La part de lignes « grosses » est de **8,4 %** en Boutique, **8,3 %** sur le Site et **7,8 %** pour les Réseaux : la Boutique est en tête, mais l'écart est minime ; sur ces données simulées, la taille d'une ligne ne dépend guère du canal.

Corrigé 2.4

- (a) Le barème approché lit « la plus grande quantité inférieure ou égale » : 4 articles tombent donc dans la tranche de 3 (5 %). La remise moyenne est de **0,53 %** : la grande majorité des lignes (25 366 sur 29 827) ne comptent qu'un article.

```

bar = [{"q", "r"}, [1, 0.0], [2, 0.03], [3, 0.05], [5, 0.08]]
res = 0.evaluer({"moyenne": f"AVERAGE(Lignes!N2:N{n})", {"Lignes": L}, extra={"Ba": bar},
↪ colonnes={"Lignes": {"rem": "=VLOOKUP(J{r},Ba!A2:B5,2,TRUE)"}})
att = np.array([0.0, 0.03, 0.05, 0.08])[np.searchsorted([1, 2, 3, 5], L["quantite"], side="right") -
↪ 1]
print("remise moyenne : feuille de calcul", round(res["moyenne"], 6), "| pandas",
↪ round(float(att.mean()), 6), "| répartition des quantités :",
↪ L["quantite"].value_counts().sort_index().to_dict())

```

```

remise moyenne : feuille de calcul 0.00529 | pandas 0.00529 | répartition des quantités : {1: 25366,
↪ 2: 3264, 3: 881, 4: 316}

```

- (b) Avec le barème **non trié** (5, 1, 3, 2), la correspondance approchée n'a plus de sens :

```

ba_n = [{"q", "r"}, [5, 0.08], [1, 0.0], [3, 0.05], [2, 0.03]]
r = 0.evaluer({"q3": "=VLOOKUP(3,Ba!A2:B5,2,TRUE)", "q4": "=VLOOKUP(4,Ba!A2:B5,2,TRUE)"},
↪ extra={"Ba": ba_n})
print(r)

```

```
{'q3': '#N/A', 'q4': '#N/A'}
```

Ici le tableur renvoie #N/A ; dans Excel, le résultat sur une table non triée peut être faux **sans erreur**, ce qui est pire. Règle : la correspondance approchée exige une première colonne **triée par ordre croissant**.

Corrigé 2.5

- (a) Le 25 décembre 2025 est un **jeudi**. (b) Le dernier jour de février 2024 est le **29 février** (année bissextile), numéro de série 45 351. (c) Décembre 2025 compte **23 jours ouvrés** (31 jours moins 8 jours de week-end). (d) **35 ans** (le 12 mai 1990 est antérieur au 31 décembre 2025 de 35 ans, 7 mois et 19 jours).

```

F = {"jour": '=TEXT(DATE(2025,12,25),"dddd')', "fin_fev": "=EOMONTH(DATE(2024,2,15),0)", "ouvres":
↪ "=NETWORKDAYS(DATE(2025,12,1),DATE(2025,12,31))",
    "age": '=DATEDIF(DATE(1990,5,12),DATE(2025,12,31),"Y")'}
r = 0.evaluer(F)
print(r["jour"], "|", (pd.Timestamp("1899-12-30") + pd.Timedelta(days=int(r["fin_fev"]))).date(),
↪ "|", int(r["ouvres"]), "|", int(r["age"]))
print(pd.Timestamp("2025-12-25").day_name(), "|", np.busday_count("2025-12-01", "2026-01-01"), "|",
↪ (pd.Timestamp("2025-12-31") - pd.Timestamp("1990-05-12")).days // 365.25)

```

```

jeudi | 2024-02-29 | 23 | 35
Thursday | 23 | 35.0

```

Corrigé 2.6

Sommes : Cuisine × Boutique = 50, Cuisine × Site = 40, Jardin × Boutique = 100, Jardin × Site = 90 ; totaux : Cuisine 90, Jardin 190, Boutique 150, Site 130 ; total général 280. En % du total de la ligne : Cuisine 55,6 % Boutique / 44,4 % Site ; Jardin 52,6 % / 47,4 % ; ensemble 53,6 % / 46,4 %.

```
t8 = pd.DataFrame(lg8[1:], columns=lg8[0])
pt8 = t8.pivot_table(index="catégorie", columns="canal", values="montant", aggfunc="sum",
    ↪ margins=True, margins_name="Total")
print(pt8.to_string()); print((pt8.div(pt8["Total"], axis=0) * 100).round(1).to_string())
```

canal	Boutique	Site	Total
catégorie			
Cuisine	50	40	90
Jardin	100	90	190
Total	150	130	280

canal	Boutique	Site	Total
catégorie			
Cuisine	55.6	44.4	100.0
Jardin	52.6	47.4	100.0
Total	53.6	46.4	100.0

Corrigé 2.7

Taux par ligne : 50 %, 30 % et 30 % ; **moyenne** des taux = $(50 + 30 + 30) / 3 = 36,7 \%$. Taux **global** = $(5 + 30 + 60) / (10 + 100 + 200) = 95 / 310 = 30,6 \%$. Les deux diffèrent parce que la ligne à 10 € (taux élevé) compte autant que les lignes à 100 et 200 € dans la moyenne, et presque rien dans le ratio des sommes. Un champ calculé de tableau croisé donne le **ratio des sommes** (30,6 %). Sur les ventes 2025, les deux mesures sont presque identiques par canal (de 48,1 % à 48,4 %), car les lignes ont des taux de marge voisins : l'écart de l'exemple à la main est pédagogique, il est plus faible sur nos données.

```
print("moyenne des taux :", round(np.mean([5 / 10, 30 / 100, 60 / 200]) * 100, 1), "% | ratio des
    ↪ sommes :", round(95 / 310 * 100, 1), "%")
M = L.merge(P[["id_produit", "cout_achat"]], on="id_produit"); M["marge"] = M["montant"] -
    ↪ M["quantite"] * M["cout_achat"]
par = M.groupby("canal").apply(lambda g: pd.Series({"moyenne des taux (%)": (g["marge"] /
    ↪ g["montant"]).mean() * 100, "ratio des sommes (%)": g["marge"].sum() / g["montant"].sum() *
    ↪ 100}), include_groups=False)
print(par.round(2).to_string())
```

moyenne des taux : 36.7 % ratio des sommes : 30.6 %	
	moyenne des taux (%) ratio des sommes (%)
canal	
Boutique	48.26 48.33
Réseaux	48.20 48.43
Site	48.08 48.24

Corrigé 2.8

Si l'on convertit Qté en entier avant d'avoir retiré les en-têtes répétés et la ligne de total, la colonne contient encore du texte (« Qté ») et du vide :

```
b = pd.read_csv("donnees/export_caisse_brut.csv", sep=";", encoding="cp1252", header=None, dtype=str,
    ↪ skip_blank_lines=False)
t = b.iloc[3:].reset_index(drop=True); t.columns = t.iloc[0]; t = t.iloc[1:]
try:
    t["Qté"].astype(int)
except Exception as e:
    print("avant le nettoyage :", type(e).__name__, "-", str(e)[:60])
```

```
t2 = t[t["N° ticket"] != "N° ticket"]; t2 = t2[t2["Qté"].notna()]
print("après le nettoyage :", int(t2["Qté"].astype(int).sum()), "articles")
```

avant le nettoyage : ValueError – cannot convert float NaN to integer
 après le nettoyage : 338 articles

L'erreur est **visible** : la requête s'arrête au lieu de produire un résultat faux. C'est un bon signe : une conversion de type qui échoue est le moyen le plus simple de découvrir qu'une ligne parasite traîne dans les données. Dans Power Query, l'étape « Type modifié » afficherait des cellules Error ; il ne faut pas les remplacer mécaniquement par des valeurs vides.

Corrigé 2.9

```
mois = L["date_commande"].dt.month
large = L.pivot_table(index="canal", columns=mois, values="montant", aggfunc="sum")
long = large.reset_index().melt(id_vars="canal", var_name="mois", value_name="montant")
retour = long.pivot(index="canal", columns="mois", values="montant")
print(large.shape, "→", long.shape, "| identique après repivot :", bool(np.allclose(retour.values,
↪ large.values)), "| total :", round(long["montant"].sum(), 2))
```

(3, 12) → (36, 3) | identique après repivot : True | total : 1324763.72

Les 3 × 12 cases deviennent 36 lignes ; le repivot redonne le tableau de départ, et le total (1 324 763,72 €) est conservé.

Corrigé 2.10

```
mp = L.assign(mois=L["date_commande"].dt.month).pivot_table(index="mois", columns="categorie",
↪ values="montant", aggfunc="sum").round(0)
cats = ["Cuisine", "Maison"]; noms = ["Janvier", "Février", "Mars", "Avril"]
lignes = [["Ventes 2025 (en €)", None, None], [None, None, None], ["Mois", *cats]]
for m in (1, 2, 3):
    lignes.append([noms[m - 1], *[float(mp.loc[m, c]) for c in cats]])
lignes.append(["Sous-total T1", *[float(sum(mp.loc[m, c] for m in (1, 2, 3))) for c in cats]])
lignes.append([noms[3], *[float(mp.loc[4, c]) for c in cats]])
sale = pd.DataFrame(lignes)
```

```
t = sale.iloc[3:].copy(); t.columns = sale.iloc[2]; t = t.rename(columns={"Mois": "mois"})
t = t[~t["mois"].str.startswith("Sous-total")]
propre = t.melt(id_vars="mois", var_name="categorie", value_name="montant")
total_attendu = float(sale.iloc[3:, 1:].astype(float).sum().sum() - sale.iloc[6,
↪ 1:].astype(float).sum())
print(propre.shape, "| total conservé :", float(propre["montant"].sum()) == total_attendu)
```

(8, 3) | total conservé : True

Les étapes sont celles que fait Power Query : sauter les trois premières lignes, promouvoir l'en-tête, **supprimer les lignes de sous-total** (sinon le total est compté deux fois), dépivoter. Le contrôle compare le total du tableau ordonné au total des lignes de détail.

Corrigé 2.11

```
S = L.head(2000).copy()
S.loc[S.index[:7], "id_client"] = 999999 # clients inconnus
S.loc[S.index[10:14], "date_commande"] = pd.Timestamp("2024-12-30") # dates hors période
S = pd.concat([S, S.iloc[20:23]], ignore_index=True) # doublons
```

```

S.loc[S.index[40:45], "canal"] = "Boutique " # canaux mal écrits
ns = len(S) + 1; ref_total = float(L.head(2000)["montant"].sum())
r = lambda c: f"S!{c}2:{c}{ns}"
F = {"clients inconnus": f"=SUMPRODUCT(--ISNA(MATCH({r('D')}),Clients!A2:A6001,0)))",
     "dates hors période":
     ↪ f'=COUNTIFS({r("C")}, "<"&DATE(2025,1,1))+COUNTIFS({r("C")}, ">"&DATE(2025,12,31))',
     "doublons": f"=ROWS({r('A')})-COUNTA(UNIQUE({r('A')}))",
     "canaux hors liste": f"=ROWS({r("E")})-COUNTIFS({r("E")}, "Site")-COUNTIFS({r("E")}, "Boutique")-
     ↪ COUNTIFS({r("E")}, "Réseaux")",
     "total (écart à la référence)": f"=SUM({r('M')})-{ref_total}"
res = 0.evaluer(F, {"S": S, "Clients": C})
for k, v in res.items():
    print(f"{k:30s}{0.fr(v)}")

```

```

clients inconnus      7
dates hors période    4
doublons              3
canaux hors liste     5
total (écart à la référence) 84,16

```

Le tableau « défaut → contrôle » : **clients inconnus** → contrôle 1 (7) ; **dates de 2024** → contrôle 2 (4) ; **doublons** → contrôle 3 (3 lignes ; ils font aussi monter le total de 84,16 €, donc le contrôle 5 les voit) ; **canaux mal écrits** → contrôle 4 (5). Le contrôle du **total** ne voit que les défauts qui changent la somme (ici, les doublons) : il est nécessaire, mais **insuffisant** à lui seul.

Corrigé 2.12

```

canaux = ["Boutique", "Site", "Réseaux"]
F = {}
for c in canaux:
    F[f"ca|{c}"] = f'=SUMIFS({R("M")},{R("E")},{c})'
    F[f"nb|{c}"] = f'=COUNTA(UNIQUE(FILTER({R("B")},{R("E")}={c})))'
res = 0.evaluer(F, {"Lignes": L})
exc = {c: res[f"ca|{c}"] / res[f"nb|{c}"]} for c in canaux
con = sqlite3.connect("donnees/boutique.db")
sql = pd.read_sql("""SELECT c.canal, ROUND(SUM(l.montant) / COUNT(DISTINCT c.id_commande), 4) AS
↪ panier FROM lignes_commande l JOIN commandes c ON c.id_commande = l.id_commande
WHERE c.date_commande >= '2025-01-01' GROUP BY c.canal""",
↪ con).set_index("canal")["panier"]
pdp = L.groupby("canal").apply(lambda g: g["montant"].sum() / g["id_commande"].nunique(),
↪ include_groups=False)
print({c: round(exc[c], 2) for c in canaux}); print("écarts maximaux : SQL", round(max(abs(exc[c] -
↪ sql[c]) for c in canaux), 4), "| pandas", round(max(abs(exc[c] - pdp[c]) for c in canaux), 4))

```

```

{'Boutique': 103.08, 'Site': 101.63, 'Réseaux': 102.44}
écarts maximaux : SQL 0.0 | pandas 0.0

```

Les trois outils donnent les mêmes paniers moyens : **103,08 €** en Boutique, **101,63 €** sur le Site, **102,44 €** pour les Réseaux. La mesure DAX équivalente est Panier moyen := DIVIDE (SUM (Lignes[montant]), DISTINCTCOUNT (Lignes[id_commande])) : placée dans un tableau croisé par canal, elle s'évalue canal par canal (non exécuté). La formule QUERY de Google Sheets est =QUERY(Lignes!A1:M ; "select E, sum(M) group by E" ; 1) pour le chiffre d'affaires, à diviser par le nombre de commandes distinctes (par exemple =COUNTUNIQUE(FILTER(Lignes!B:B ; Lignes!E:E = "Site"))): non exécuté, syntaxe à vérifier.

Chapitre 3 : SQL — exercices et applications

Ce chapitre du cahier prolonge le chapitre 3 du livre. Les **applications** sont de petites études guidées sur la base `boutique.db`, à refaire pas à pas ; les **exercices** (★ facile, ★★ moyen, ★★★ plus long) vous demandent d'écrire vos propres requêtes ; les **corrigés** suivent. Une règle d'or : **avant** de lire un corrigé, écrivez votre requête, exécutez-la, et vérifiez au moins **un total** par un autre chemin. Le cahier est autonome : la cellule suivante ouvre une **copie jetable** de la base (vous pouvez y créer des index et des vues sans rien abîmer).

```
import os, shutil, sqlite3, tempfile
import pandas as pd

TMP3C = tempfile.mkdtemp(prefix="sql3c_", dir=os.environ.get("TMPDIR"))
shutil.copy(os.path.join(os.environ["DONNEES"], "boutique.db"), os.path.join(TMP3C, "boutique.db"))
con = sqlite3.connect(os.path.join(TMP3C, "boutique.db"))
```

Applications

Application 3.1 — Découvrir la base (section 3.1)

Objectif. Avant toute analyse, on **inspecte** la base : que contient-elle, sur quelle période, avec quelles valeurs, et les données sont-elles saines ? Cette application est le rituel d'ouverture de n'importe quelle étude.

Étape 1 — Les tables et leurs colonnes. Une petite boucle Python interroge le catalogue de la base :

```
for t in ("clients", "produits", "commandes", "lignes_commande", "retours", "jours_exploitation"):
    nb = con.execute(f"SELECT COUNT(*) FROM {t}").fetchone()[0]
    cols = [r[1] for r in con.execute(f"PRAGMA table_info({t})")]
    print(f"{t:19s} {nb:>6d} lignes, {len(cols)} colonnes : {' '.join(cols[:4])}...")
```

<code>clients</code>	6000 lignes, 8 colonnes : <code>id_client</code> , <code>date_inscription</code> , <code>annee_naissance</code> , <code>ville...</code>
<code>produits</code>	120 lignes, 7 colonnes : <code>id_produit</code> , <code>nom_produit</code> , <code>categorie</code> , <code>prix_vente...</code>
<code>commandes</code>	36395 lignes, 7 colonnes : <code>id_commande</code> , <code>date_commande</code> , <code>heure</code> , <code>id_client...</code>
<code>lignes_commande</code>	83905 lignes, 7 colonnes : <code>id_ligne</code> , <code>id_commande</code> , <code>id_produit</code> , <code>quantite...</code>
<code>retours</code>	5002 lignes, 5 colonnes : <code>id_retour</code> , <code>id_ligne</code> , <code>date_retour</code> , <code>motif...</code>
<code>jours_exploitation</code>	1096 lignes, 8 colonnes : <code>date</code> , <code>jour_semaine</code> , <code>nb_commandes</code> , <code>chiffre_affaires...</code>

Étape 2 — La période couverte. Les trois dates à connaître : première et dernière commande, première et dernière inscription.

```
SELECT (SELECT MIN(date_commande) FROM commandes) AS premiere_commande,
       (SELECT MAX(date_commande) FROM commandes) AS derniere_commande,
       (SELECT MIN(date_inscription) FROM clients) AS premiere_inscription,
       (SELECT MAX(date_inscription) FROM clients) AS derniere_inscription
```

```
premiere_commande derniere_commande premiere_inscription derniere_inscription
2023-01-01        2025-12-31        2018-01-01        2025-12-30
```

Étape 3 — Les modalités des variables qualitatives. GROUP BY + COUNT(*) donne la distribution de chaque colonne de texte ; on regarde si les libellés sont cohérents (casse, fautes, valeurs inattendues) :

```
SELECT 'canal' AS variable, canal AS modalite, COUNT(*) AS nb FROM commandes GROUP BY canal
UNION ALL SELECT 'code_promo', CASE WHEN code_promo = '' THEN '(aucun)' ELSE code_promo END, COUNT(*)
  FROM commandes GROUP BY code_promo
UNION ALL SELECT 'motif de retour', motif, COUNT(*) FROM retours GROUP BY motif
ORDER BY variable, nb DESC
```

variable	modalite	nb
canal	Boutique	16975
canal	Site	15463
canal	Réseaux	3957
code_promo	(aucun)	30652
code_promo	SOLDES	3006
code_promo	FIDELITE	2423
code_promo	BIENVENUE	314
motif de retour	Mauvais choix	1571
motif de retour	Changement d'avis	1518
motif de retour	Défaut	908
motif de retour	Livraison tardive	605
motif de retour	Autre	400

Étape 4 — Les contrôles de santé. Trois contrôles qui ne doivent rien renvoyer : une clé en double, une quantité ou un montant qui n'est pas strictement positif, un retour **antérieur** à la commande.

```
SELECT 'clés de commande en double' AS controle, COUNT(*) - COUNT(DISTINCT id_commande) AS anomalies
  FROM commandes
UNION ALL SELECT 'lignes avec quantité ou montant <= 0', COUNT(*) FROM lignes_commande WHERE quantite
  <= 0 OR montant <= 0
UNION ALL SELECT 'retours avant la commande', COUNT(*)
  FROM retours r JOIN lignes_commande l USING (id_ligne) JOIN commandes c USING (id_commande)
  WHERE r.date_retour < c.date_commande
```

	controle	anomalies
clés de commande en double		0
lignes avec quantité ou montant <= 0		0
retours avant la commande		0

Lecture. La base couvre les commandes du **1er janvier 2023 au 31 décembre 2025**, et les inscriptions de 2018 à 2025 (4 000 clients étaient déjà inscrits en 2023). Les canaux sont bien trois, sans variante d'écriture ; **84 % des commandes n'ont aucun code promo** (enregistré par un texte vide : voir 3.1.8). Les trois contrôles renvoient 0 : on peut travailler. Sur une vraie base, un contrôle non nul n'est pas un échec mais une **découverte** à documenter.

À vous. Ajoutez deux contrôles de votre choix (une date de commande postérieure à la date du jour, un prix de vente inférieur au coût d'achat).

Application 3.2 — Les ventes par canal et par année (sections 3.1 et 3.2)

Objectif. Construire le tableau que la gérante réclame depuis des mois : le chiffre d'affaires de chaque canal, année par année, et la part du site.

Étape 1 — Un tableau croisé en SQL. Pour mettre les canaux en **colonnes**, on utilise un `SUM(CASE WHEN ...)` par canal :

```
SELECT strftime('%Y', c.date_commande) AS annee,
       ROUND(SUM(CASE WHEN c.canal = 'Boutique' THEN l.montant END)) AS boutique,
       ROUND(SUM(CASE WHEN c.canal = 'Site' THEN l.montant END)) AS site,
       ROUND(SUM(CASE WHEN c.canal = 'Réseaux' THEN l.montant END)) AS reseaux,
       ROUND(SUM(l.montant)) AS total
FROM commandes c JOIN lignes_commande l USING (id_commande)
GROUP BY annee
ORDER BY annee
```

annee	boutique	site	reseaux	total
2023	593612.0	422440.0	122880.0	1138932.0
2024	558143.0	502531.0	128787.0	1189461.0
2025	560974.0	617715.0	146074.0	1324764.0

Étape 2 — La part du site et la croissance. Le même calcul, exprimé en pourcentages :

```
WITH t AS (
  SELECT strftime('%Y', c.date_commande) AS annee,
         SUM(CASE WHEN c.canal = 'Site' THEN l.montant END) AS site, SUM(l.montant) AS total
  FROM commandes c JOIN lignes_commande l USING (id_commande) GROUP BY annee)
SELECT annee, ROUND(100.0 * site / total, 1) AS part_site_pct,
       ROUND(100.0 * (total / LAG(total) OVER (ORDER BY annee) - 1), 1) AS croissance_totale_pct
FROM t ORDER BY annee
```

annee	part_site_pct	croissance_totale_pct
2023	37.1	NaN
2024	42.2	4.4
2025	46.6	11.4

Lecture. Le chiffre d'affaires passe de 1,14 million d'euros en 2023 à **1,32 million en 2025**. La **boutique** recule légèrement (593 612 € puis 558 143 €, 560 974 €) tandis que le **site** progresse très fortement (422 440 € puis 502 531 €, 617 715 €) : sa part passe de 37,1 % à 46,6 %, et il dépasse la boutique en 2025. Le site ne « mange » pas (seulement) la boutique : la croissance totale est positive chaque année (+4,4 % puis +11,4 %). C'est l'élément de réponse à la question que la gérante se pose ; il ne la tranche pas (les clients du site sont-ils les mêmes ?), ce que l'application 3.7 permet d'examiner.

À vous. Ajoutez une colonne « part des réseaux » et reproduisez le tableau par **catégorie de produit** au lieu du canal (il faudra joindre produits).

Application 3.3 — Les meilleurs clients (sections 3.2 et 3.4)

Objectif. Aller plus loin que « les dix premiers » du livre : qui sont les meilleurs clients, d'où viennent-ils, et le sont-ils d'une année sur l'autre ?

Étape 1 — Le chiffre d'affaires par client et par ville. On part d'une CTE du chiffre d'affaires 2025 par client, puis on joint les clients pour récupérer leur ville :

```
WITH ca AS (
  SELECT c.id_client, SUM(l.montant) AS ca
  FROM commandes c JOIN lignes_commande l USING (id_commande)
  WHERE c.date_commande >= '2025-01-01' GROUP BY c.id_client)
SELECT cl.ville, COUNT(*) AS clients, ROUND(SUM(ca.ca)) AS ca, ROUND(AVG(ca.ca), 1) AS ca_moyen
```

```
FROM ca JOIN clients cl USING (id_client)
GROUP BY cl.ville ORDER BY ca DESC LIMIT 5
```

ville	clients	ca	ca_moyen
Ville A	532	184324.0	346.5
Ville B	471	164880.0	350.1
Ville C	401	121696.0	303.5
Ville D	364	116870.0	321.1
Ville E	330	102757.0	311.4

Étape 2 — La persistance du top 100. Combien des cent meilleurs clients de 2025 étaient déjà dans les cent meilleurs de 2024 ? INTERSECT répond :

```
WITH ca25 AS (
  SELECT c.id_client, SUM(l.montant) AS ca FROM commandes c JOIN lignes_commande l USING (id_commande)
  WHERE c.date_commande >= '2025-01-01' GROUP BY c.id_client ORDER BY ca DESC LIMIT 100),
ca24 AS (
  SELECT c.id_client, SUM(l.montant) AS ca FROM commandes c JOIN lignes_commande l USING (id_commande)
  WHERE c.date_commande >= '2024-01-01' AND c.date_commande < '2025-01-01' GROUP BY c.id_client ORDER
  BY ca DESC LIMIT 100)
SELECT COUNT(*) AS dans_les_deux_top100 FROM (SELECT id_client FROM ca25 INTERSECT SELECT id_client
FROM ca24)
```

```

dans_les_deux_top100
                28
```

Lecture. Les villes A et B concentrent à elles deux un quart du chiffre d'affaires de 2025 (184 324 € et 164 880 €), avec un chiffre d'affaires moyen par client voisin de 350 € ; les autres villes tournent autour de 300 €. Et **28 clients seulement** sur 100 figurent dans le top 100 des deux années : le « meilleur client » d'une année **n'est pas** celui de l'année suivante. Un top de clients est donc une photographie, pas une caste : **72 clients sur 100 changent** d'une année à l'autre, et bâtir un programme de fidélité sur la liste d'une seule année serait coûteux et peu efficace (c'est ce que l'on appelle la **régression vers la moyenne** : un client exceptionnel une année l'est moins l'année suivante).

À vous. Calculez la part de chaque ville dans le chiffre d'affaires **total** avec une fonction fenêtre (SUM(ca) OVER ()).

Application 3.4 — Panier moyen et double comptage (section 3.2.6)

Objectif. Mettre en pratique le **test des trois comptes** et mesurer précisément les dégâts d'un grain mal compris.

Étape 1 — Combien de commandes ? Trois façons de « compter les commandes » donnent trois résultats différents :

```
SELECT (SELECT COUNT(*) FROM commandes) AS dans_commandes,
       (SELECT COUNT(*) FROM commandes c JOIN lignes_commande l USING (id_commande)) AS apres_jointure,
       (SELECT COUNT(DISTINCT c.id_commande) FROM commandes c JOIN lignes_commande l USING
       (id_commande)) AS apres_jointure_distinct
```

```

dans_commandes  apres_jointure  apres_jointure_distinct
                36395          83905          36395
```

Étape 2 — Panier moyen, par canal et par année. On construit d'abord le grain « une ligne par commande » dans une CTE, puis on moyenne :

```
WITH par_commande AS (
  SELECT c.id_commande, c.canal, strftime('%Y', c.date_commande) AS annee, SUM(l.montant) AS panier
  FROM commandes c JOIN lignes_commande l USING (id_commande) GROUP BY c.id_commande
  SELECT annee, canal, COUNT(*) AS nb_commandes, ROUND(AVG(panier), 2) AS panier_moyen
  FROM par_commande GROUP BY annee, canal ORDER BY annee, canal
```

annee	canal	nb_commandes	panier_moyen
2023	Boutique	5916	100.34
2023	Réseaux	1230	99.90
2023	Site	4272	98.89
2024	Boutique	5617	99.37
2024	Réseaux	1301	98.99
2024	Site	5113	98.28
2025	Boutique	5442	103.08
2025	Réseaux	1426	102.44
2025	Site	6078	101.63

Étape 3 — Le même calcul, faux. Moyenner directement les lignes donne un tout autre nombre :

```
SELECT ROUND(AVG(l.montant), 2) AS montant_moyen_par_ligne,
       ROUND(SUM(l.montant) / COUNT(DISTINCT c.id_commande), 2) AS panier_moyen
FROM commandes c JOIN lignes_commande l USING (id_commande)
```

montant_moyen_par_ligne	panier_moyen
43.54	100.38

Lecture. Les trois comptes sont **36 395** (commandes), **83 905** (après jointure : des lignes) et **36 395** (après jointure, en comptant des commandes distinctes) : la jointure multiplie par 2,3 le nombre de lignes, et seul le **DISTINCT** rend le bon résultat. Le panier moyen est remarquablement **stable** : entre 98 € et 103 € dans les neuf combinaisons année × canal. Le seul saut visible est celui de 2025 (environ +3 %, de 99 € à 102 € en moyenne), qui correspond à la **hausse de prix de 3 % du 1er janvier 2025** (une vérité programmée dans les données) : hors hausse de prix, la croissance du chiffre d'affaires vient du **nombre de commandes**, pas du panier. L'étape 3 oppose le **montant moyen par ligne** (43,54 €) au panier moyen (100,38 €) : la première moyenne répond à « combien coûte un article acheté », la seconde à « combien dépense un client à chaque passage ». Les deux sont justes, à condition de savoir laquelle on cite.

À vous. Pourquoi le panier moyen par ligne est-il plus faible que le prix catalogue moyen ? (Indice : regardez `remise_pct` et le poids des petits prix.)

Application 3.5 — Les retours : combien, d'où, pourquoi (sections 3.2 et 3.4)

Objectif. Quantifier les retours pour la gérante : quels motifs, quels canaux, quelles catégories ?

Étape 1 — Les motifs de retour par canal (les six premiers) :

```
SELECT r.motif, c.canal, COUNT(*) AS nb
FROM retours r JOIN lignes_commande l USING (id_ligne) JOIN commandes c USING (id_commande)
GROUP BY r.motif, c.canal ORDER BY nb DESC LIMIT 6
```

motif	canal	nb
Mauvais choix	Site	1000
Changement d'avis	Site	970
Défaut	Site	572
Mauvais choix	Boutique	382
Livraison tardive	Site	377
Changement d'avis	Boutique	373

Étape 2 — Le taux de retour par canal puis par catégorie, sur les trois années (la jointure à gauche garde les lignes sans retour) :

```
SELECT 'canal' AS axe, c.canal AS modalite, COUNT(*) AS lignes, COUNT(r.id_retour) AS retours,
       ROUND(100.0 * COUNT(r.id_retour) / COUNT(*), 1) AS taux_pct
FROM lignes_commande l JOIN commandes c USING (id_commande) LEFT JOIN retours r ON r.id_ligne =
  ↳ l.id_ligne
GROUP BY c.canal
UNION ALL
SELECT 'catégorie', p.categorie, COUNT(*), COUNT(r.id_retour), ROUND(100.0 * COUNT(r.id_retour) /
  ↳ COUNT(*), 1)
FROM lignes_commande l JOIN produits p USING (id_produit) LEFT JOIN retours r ON r.id_ligne =
  ↳ l.id_ligne
GROUP BY p.categorie
```

axe	modalite	lignes	retours	taux_pct
canal	Boutique	39362	1211	3.1
canal	Réseaux	9071	605	6.7
canal	Site	35472	3186	9.0
catégorie	Bien-être	10347	629	6.1
catégorie	Cuisine	14750	907	6.1
catégorie	Décoration	16575	1001	6.0
catégorie	Jardin	14977	922	6.2
catégorie	Maison	14596	855	5.9
catégorie	Papeterie	12660	688	5.4

Lecture. Les trois premiers motifs sont tous sur le **site** : « mauvais choix » (1 000 retours), « changement d'avis » (970) et « défaut » (572). Le taux de retour **varie très peu d'une catégorie à l'autre** (de 5,4 % pour la papeterie à 6,2 % pour le jardin) mais **beaucoup d'un canal à l'autre** (3,1 % pour la boutique, 6,7 % pour les réseaux, **9,0 % pour le site**, sur les trois années). Le problème des retours est donc un problème de **canal** (vente à distance, impossibilité d'essayer), non de produit. Le tableau le montre avec des chiffres, ce qui évite d'accuser à tort les fournisseurs.

À vous. Calculez le montant remboursé par catégorie et le rapport entre ce montant et le chiffre d'affaires de la catégorie.

Application 3.6 — Évolution mensuelle, cumul et comparaison à l'an dernier (section 3.3)

Objectif. Produire le tableau de bord mensuel que la gérante lira chaque début de mois : le chiffre d'affaires du mois, le même mois un an plus tôt et l'évolution.

Étape 1 — Comparer à l'an dernier avec LAG(..., 12). Les mois étant consécutifs, le même mois de l'année précédente est 12 lignes plus haut :

```
WITH m AS (
  SELECT strftime('%Y-%m', c.date_commande) AS mois, SUM(l.montant) AS ca
  FROM commandes c JOIN lignes_commande l USING (id_commande) GROUP BY mois)
SELECT mois, ROUND(ca) AS ca, ROUND(LAG(ca, 12) OVER (ORDER BY mois)) AS ca_an_dernier,
       ROUND(100.0 * (ca / LAG(ca, 12) OVER (ORDER BY mois) - 1), 1) AS evol_an_pct
FROM m ORDER BY mois DESC LIMIT 6
```

mois	ca	ca_an_dernier	evol_an_pct
2025-12	183845.0	157304.0	16.9
2025-11	143892.0	131726.0	9.2
2025-10	120064.0	101693.0	18.1
2025-09	113453.0	101340.0	12.0
2025-08	89898.0	81188.0	10.7
2025-07	111221.0	93847.0	18.5

Étape 2 — Le cumul depuis janvier, recommencé chaque année. `PARTITION BY` sur l'année recommence le cumul. Le cumul est calculé sur **tous** les mois dans une première étape, et le filtre sur juin et décembre n'intervient qu'ensuite (le piège de la section 3.3.2 : un `WHERE` placé au même niveau que la fenêtre priverait le cumul de ses mois) :

```
WITH m AS (
  SELECT strftime('%Y', c.date_commande) AS annee, strftime('%m', c.date_commande) AS mois,
         SUM(l.montant) AS ca
  FROM commandes c JOIN lignes_commande l USING (id_commande) GROUP BY annee, mois),
cumul AS (SELECT *, SUM(ca) OVER (PARTITION BY annee ORDER BY mois) AS cumul FROM m)
SELECT annee, mois, ROUND(ca) AS ca, ROUND(cumul) AS cumul
FROM cumul WHERE mois IN ('06', '12') ORDER BY mois, annee
```

annee	mois	ca	cumul
2023	06	91604.0	490269.0
2024	06	103445.0	522363.0
2025	06	106930.0	562391.0
2023	12	157052.0	1138932.0
2024	12	157304.0	1189461.0
2025	12	183845.0	1324764.0

Lecture. Sur les six derniers mois de 2025, le chiffre d'affaires est **supérieur à celui de l'année précédente de 9,2 % à 18,5 %** selon le mois (+16,9 % en décembre). Le cumul à fin juin et à fin décembre de chaque année (le cumul de décembre est le total annuel : 1 138 932 €, 1 189 461 € puis 1 324 764 €) montre que le **second semestre pèse plus que le premier** (en 2025, 562 391 € à fin juin contre 762 373 € pour les six mois suivants), en raison de la saisonnalité. Comparer des mois à l'an dernier (et non au mois précédent) est la bonne façon de **neutraliser la saisonnalité**.

À vous. Ajoutez une colonne « rang du mois dans l'année » avec `RANK()` et repérez le meilleur mois de chaque année.

Application 3.7 — Fidélité et cohortes (sections 3.3 et 3.4)

Objectif. Une **cohorte** regroupe les clients selon l'année de leur **première commande** ; on suit ensuite combien de clients de chaque cohorte restent actifs. C'est l'outil de base de l'analyse de fidélité.

```
WITH premiere AS (SELECT id_client, MIN(date_commande) AS d1 FROM commandes GROUP BY id_client),
actif AS (SELECT DISTINCT id_client, strftime('%Y', date_commande) AS an FROM commandes)
SELECT strftime('%Y', p.d1) AS cohorte, COUNT(DISTINCT p.id_client) AS clients,
       SUM(a.an = '2023') AS actifs_2023, SUM(a.an = '2024') AS actifs_2024, SUM(a.an = '2025') AS
       actifs_2025
FROM premiere p JOIN actif a USING (id_client)
GROUP BY cohorte ORDER BY cohorte
```

cohorte	clients	actifs_2023	actifs_2024	actifs_2025
2023	3112	3112	2527	2502
2024	952	0	952	631
2025	742	0	0	742

Lecture. La cohorte « 2023 » (3 112 clients dont la première commande tombe en 2023) compte **2 527 clients actifs en 2024** et **2 502 en 2025** : environ **81 % et 80 %** de rétention, remarquablement stable d'une année à l'autre. La cohorte 2024 (952 clients) en garde 631 en 2025, soit **66 %**. Deux précautions : (1) la cohorte « 2023 » contient aussi les clients **déjà inscrits avant 2023**, dont la vraie première commande est antérieure à la base : on parle de **troncature à gauche**, et leur fidélité est probablement surestimée ; (2) la cohorte 2025 n'a pas encore pu être revue. Ces limites sont la règle en analyse de cohortes : **toujours se demander depuis quand on observe**.

À vous. Exprimez les taux de rétention en pourcentage de la taille de la cohorte, et ajoutez le canal d'acquisition comme deuxième axe.

Application 3.8 — Une segmentation RFM en SQL (sections 3.3 et 3.4)

Objectif. La segmentation **RFM** note chaque client sur la **R**écence (jours depuis la dernière commande), la **F**réquence (nombre de commandes) et le **M**ontant (chiffre d'affaires cumulé), puis regroupe les clients dans des segments parlants. Nous la calculons sur toute la période, au 31 décembre 2025.

Étape 1 — Les trois mesures par client, dans une CTE :

```
CREATE VIEW v_rfm AS
SELECT c.id_client,
       CAST(julianday('2025-12-31') - julianday(MAX(c.date_commande)) AS INTEGER) AS recence_jours,
       COUNT(DISTINCT c.id_commande) AS frequence,
       ROUND(SUM(l.montant), 2) AS montant
FROM commandes c JOIN lignes_commande l USING (id_commande)
GROUP BY c.id_client
```

Étape 2 — Des segments définis par des seuils lisibles. Les seuils (90 et 365 jours, 8 commandes) sont des **choix métier**, que l'on écrit en clair :

```
SELECT CASE WHEN recence_jours <= 90 AND frequence >= 8 THEN '1 champions'
            WHEN recence_jours <= 90 THEN '2 actifs'
            WHEN recence_jours <= 365 THEN '3 à relancer'
            ELSE '4 dormants' END AS segment,
       COUNT(*) AS clients, ROUND(AVG(frequence), 1) AS frequence_moy, ROUND(SUM(montant)) AS ca_total
FROM v_rfm GROUP BY segment ORDER BY segment
```

	segment	clients	frequence_moy	ca_total
1	champions	1326	16.8	2240226.0
2	actifs	1148	3.9	445610.0
3	à relancer	1407	5.3	751237.0
4	dormants	925	2.3	216084.0

Lecture. Les 1 326 « **champions** » (28 % des clients) réalisent **2,24 millions d'euros**, soit **61 %** du chiffre d'affaires cumulé des trois ans ; les 925 « **dormants** » (sans commande depuis plus d'un an) n'en représentent que 6 %. La **vérification** est ici très simple : les quatre segments totalisent **4 806 clients** (tous ceux de la vue) et **3 653 157 €**, c'est-à-dire exactement le chiffre d'affaires de la base : aucun client n'est perdu ni compté deux fois. Les seuils sont des choix : un autre découpage (par exemple des quantiles avec NTILE) donnerait de seuils d'autres segments : une segmentation est un **outil de décision**, pas une vérité.

À vous. Remplacez les seuils par NTILE(3) sur chaque mesure, et observez pourquoi NTILE appliqué à la fréquence (très peu de valeurs distinctes) coupe des clients **identiques** dans des tranches différentes.

Application 3.9 — Index et plan d'exécution (section 3.5)

Objectif. Voir un index changer un plan d'exécution, puis constater qu'il ne sert à rien quand on écrit mal sa condition.

```
def plan(requete):
    return [ligne[3] for ligne in con.execute("EXPLAIN QUERY PLAN " + requete)]

q_ville = "SELECT COUNT(*) FROM clients WHERE ville = 'Ville C'"
print("avant :", plan(q_ville))
```

```
con.execute("CREATE INDEX idx_clients_ville ON clients(ville)")
print("après :", plan(q_ville))
print("avec une fonction :", plan("SELECT COUNT(*) FROM clients WHERE UPPER(ville) = 'VILLE C'"))
print("recherche en tête de motif :", plan("SELECT COUNT(*) FROM clients WHERE ville LIKE 'Ville
↵ C%'"))
```

```
avant : ['SCAN clients']
après : ['SEARCH clients USING COVERING INDEX idx_clients_ville (ville=?)']
avec une fonction : ['SCAN clients USING COVERING INDEX idx_clients_ville']
recherche en tête de motif : ['SCAN clients USING COVERING INDEX idx_clients_ville']
```

Lecture. Avant l'index, la base lit toute la table `clients` (SCAN). Après, elle fait une **recherche par l'index** (SEARCH ... USING COVERING INDEX). En revanche, appliquer `UPPER()` à la colonne indexée **empêche** l'index de servir : on retombe sur un balayage complet, même si l'index existe. Pour `LIKE 'Ville C%'`, SQLite ne fait pas non plus de recherche directe dans l'index : son `LIKE` ignore la casse alors que l'index la respecte, ce qui l'empêche de s'en servir pour une recherche par préfixe. **Lisez toujours le plan**, ne le supposez pas.

À vous. Créez un index sur `commandes(canal, date_commande)` et comparez le plan de la requête « commandes du site en décembre 2025 » avant et après.

Exercices

Exercice 3.1 ★ — Les produits à forte marge (section 3.1.2)

Listez les produits de la catégorie « Maison » dont le **taux de marge** (marge divisée par le prix de vente) dépasse 55 %, triés par taux décroissant, avec leur prix, leur coût et leur taux arrondi à un chiffre après la virgule. Combien y en a-t-il ?

Exercice 3.2 ★ — Une condition composée (section 3.1.3)

Combien de commandes du canal « Réseaux » ont utilisé le code FIDELITE en décembre 2025 ? Écrivez la condition de date **sans** BETWEEN.

Exercice 3.3 ★★ — Les heures de pointe (section 3.1.5)

Classez les commandes en quatre tranches horaires (matin avant 12 h, midi de 12 h à 14 h, après-midi de 14 h à 18 h, soir à partir de 18 h) avec CASE, et comptez-les. Quelle tranche concentre le plus de commandes ? (L'heure est un texte HH:MM : `substr(heure, 1, 2)` donne les deux premiers caractères.)

Exercice 3.4 ★★ — Pourquoi 0 ? (section 3.1.9)

Un collègue écrit, pour calculer la part des commandes avec un code promo, `SELECT SUM(CASE WHEN code_promo <> '' THEN 1 ELSE 0 END) / COUNT(*) FROM commandes` et obtient 0. Expliquez, corrigez, et donnez la part en pourcentage.

Exercice 3.5 ★ — Le chiffre d'affaires par ville (section 3.2.2)

Calculez le chiffre d'affaires **2025** des cinq villes les plus rentables (il faut recoller clients, commandes et lignes_commande). Vérifiez que la somme des chiffres d'affaires de **toutes** les villes retombe sur le total de 2025.

Exercice 3.6 ★★ — Les inscrits sans commande (section 3.2.3)

Combien de clients inscrits en **2024** n'ont **jamais** passé de commande ? Donnez la réponse par une anti-jointure, puis par NOT EXISTS, et vérifiez que les deux comptes coïncident.

Exercice 3.7 ★★ — Un chiffre d'affaires trop beau (section 3.2.6)

Un stagiaire rapproche le chiffre d'affaires journalier de la table jours_exploitation des commandes : `SELECT SUM(j.chiffre_affaires) FROM jours_exploitation j JOIN commandes c ON c.date_commande = j.date`. Que calcule réellement sa requête ? Comparez-la au chiffre d'affaires réel (`SELECT SUM(chiffre_affaires) FROM jours_exploitation`), expliquez l'écart, puis corrigez la requête pour le rapprochement qu'il voulait faire : comparer, **jour par jour**, le chiffre d'affaires de jours_exploitation à celui recalculé depuis les lignes de commande. Quel est l'écart maximal ?

Exercice 3.8 ★★★ — Les clients perdus, par canal d'acquisition (section 3.2.7)

Parmi les clients actifs en 2024, combien n'ont pas commandé en 2025, selon leur **canal d'acquisition** ? Exprimez ce nombre en pourcentage des clients actifs en 2024 de chaque canal d'acquisition. Quel canal fidélise le mieux ?

Exercice 3.9 ★★ — Les trois meilleurs produits par catégorie (section 3.3.3)

Pour chaque catégorie, donnez les **trois produits au plus fort chiffre d'affaires en 2025** avec RANK(). Pourquoi peut-il y avoir plus de trois lignes pour une catégorie ? Que deviendrait le résultat avec ROW_NUMBER() ?

Exercice 3.10 ★★ — Le délai de la deuxième commande (section 3.3.4)

Pour les clients qui ont passé au moins deux commandes, calculez le délai moyen **entre la première et la deuxième commande**, selon leur canal d'acquisition. Les clients acquis par la boutique recommandent-ils plus vite ?

Exercice 3.11 ★★★ — La meilleure semaine (section 3.3.5)

Avec les données de jours_exploitation, calculez le **nombre de commandes sur sept jours glissants** et repérez les trois semaines glissantes les plus chargées. À quelle période de l'année correspondent-elles ?

Exercice 3.12 ★★ — Un organigramme (section 3.4.5)

Soit une équipe de six personnes : la gérante (sans chef), deux responsables (boutique et site, dont le chef est la gérante), deux vendeuses (chef : responsable boutique) et un préparateur (chef : responsable site). Avec une CTE récursive, affichez pour chaque personne son **niveau** hiérarchique et son **chemin** complet, par exemple « Gérante > Responsable boutique > Vendeuse A ».

Exercice 3.13 ★★ — Changer de dialecte (section 3.6)

Traduisez en SQL Server, puis en Oracle, la requête « les cinq produits les plus chers » et la requête « date de la commande + 7 jours » (la colonne `date_commande` est un `DATE`). Vérifiez ensuite la première dans DuckDB, qui est exécutable ici, en lisant le fichier `produits.csv`.

Exercice 3.14 ★★★ — Sous-requête ou fenêtre ? (sections 3.5.1 et 3.5.2)

Pour chaque catégorie, listez le produit le plus cher avec une **sous-requête corrélée**, puis avec une **fonction fenêtre**. Vérifiez que les deux écritures donnent le même résultat, et comparez leurs plans d'exécution (`EXPLAIN QUERY PLAN`). Laquelle préférez-vous, et pourquoi ?

Corrigés

Corrigé 3.1

```
SELECT nom_produit, prix_vente, cout_achat,
       ROUND(100.0 * (prix_vente - cout_achat) / prix_vente, 1) AS taux_marge
FROM produits
WHERE categorie = 'Maison' AND 100.0 * (prix_vente - cout_achat) / prix_vente > 55
ORDER BY taux_marge DESC
```

nom_produit	prix_vente	cout_achat	taux_marge
Rideau design	31.9	12.84	59.7
Boîte rustique	50.9	21.14	58.5
Lampe design	126.9	53.39	57.9
Coussin mat	50.9	21.55	57.7
Étagère compact	33.9	15.21	55.1

Cinq produits de la maison dépassent 55 % de marge. Comme SQLite tolère l'alias dans le `WHERE`, on aurait pu écrire `taux_marge > 55`, mais c'est **non portable** (section 3.1.7) : on répète l'expression. La division par 100.0 (et non 100) n'est pas nécessaire ici, car `prix_vente` est décimal ; c'est un réflexe de prudence.

Corrigé 3.2

```
SELECT COUNT(*) AS commandes
FROM commandes
WHERE canal = 'Réseaux' AND code_promo = 'FIDELITE'
      AND date_commande >= '2025-12-01' AND date_commande < '2026-01-01'
```

```
commandes
13
```

Treize commandes. La borne supérieure **stricte** (`< '2026-01-01'`) vaut mieux qu'un `BETWEEN ... AND '2025-12-31'` : elle reste correcte si la colonne contient un jour une heure (piège 3 de la section 3.1.9).

Corrigé 3.3

```
SELECT CASE WHEN CAST(substr(heure, 1, 2) AS INTEGER) < 12 THEN '1 matin'
            WHEN CAST(substr(heure, 1, 2) AS INTEGER) < 14 THEN '2 midi'
            WHEN CAST(substr(heure, 1, 2) AS INTEGER) < 18 THEN '3 après-midi'
            ELSE '4 soir' END AS tranche, COUNT(*) AS nb
FROM commandes GROUP BY tranche ORDER BY tranche
```

tranche	nb
1 matin	3830
2 midi	5819
3 après-midi	17012
4 soir	9734

L'**après-midi** concentre le plus de commandes (17 012 sur 36 395, soit 47 %), devant le soir (9 734) ; le matin est la tranche la plus creuse (3 830). On a numéroté les tranches (« 1 matin », « 2 midi »...) pour que l'ORDER BY alphabétique donne l'ordre de la journée. Attention : les tranches n'ont pas la même durée (quatre heures d'après-midi, deux heures de midi), la comparaison brute surévalue donc les tranches longues.

Corrigé 3.4

La somme SUM(CASE ... THEN 1 ELSE 0 END) est un **entier**, COUNT(*) aussi : SQLite fait une **division entière** ($5\,743 / 36\,395 = 0,157\dots$, tronqué à 0). La correction force la division décimale :

```
SELECT ROUND(100.0 * SUM(CASE WHEN code_promo <> '' THEN 1 ELSE 0 END) / COUNT(*), 1) AS
↪ part_avec_code_pct
FROM commandes
```

part_avec_code_pct
15.8

La part des commandes avec un code promo est de **15,8 %**. Remarquez que ce chiffre est le même que celui des **lignes** remises vu en 3.1.9 : c'est normal, la remise s'applique à toute la commande.

Corrigé 3.5

```
WITH ca AS (
  SELECT cl.ville, SUM(l.montant) AS ca
  FROM clients cl JOIN commandes c USING (id_client) JOIN lignes_commande l USING (id_commande)
  WHERE c.date_commande >= '2025-01-01' GROUP BY cl.ville)
SELECT ville, ROUND(ca) AS ca_2025, (SELECT ROUND(SUM(ca)) FROM ca) AS total_toutes_villes
FROM ca ORDER BY ca DESC LIMIT 5
```

ville	ca_2025	total_toutes_villes
Ville A	184324.0	1324764.0
Ville B	164880.0	1324764.0
Ville C	121696.0	1324764.0
Ville D	116870.0	1324764.0
Ville E	102757.0	1324764.0

Les cinq villes sont A, B, C, D et E (184 324 €, 164 880 €, 121 696 €, 116 870 € et 102 757 €), et le total de toutes les villes est de **1 324 764 €**, c'est-à-dire exactement le chiffre d'affaires 2025 des autres sections : la jointure n'a ni perdu ni dupliqué de lignes.

Corrigé 3.6

```
SELECT COUNT(*) AS par_anti_jointure
FROM clients c LEFT JOIN commandes o ON o.id_client = c.id_client
WHERE strftime('%Y', c.date_inscription) = '2024' AND o.id_commande IS NULL;
```

```
SELECT COUNT(*) AS par_not_exists
FROM clients c
WHERE strftime('%Y', c.date_inscription) = '2024'
AND NOT EXISTS (SELECT 1 FROM commandes o WHERE o.id_client = c.id_client)
```

```
par_anti_jointure
154
```

```
par_not_exists
154
```

Les deux requêtes renvoient **154 clients** : sur les **666 clients inscrits en 2024**, 154 (23 %, près d'un sur quatre) n'ont jamais commandé. Ici le filtre sur l'inscription porte sur la table **de gauche**, il peut donc rester dans le WHERE sans piège (c'est un filtre sur les lignes finales, pas sur les correspondances).

Corrigé 3.7

La requête du stagiaire **joint** jours_exploitation (un jour = une ligne) à commandes (plusieurs lignes par jour) : le chiffre d'affaires de chaque jour est recopié autant de fois que le jour compte de commandes. Pour comparer :

```
SELECT (SELECT ROUND(SUM(chiffre_affaires)) FROM jours_exploitation) AS ca_reel,
       (SELECT ROUND(SUM(j.chiffre_affaires)) FROM jours_exploitation j JOIN commandes c ON
       ↪ c.date_commande = j.date) AS ca_apres_jointure
```

```
ca_reel  ca_apres_jointure
3653157.0 138730005.0
```

Le chiffre d'affaires réel est de 3 653 157 € ; la jointure en annonce **138 730 005 €**, soit **38 fois** trop, puisque chaque jour compte plus de trente commandes en moyenne. Pour rapprocher correctement les deux sources, on **agrège avant de joindre**, au grain du jour :

```
WITH calcule AS (
  SELECT c.date_commande AS jour, SUM(l.montant) AS ca_lignes
  FROM commandes c JOIN lignes_commande l USING (id_commande) GROUP BY c.date_commande)
SELECT COUNT(*) AS jours, ROUND(MAX(ABS(j.chiffre_affaires - calcule.ca_lignes)), 2) AS ecart_max
FROM jours_exploitation j JOIN calcule ON calcule.jour = j.date
```

```
jours  ecart_max
1096   0.0
```

L'écart maximal est **nul** (0,0 €) sur les 1 096 jours : la table de synthèse jours_exploitation est parfaitement cohérente avec le détail des lignes. C'est exactement le contrôle de réconciliation que l'on fait entre un tableau de bord et ses sources.

Corrigé 3.8

```
WITH actifs24 AS (SELECT DISTINCT id_client FROM commandes WHERE date_commande >= '2024-01-01' AND
↪ date_commande < '2025-01-01'),
perdus AS (SELECT id_client FROM actifs24 EXCEPT SELECT id_client FROM commandes WHERE date_commande
↪ >= '2025-01-01')
```

```
SELECT cl.canal_acquisition, COUNT(*) AS actifs_2024,
       SUM(p.id_client IS NOT NULL) AS perdus,
       ROUND(100.0 * SUM(p.id_client IS NOT NULL) / COUNT(*), 1) AS taux_perte_pct
FROM actifs24 a JOIN clients cl USING (id_client) LEFT JOIN perdus p USING (id_client)
GROUP BY cl.canal_acquisition ORDER BY taux_perte_pct
```

canal_acquisition	actifs_2024	perdus	taux_perte_pct
Réseaux	412	66	16.0
Boutique	1720	328	19.1
Site	1347	257	19.1

Les clients acquis par les **réseaux** sont les mieux fidélisés : **16,0 %** d'entre eux (66 sur 412) n'ont pas recommandé en 2025, contre **19,1 %** pour la boutique (328 sur 1 720) et pour le site (257 sur 1 347). L'écart est de trois points ; or, sur 412 clients, l'incertitude statistique d'un taux d'environ 17 % est de l'ordre de **± 3,6 points** (à 95 %) : la différence n'est donc **pas démontrée**. Une conclusion prudente : le canal d'acquisition ne prédit guère la fidélité (les tests statistiques du volume III de cette série donnent le moyen de trancher). Remarquez l'astuce SUM(p.id_client IS NOT NULL) : après une jointure à gauche, un id_client non nul signifie « client perdu ».

Corrigé 3.9

```
WITH ca AS (
  SELECT p.categorie, p.nom_produit, SUM(l.montant) AS ca
  FROM lignes_commande l JOIN commandes c USING (id_commande) JOIN produits p USING (id_produit)
  WHERE c.date_commande >= '2025-01-01' GROUP BY p.id_produit),
r AS (SELECT *, RANK() OVER (PARTITION BY categorie ORDER BY ca DESC) AS rang FROM ca)
SELECT categorie, nom_produit, ROUND(ca) AS ca, rang FROM r WHERE rang <= 3 ORDER BY categorie, rang
```

categorie	nom_produit	ca	rang
Bien-être	Tisane nordique	12433.0	1
Bien-être	Encens compact	11279.0	2
Bien-être	Peignoir design	10527.0	3
Cuisine	Casserole nordique	25842.0	1
Cuisine	Bol design	21628.0	2
Cuisine	Théière rustique	20885.0	3
Décoration	Statuette rustique	31316.0	1
Décoration	Bougeoir rustique	24527.0	2
Décoration	Vase mat	20436.0	3
Jardin	Transat nordique	66012.0	1
Jardin	Lanterne mat	47447.0	2
Jardin	Pot mat	46421.0	3
Maison	Panier mat	43955.0	1
Maison	Coussin mat	29856.0	2
Maison	Lampe design	28665.0	3
Papeterie	Classeur rustique	7274.0	1
Papeterie	Calendrier design	5968.0	2
Papeterie	Cahier nordique	5595.0	3

Le résultat compte autant de lignes que de produits classés dans les trois premiers rangs : **plus de trois** si deux produits ont **exactement** le même chiffre d'affaires (c'est le propre de RANK, qui garde les ex æquo). Ici, les chiffres d'affaires en euros sont des nombres décimaux quasi uniques et l'on obtient **trois lignes par catégorie**, soit 18 au total (la situation de la section 3.3.3, où des ex æquo apparaissaient pour des **quantités entières**, ne se reproduit pas). Avec ROW_NUMBER(), on aurait **toujours** trois lignes par catégorie, en départageant arbitrairement les ex æquo éventuels : bon pour « exactement trois », mauvais pour « tous les gagnants ». Les classements en euros et en quantités ne coïncident pas toujours : le meilleur produit en quantités n'est pas forcément celui qui rapporte le plus (un article cher se vend moins).

Corrigé 3.10

```
WITH rang AS (
  SELECT cl.canal_acquisition, c.id_client, c.date_commande,
         ROW_NUMBER() OVER (PARTITION BY c.id_client ORDER BY c.date_commande, c.id_commande) AS n
  FROM commandes c JOIN clients cl USING (id_client)),
paire AS (
  SELECT a.canal_acquisition, julianday(b.date_commande) - julianday(a.date_commande) AS delai
  FROM rang a JOIN rang b ON a.id_client = b.id_client AND a.n = 1 AND b.n = 2)
SELECT canal_acquisition, COUNT(*) AS clients, ROUND(AVG(delai), 1) AS delai_moyen_jours
FROM paire GROUP BY canal_acquisition
```

canal_acquisition	clients	delai_moyen_jours
Boutique	1958	146.2
Réseaux	460	145.6
Site	1555	151.1

Les clients acquis par la boutique recommandent après **146 jours** en moyenne, ceux des réseaux après 146 jours, ceux du site après **151 jours** : l'écart est faible (cinq jours sur près de cinq mois). On a numéroté les commandes de chaque client par `ROW_NUMBER`, puis **auto-joint** le rang 1 au rang 2. Ce délai est calculé uniquement sur les clients qui ont **recommandé** (3 973 clients sur 4 806) : les autres, qui n'ont pas recommandé, sont exclus (**biais de survie** : le vrai délai moyen est plus long).

Corrigé 3.11

```
WITH j AS (
  SELECT date, SUM(nb_commandes) OVER (ORDER BY date ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS
  ↪ sur_7_jours,
         ROW_NUMBER() OVER (ORDER BY date) AS rn
  FROM jours_exploitation)
SELECT date AS fin_de_la_fenetre, sur_7_jours FROM j WHERE rn >= 7 ORDER BY sur_7_jours DESC LIMIT 3
```

fin_de_la_fenetre	sur_7_jours
2025-12-04	451
2025-12-02	450
2025-12-26	446

Les trois fenêtres de sept jours les plus chargées se terminent **les 4 et 2 décembre et le 26 décembre 2025** (451, 450 et 446 commandes) : toutes en **décembre**, au pic des fêtes de fin d'année et des promotions. Le filtre `rn >= 7` écarte les six premiers jours, pour lesquels la fenêtre est incomplète (piège de la section 3.3.5).

Corrigé 3.12

```
WITH RECURSIVE equipe(id, nom, chef) AS (VALUES
  (1, 'Gérante', NULL), (2, 'Responsable boutique', 1), (3, 'Vendeuse A', 2),
  (4, 'Vendeuse B', 2), (5, 'Responsable site', 1), (6, 'Préparateur', 5)),
chaîne(id, nom, niveau, chemin) AS (
  SELECT id, nom, 0, nom FROM equipe WHERE chef IS NULL
  UNION ALL
  SELECT e.id, e.nom, c.niveau + 1, c.chemin || ' > ' || e.nom FROM equipe e JOIN chaîne c ON e.chef =
  ↪ c.id)
SELECT niveau, chemin FROM chaîne ORDER BY chemin
```

niveau	chemin
0	Gérante
1	Gérante > Responsable boutique

```

2 Gérante > Responsable boutique > Vendeuse A
2 Gérante > Responsable boutique > Vendeuse B
1           Gérante > Responsable site
2   Gérante > Responsable site > Préparateur

```

La partie initiale sélectionne la racine (la gérante, au niveau 0) ; la partie récursive ajoute à chaque tour les personnes dont le chef vient d'être trouvé, en augmentant le niveau et en prolongeant le chemin par une concaténation (||). On trie par chemin pour obtenir l'ordre de lecture de l'organigramme. La récursion s'arrête d'elle-même, quand plus personne n'a pour chef une personne déjà trouvée.

Corrigé 3.13

```

-- SQL Server – non exécuté
SELECT TOP 5 nom_produit, prix_vente FROM produits ORDER BY prix_vente DESC;
SELECT DATEADD(day, 7, date_commande) AS date_plus_7 FROM commandes;

-- Oracle – non exécuté
SELECT nom_produit, prix_vente FROM produits ORDER BY prix_vente DESC FETCH FIRST 5 ROWS ONLY;
SELECT date_commande + 7 AS date_plus_7 FROM commandes;

```

Et la vérification de la première dans DuckDB, depuis le fichier d'origine :

```

import duckdb
res = duckdb.sql(f"SELECT nom_produit, prix_vente FROM
↳ read_csv('{os.environ['DONNEES']}/produits.csv') ORDER BY prix_vente DESC LIMIT 5").df()
print(res.to_string(index=False))

```

nom_produit	prix_vente
Transat nordique	152.9
Lampe design	126.9
Lanterne mat	117.9
Étagère compact	105.9
Panier mat	104.9

DuckDB renvoie les mêmes cinq produits que la requête SQLite de la section 3.1.4 (le transat à 152,90 € en tête). Les écritures SQL Server et Oracle sont données **de mémoire** : à vérifier dans la documentation de votre version.

Corrigé 3.14

```

SELECT p.categorie, p.nom_produit, p.prix_vente FROM produits p
WHERE p.prix_vente = (SELECT MAX(q.prix_vente) FROM produits q WHERE q.categorie = p.categorie)
ORDER BY p.categorie;

SELECT categorie, nom_produit, prix_vente FROM (
  SELECT categorie, nom_produit, prix_vente, RANK() OVER (PARTITION BY categorie ORDER BY prix_vente
↳ DESC) AS rang
  FROM produits) WHERE rang = 1 ORDER BY categorie

```

categorie	nom_produit	prix_vente
Bien-être	Tisane nordique	68.9
Cuisine	Casserole nordique	57.9
Décoration	Horloge nordique	96.9
Jardin	Transat nordique	152.9
Maison	Lampe design	126.9
Papeterie	Trousse mat	21.9
categorie	nom_produit	prix_vente

Bien-être	Tisane nordique	68.9
Cuisine	Casserole nordique	57.9
Décoration	Horloge nordique	96.9
Jardin	Transat nordique	152.9
Maison	Lampe design	126.9
Papeterie	Trousse mat	21.9

Les deux requêtes renvoient les mêmes six lignes (on peut le vérifier en les comparant avec EXCEPT dans les deux sens : aucun écart). Voici les plans :

```
corr = "SELECT p.categorie FROM produits p WHERE p.prix_vente = (SELECT MAX(q.prix_vente) FROM
↳ produits q WHERE q.categorie = p.categorie)"
fen = "SELECT categorie FROM (SELECT categorie, RANK() OVER (PARTITION BY categorie ORDER BY
↳ prix_vente DESC) AS rang FROM produits) WHERE rang = 1"
for nom, q in (("corrélée", corr), ("fenêtre", fen)):
    print(nom, plan(q))
```

```
corrélée ['SCAN p', 'CORRELATED SCALAR SUBQUERY 1', 'SEARCH q']
fenêtre ['CO-ROUTINE (subquery-1)', 'CO-ROUTINE (subquery-3)', 'SCAN produits', 'USE TEMP B-TREE FOR
↳ ORDER BY', 'SCAN (subquery-3)', 'SCAN (subquery-1)']
```

La version corrélée **relit** la table produits pour chaque produit (on voit une sous-requête CORRELATED SCALAR SUBQUERY dans le plan) ; la version fenêtre fait **un seul passage** avec un tri. Sur 120 produits la différence est invisible ; sur des millions de lignes, elle est considérable. Je préfère la fenêtre : elle est plus **lisible** (la logique « classer dans la catégorie, garder le rang 1 » se lit), plus **rapide** à grande échelle, et plus **souple** (garder les trois premiers, comparer à la médiane, etc.).

Chapitre 4 : Python et R pour l'analyse — exercices et applications

Comment utiliser ce chapitre. Les **applications** sont de petites études guidées sur les données de la boutique : lisez l'énoncé, essayez, puis comparez avec le code et la lecture proposés. Les **exercices** (★ facile, ★★ moyen, ★★★ difficile) renvoient chacun à une section du livre ; leurs **corrigés** sont à la fin. Chaque bloc de code est court et peut être exécuté tel quel. Tout est **simulé** : la boutique n'existe pas. Le chapitre est **autonome** : il recharge ses données et ses bibliothèques.

Applications

Application 4.1 — Premier contact avec les commandes (sections 4.1.2 à 4.1.4)

Objectif. Lire une table avec pandas, la regarder, la découper. **Question.** Sur quelle période les commandes s'étaient-elles, combien de clients différents ont commandé, et combien de commandes par an ?

Étape 1 — Lire et regarder. On lit `commandes.csv` en précisant la colonne de dates.

```
commandes = pd.read_csv("donnees/commandes.csv", parse_dates=["date_commande"])
print(commandes.shape)
print(commandes["date_commande"].min().date(), commandes["date_commande"].max().date())
print(commandes["id_client"].nunique())
```

```
(36395, 7)
2023-01-01 2025-12-31
4806
```

Étape 2 — Compter par année. On extrait l'année de la date avec `dt.year`, puis on compte et on trie par année.

```
print(commandes["date_commande"].dt.year.value_counts().sort_index())
```

```
date_commande
2023      11418
2024      12031
2025      12946
Name: count, dtype: int64
```

Étape 3 — Sélectionner. Les trois premières commandes du Site, avec trois colonnes seulement.

```
print(commandes.loc[commandes["canal"] == "Site", ["id_commande", "date_commande",
↪ "code_promo"]].head(3))
```

```
   id_commande  date_commande  code_promo
0             1    2023-01-01         NaN
8             9    2023-01-01    FIDELITE
10            11    2023-01-01         NaN
```

Lecture. La table compte 36 395 commandes, du 1er janvier 2023 au 31 décembre 2025, passées par 4 806 clients différents (sur 6 000 inscrits : tous n'ont pas commandé pendant la période). Le nombre de commandes par an passe de 11 418 à 12 031 puis 12 946, soit environ +5 % puis +8 %. Les numéros de ligne 0, 8 et 10 affichés à gauche des commandes du Site rappellent que `.loc` conserve les **étiquettes** de la table d'origine.

Application 4.2 — Les retours (sections 4.1.5 à 4.1.7)

Objectif. Filtrer, compter, repérer les manquants. **Question.** Quels sont les motifs de retour, et combien de retours dépassent 100 € ?

Étape 1 — Lire et résumer. `describe` donne les statistiques de la colonne des montants remboursés.

```
retours = pd.read_csv("donnees/retours.csv", parse_dates=["date_retour"])
print(retours["montant_rembourse"].describe().round(1))
print(retours.isna().sum().sum())
```

```
count    5002.0
mean      44.2
std       39.9
min        2.3
25%       18.9
50%       32.9
75%       56.6
max      472.5
Name: montant_rembourse, dtype: float64
0
```

Étape 2 — Les motifs. Les proportions de chaque motif, en pourcentage.

```
print((retours["motif"].value_counts(normalize=True) * 100).round(1))
```

```
motif
Mauvais choix      31.4
Changement d'avis  30.3
Défaut             18.2
Livraison tardive  12.1
Autre               8.0
Name: proportion, dtype: float64
```

Étape 3 — Filtrer. Les retours de 100 € ou plus, puis ceux de 100 € ou plus **et** motivés par un défaut.

```
gros = retours[retours["montant_rembourse"] >= 100]
print(len(gros), len(gros[gros["motif"] == "Défaut"]))
```

```
437 83
```

Lecture. Les 5 002 retours n'ont aucune valeur manquante. Un retour rembourse en moyenne 44,2 €, mais la **médiane** est de 32,9 € et le maximum de 472,5 € : quelques gros remboursements tirent la moyenne vers le haut (c'est le signe d'une distribution asymétrique, vue en 1.1). « Mauvais

choix » (31,4 %) et « Changement d'avis » (30,3 %) représentent six retours sur dix ; les défauts de produit n'en sont que 18,2 %. Au total, 437 retours dépassent 100 €, dont 83 pour un défaut.

Application 4.3 — Le même travail en R (section 4.2)

Objectif. Traduire les applications 4.1 et 4.2 en tidyverse, et vérifier que les chiffres coïncident.

Étape 1 — Les commandes. Le nombre de lignes, le nombre de clients distincts, les commandes par année.

```
commandes <- read_csv("donnees/commandes.csv", show_col_types = FALSE)
cat(nrow(commandes), n_distinct(commandes$id_client), "\n")
par_annee <- commandes |> count(annee = year(date_commande))
par_annee
```

```
36395 4806
# A tibble: 3 × 2
  annee     n
  <dbl> <int>
1  2023 11418
2  2024 12031
3  2025 12946
```

Étape 2 — Les retours. Les motifs en pourcentage, et le nombre de retours d'au moins 100 €.

```
retours <- read_csv("donnees/retours.csv", show_col_types = FALSE)
motifs <- retours |> count(motif, sort = TRUE) |> mutate(pct = round(100 * n / sum(n), 1))
motifs
cat(sum(retours$montant_rembourse >= 100), "\n")
```

```
# A tibble: 5 × 3
  motif              n  pct
  <chr>          <int> <dbl>
1 Mauvais choix    1571  31.4
2 Changement d'avis 1518  30.3
3 Défaut           908  18.2
4 Livraison tardive 605  12.1
5 Autre            400   8
437
```

Étape 3 — Vérifier. Les deux langages donnent-ils les mêmes comptes ? Le bloc caché écrit les résultats de pandas et les compare ; son verdict s'affiche ci-dessous.

```
identique au résultat de pandas : TRUE
identique au résultat de pandas : TRUE
```

Lecture. Les deux langages donnent les mêmes comptes (36 395 lignes, 4 806 clients, les mêmes effectifs par année, 437 retours d'au moins 100 €). Notez deux différences de forme : `count(annee = year(date_commande))` crée la variable à l'intérieur de `count`, et R appelle `n` la colonne des effectifs.

Application 4.4 — Remettre en état l'export de caisse (section 4.3.1)

Objectif. Transformer un fichier fait pour être lu par un humain en une table exploitable, et le

réconcilier. Question. Quel est le chiffre d'affaires de la semaine par catégorie, et quels sont les cinq articles les plus vendus ?

Étape 1 — Lire en sautant les titres. Tout est lu en texte (les lignes parasites empêchent toute conversion).

```
brut = pd.read_csv("donnees/export_caisse_brut.csv", sep=";", encoding="cp1252", skiprows=3)
print(brut.shape)
```

```
(285, 8)
```

Étape 2 — Écartier les lignes parasites et convertir.

```
caisse = brut[brut["N° ticket"].notna() & (brut["N° ticket"] != "N° ticket")].copy()
caisse["Date"] = pd.to_datetime(caisse["Date"], format="%d/%m/%Y")
for col in ["Qté", "Prix unitaire", "Montant"]:
    caisse[col] = pd.to_numeric(caisse[col].str.replace(",", "."))
caisse["Montant"] = caisse["Montant"].fillna((caisse["Qté"] * caisse["Prix unitaire"]).round(2))
caisse["Catégorie"] = caisse["Catégorie"].str.capitalize()
caisse["Article"] = caisse["Article"].str.capitalize()
print(len(caisse), round(caisse["Montant"].sum(), 2))
```

```
280 11564.09
```

Étape 3 — Répondre à la question.

```
print(caisse.groupby("Catégorie")["Montant"].sum().round(2).sort_values(ascending=False))
print(caisse.groupby("Article")["Qté"].sum().sort_values(ascending=False).head(5))
```

```
Catégorie
Maison      3033.93
Décoration  2882.50
Cuisine     2441.00
Jardin      1482.25
Bien-être   1114.11
Papeterie    610.30
Name: Montant, dtype: float64
Article
Bougie nordique    16
Huile mat          15
Cadre design       11
Bol design         11
Lampe design       11
Name: Qté, dtype: int64
```

Étape 4 — Réconcilier. Le total du fichier est de 11 561,47 € : où se situe l'écart ?

```
total_fichier = float(brut.iloc[-1]["Montant"].replace(",", "."))
print(round(caisse["Montant"].sum() - total_fichier, 2))
```

```
2.62
```

Lecture. Sur les 280 ventes retenues, le chiffre d'affaires reconstitué est de 11 564,09 €, soit 2,62 € de plus que le total du fichier : c'est la remise de 5 % d'une ligne dont le montant manquait (voir 4.3.1). Par catégorie, la maison arrive en tête (3 033,93 €), devant la décoration (2 882,50 €) et la cuisine (2 441,00 €) ; la papeterie ferme la marche (610,30 €). L'article le plus vendu est la bougie nordique (16 pièces), devant l'huile mat (15).

Application 4.5 — Jointures et marge par produit (sections 4.3.2 et 4.3.3)

Objectif. Relier trois tables et calculer des marges. **Question.** Quels sont les cinq produits qui rapportent la plus grande **marge** totale, et la marge dépend-elle du canal ?

Étape 1 — Construire la table de ventes. Lignes, commandes et produits, avec `validate` pour garantir qu'aucune ligne n'est multipliée.

```
lignes = pd.read_csv("donnees/lignes_commande.csv")
produits = pd.read_csv("donnees/produits.csv")
ventes = (lignes.merge(commandes, on="id_commande", validate="m:1")
          .merge(produits[["id_produit", "nom_produit", "categorie", "cout_achat"]],
                on="id_produit", validate="m:1"))
print(len(lignes), len(ventes))
```

```
83905 83905
```

Étape 2 — La marge de chaque ligne. Marge = montant payé – quantité × coût d'achat.

```
ventes["marge"] = ventes["montant"] - ventes["quantite"] * ventes["cout_achat"]
top = ventes.groupby("nom_produit")["marge"].sum().sort_values(ascending=False).head(5)
print(top.round(0))
```

```
nom_produit
Transat nordique      96041.0
Jardinière design     73806.0
Statuette rustique    68237.0
Lanterne mat          59265.0
Lampe design          58435.0
Name: marge, dtype: float64
```

Étape 3 — La marge par canal.

```
par_canal = ventes.groupby("canal")[["marge", "montant"]].sum()
print((par_canal["marge"] / par_canal["montant"] * 100).round(1))
```

```
canal
Boutique      47.4
Réseaux       47.3
Site          47.4
dtype: float64
```

Lecture. La jointure garde les 83 905 lignes : aucune n'a été multipliée. Les cinq produits à la plus grande marge totale sont le transat nordique (96 041 €), la jardinière design (73 806 €), la statuette rustique (68 237 €), la lanterne mat (59 265 €) et la lampe design (58 435 €) ; trois d'entre eux sont des articles de jardin. Le **taux** de marge est le même dans les trois canaux (47,3 à 47,4 %) : le canal change le volume, pas la marge unitaire.

Application 4.6 — Restructurer : le chiffre d'affaires de chaque canal, mois par mois (section 4.3.4)

Objectif. Passer du format long au format large et inversement, puis calculer une part. **Question.** Quelle part du chiffre d'affaires mensuel de 2025 le Site représente-t-il, et en quel mois est-elle la plus élevée ?

Étape 1 — Le format large. Un tableau mois × canal pour 2025.

```
v25 = ventes[ventes["date_commande"].dt.year == 2025].assign(mois=lambda d:
    d["date_commande"].dt.month)
large = v25.pivot_table(index="mois", columns="canal", values="montant", aggfunc="sum").round(0)
print(large.head(3))
```

canal	Boutique	Réseaux	Site
mois			
1	38882.0	8938.0	41358.0
2	33079.0	5479.0	34084.0
3	39039.0	8440.0	42308.0

Étape 2 — La part du Site.

```
part_site = (large["Site"] / large.sum(axis=1) * 100).round(1)
print(part_site.idxmax(), part_site.max(), part_site.idxmin(), part_site.min())
```

```
9 49.0 8 41.5
```

Étape 3 — Revenir au format long, comme le demanderait un graphique.

```
long = large.reset_index().melt(id_vars="mois", var_name="canal", value_name="ca")
print(long.shape, long.head(2))
```

	mois	canal	ca
0	1	Boutique	38882.0
1	2	Boutique	33079.0

Étape 4 — En R, avec pivot_wider puis pivot_longer.

```
ventes_r <- read_csv("donnees/lignes_commande.csv", show_col_types = FALSE) |>
  left_join(read_csv("donnees/commandes.csv", show_col_types = FALSE), by = "id_commande")
large_r <- ventes_r |> filter(year(date_commande) == 2025) |> group_by(mois = month(date_commande),
  ↪ canal) |>
  summarise(ca = round(sum(montant)), .groups = "drop") |> pivot_wider(names_from = canal,
  ↪ values_from = ca)
long_r <- pivot_longer(large_r, -mois, names_to = "canal", values_to = "ca")
cat(dim(long_r), "\n")
```

```
36 3
```

```
identique au résultat de pandas : TRUE
```

Lecture. En janvier 2025, la boutique réalise 38 882 €, les réseaux 8 938 € et le Site 41 358 €. La part du Site est la plus élevée en septembre (49,0 %) et la plus basse en août (41,5 %). Le format long compte 36 lignes (12 mois × 3 canaux), et la version R, avec pivot_wider puis pivot_longer, donne un tableau identique.

Application 4.7 — Le tableau du lundi, en fonction (sections 4.3.5 et 4.3.6)

Objectif. Écrire une fonction qui, à partir d'une **date du jour**, produit le tableau de la semaine qui vient de s'achever, et qui refuse de répondre si la semaine n'est pas complète. **Question.** Que renvoie-t-elle le lundi 10 novembre 2025 ? Et le lundi 5 janvier 2026 ?

Étape 1 — Les ventes par semaine ISO.

```
iso = ventes["date_commande"].dt.isocalendar()
ventes = ventes.assign(annee_iso=iso["year"].astype(int), semaine=iso["week"].astype(int))
sem = ventes.groupby(["annee_iso", "semaine", "canal"], as_index=False)["montant"].sum()
```

Étape 2 — La semaine terminée, pour une date donnée. On retranche sept jours, on lit l'année et la semaine ISO, et on vérifie que les données vont jusqu'au dimanche.

```
def semaine_terminee(date):
    d = pd.Timestamp(date) - pd.Timedelta(days=7)
    iso = d.isocalendar()
```

```

dimanche = d + pd.Timedelta(days=6 - d.weekday())
if ventes["date_commande"].max() < dimanche:
    raise ValueError(f"semaine {iso.year}-{iso.week:02d} incomplète : les données s'arrêtent au
    ↳ {ventes['date_commande'].max().date()}")
return int(iso.year), int(iso.week)

```

Étape 3 — Le tableau. On réutilise la fonction `tableau_du_lundi` du livre (4.3.6), reprise ici.

```

def tableau_du_lundi(annee, semaine):
    cur = sem[(sem.annee_iso == annee) & (sem.semaine == semaine)].set_index("canal")["montant"]
    pre = sem[(sem.annee_iso == annee - 1) & (sem.semaine == semaine)].set_index("canal")["montant"]
    t = pd.DataFrame({"cette_semaine": cur, "meme_semaine_an_dernier": pre})
    t.loc["Total"] = t.sum()
    t["evolution_pct"] = (t["cette_semaine"] / t["meme_semaine_an_dernier"] - 1) * 100
    return t.round(1)

```

```

annee, semaine = semaine_terminee("2025-11-10")
print(annee, semaine)
print(tableau_du_lundi(semaine, annee))

```

```

2025 45
      cette_semaine  meme_semaine_an_dernier  evolution_pct
canal
Boutique          11561.5              12783.9           -9.6
Réseaux            4581.9              2856.3           60.4
Site              14096.7              12089.2           16.6
Total             30240.0              27729.4            9.1

```

Étape 4 — Le cas limite.

```

try:
    semaine_terminee("2026-01-05")
except ValueError as e:
    print("refus :", e)

```

```

refus : semaine 2026-S01 incomplète : les données s'arrêtent au 2025-12-31

```

Lecture. Le lundi 10 novembre 2025, la fonction renvoie la semaine 45 de 2025 et le même tableau que celui du livre. Le lundi 5 janvier 2026, elle **refuse** : la semaine ISO 2026-S01, du 29 décembre au 4 janvier, n'est connue que jusqu'au 31 décembre. Une fonction qui refuse vaut mieux qu'une fonction qui livre, sans le dire, un tableau fondé sur une semaine incomplète.

Application 4.8 — Un notebook reproductible (section 4.4)

Objectif. Construire un notebook par programme, l'exécuter dans un noyau neuf, vérifier qu'il se reproduit, puis y introduire un état caché et le détecter. **Question.** Le notebook « ventes par canal » donne-t-il le même résultat deux fois de suite ?

Étape 1 — Les cellules.

```

cellules = [
    ("md", "# Commandes par canal"),
    ("code", "import pandas as pd\nnc = pd.read_csv('donnees/commandes.csv')"),
    ("code", "res = c['canal'].value_counts().sort_index()\nprint(res.to_dict())"),
]

```

Étape 2 — Exécuter deux fois, comparer.

```

nb_a, err_a = 0.executer_notebook(cellules, os.getcwd())
nb_b, err_b = 0.executer_notebook(cellules, os.getcwd())
print(0.sorties_texte(nb_a)[-1])

```

```
print(0.sorties_texte(nb_a) == 0.sorties_texte(nb_b), err_a, err_b)
```

```
{'Boutique': 16975, 'Réseaux': 3957, 'Site': 15463}
True None None
```

Étape 3 — Un état caché. On ajoute une cellule qui utilise une variable définie **plus bas**. Exécutée de haut en bas dans un noyau neuf, elle échoue.

```
cellules_bug = cellules[:2] + [("code", "print(len(c) * coefficient)"), ("code", "coefficient = 2")]
_, erreur = 0.executer_notebook(cellules_bug, os.getcwd())
print(erreur)
```

```
NameError: name 'coefficient' is not defined
```

Lecture. Le notebook donne deux fois le même résultat ({'Boutique': 16975, 'Réseaux': 3957, 'Site': 15463}) et sans erreur : c'est le test de reproductibilité de base. La version avec la variable définie **après** son usage échoue avec une `NameError` dans un noyau neuf. Dans une session où la dernière cellule avait déjà été exécutée, l'erreur serait passée inaperçue : c'est l'état caché de 4.4.2.

Application 4.9 — Un graphique avec ggplot2 (section 4.5)

Objectif. Produire un graphique lisible avec trois couches. **Question.** Comment les commandes hebdomadaires de 2025 se comparent-elles à celles de 2024, canal par canal ?

Étape 1 — Préparer les données (format long).

```
hebdo <- ventes_r |> mutate(annee_iso = isoyear(date_commande), semaine = isoweek(date_commande)) |>
  filter(annee_iso %in% c(2024, 2025), semaine <= 52) |>
  group_by(annee_iso, semaine, canal) |> summarise(ca = sum(montant) / 1000, .groups = "drop")
cat(nrow(hebdo), "\n")
```

```
312
```

Étape 2 — Le graphique. Une facette par canal, une couleur par année.

```
g <- ggplot(hebdo, aes(x = semaine, y = ca, colour = factor(annee_iso))) +
  geom_line(linewidth = 0.8) +
  facet_wrap(~canal, ncol = 1, scales = "free_y") +
  scale_colour_manual(values = c("2024" = "#898781", "2025" = "#2a78d6")) +
  labs(x = "semaine ISO", y = "milliers d'€", colour = NULL) +
  theme_minimal(base_size = 10) + theme(legend.position = "top")
ggsave("figures/ch04-c-hebdo.png", g, width = 7.5, height = 5.2, dpi = 200)
```

Étape 3 — Compter les semaines gagnantes. Dans combien de semaines sur 52 l'année 2025 devance-t-elle 2024, canal par canal ?

```
hebdo |> pivot_wider(names_from = annee_iso, values_from = ca) |>
  group_by(canale) |> summarise(semaines_2025_devant = sum(`2025` > `2024`))
```

```
# A tibble: 3 × 2
  canal   semaines_2025_devant
  <chr>         <int>
1 Boutique             22
2 Réseaux              28
3 Site                44
```

Lecture. L'année 2025 devance 2024 pendant 44 semaines sur 52 sur le Site, 28 sur les réseaux et seulement 22 en boutique : on retrouve, semaine par semaine, le diagnostic annuel de 4.3.2 (le Site gagne, la boutique recule). L'échelle verticale est libre (`scales = "free_y"`) : elle laisse voir

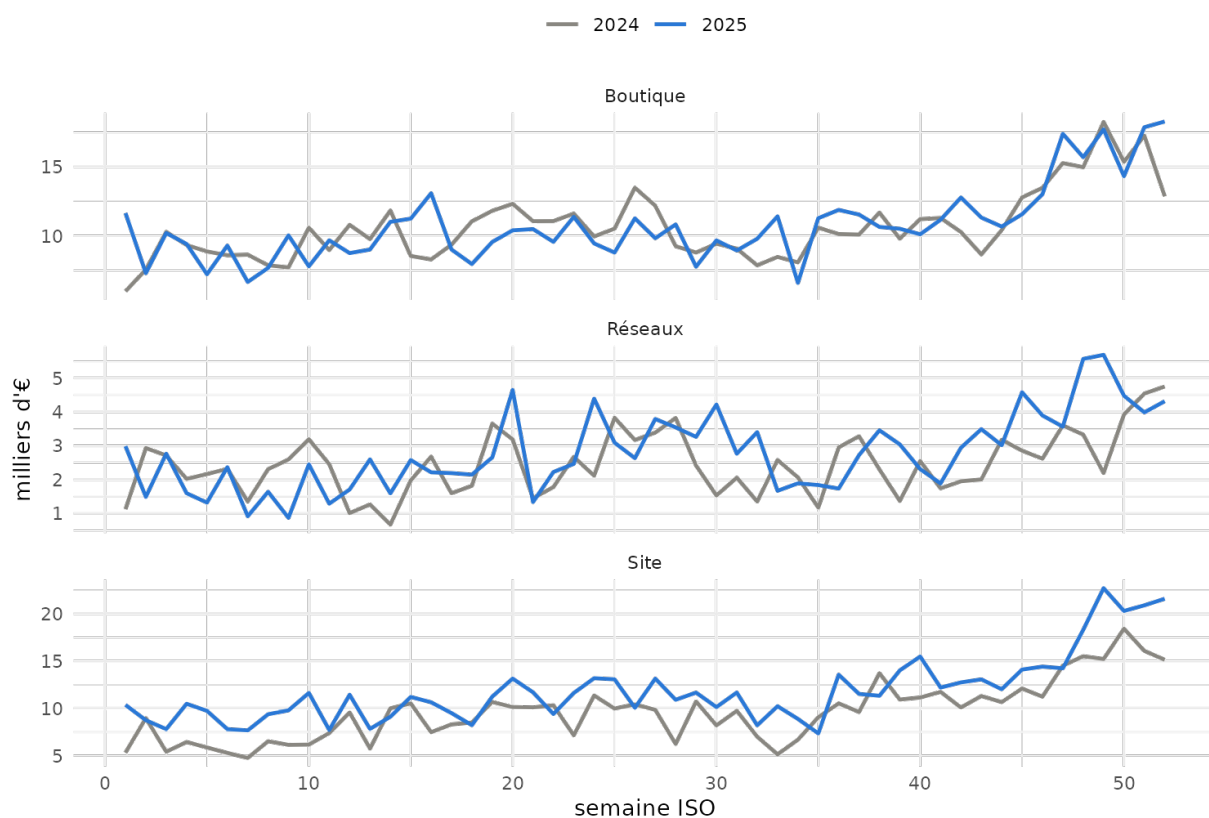


Figure 1: Chiffre d'affaires hebdomadaire par canal (milliers d'€), 2025 contre 2024. Attention : l'échelle verticale diffère d'une facette à l'autre.

la forme de chaque courbe, mais interdit de comparer la hauteur d'une facette à celle d'une autre.

Application 4.10 — polars et NumPy (section 4.6)

Objectif. Refaire un calcul avec polars (en mode paresseux) et un autre avec NumPy, et les confronter à pandas. **Question.** Combien de commandes tombent dans chaque tranche de montant (moins de 50 €, de 50 à 150 €, plus de 150 €) ?

Étape 1 — Avec polars, en mode paresseux.

```
import polars as pl
pl.Config.set_tbl_formatting("MARKDOWN")
pl.Config.set_tbl_hide_dataframe_shape(True)
paniers_pl = (pl.scan_csv("donnees/lignes_commande.csv").group_by("id_commande").agg(panier=pl.col(
↳ "montant").sum()))
    .with_columns(tranche=pl.when(pl.col("panier") < 50).then(pl.lit("1 : moins de 50 €"))
    .when(pl.col("panier") <= 150).then(pl.lit("2 : de 50 à 150
↳ €")).otherwise(pl.lit("3 : plus de 150 €")))
    .group_by("tranche").agg(commandes=pl.len()).sort("tranche").collect())
print(paniers_pl)
```

tranche	commandes
---	---
str	u32
-----	-----
1 : moins de 50 €	10874
2 : de 50 à 150 €	17990
3 : plus de 150 €	7531

Étape 2 — Avec NumPy et pandas, par le « si » vectorisé.

```
panier = lignes.groupby("id_commande")["montant"].sum().to_numpy()
tranche = np.where(panier < 50, "1", np.where(panier <= 150, "2", "3"))
comptes = pd.Series(tranche).value_counts().sort_index()
print(comptes.to_dict(), (comptes.to_numpy() == paniers_pl["commandes"].to_numpy()).all())
```

```
{'1': 10874, '2': 17990, '3': 7531} True
```

Lecture. Les deux calculs donnent les mêmes effectifs : 10 874 commandes de moins de 50 € (30 %), 17 990 de 50 à 150 € (49 %) et 7 531 de plus de 150 € (21 %). Le mode paresseux de polars n'a rien exécuté avant `collect()` ; le « si » vectorisé de NumPy remplace une boucle sur 36 395 commandes.

Exercices

Exercice 4.1 ★ — Types et lecture (section 4.1.3)

Lisez `clients.csv` **sans** puis **avec** l'argument `parse_dates=["date_inscription"]`. Quel est le type de `date_inscription` dans chaque cas ? Combien de clients se sont inscrits en 2024 ? Quelle est l'année de naissance médiane ?

Exercice 4.2 ★ — `.loc` contre `.iloc` (section 4.1.4)

Sur la table `commandes`, combien de lignes renvoient `commandes.loc[0:4]` et `commandes.iloc[0:4]` ? Expliquez la différence. Que renvoie `commandes.loc[commandes["canal"] == "Site"].iloc[0]["id_commande"]` ?

Exercice 4.3 ★ — Filtres combinés (section 4.1.5)

Combien de commandes ont été passées **sur le Site, en 2024**, avec une livraison à **domicile** et un code promotionnel renseigné ? Écrivez le filtre avec des booléens, puis avec query, et vérifiez que les deux comptes sont égaux.

Exercice 4.4 ★★ — Colonnes calculées (section 4.1.6)

Dans `lignes_commande.csv`, ajoutez la colonne `remise_eur` (le montant de la remise de chaque ligne, en euros). Quelle part des lignes a une remise ? Quel est le montant total des remises accordées sur trois ans ?

Exercice 4.5 ★ — Manquants et comptages (section 4.1.7)

Dans `commandes`, quelle proportion de commandes n'a **aucun** code promotionnel, canal par canal ? Remplacez les manquants par « aucun » et vérifiez que la somme des effectifs par code égale le nombre de commandes.

Exercice 4.6 ★ — Traduire en tidyverse (section 4.2)

Écrivez en R, avec le tube : les **trois villes** qui comptent le plus de clients inscrits **avant 2023** (utilisez `clients.csv`). Écrivez ensuite la version pandas et comparez.

Exercice 4.7 ★★ — Panier moyen par canal et par année (section 4.3.2)

Calculez le **panier moyen** (chiffre d'affaires divisé par le nombre de commandes) pour chaque canal et chaque année, puis la **variation** du panier moyen entre 2023 et 2025. Le panier moyen a-t-il changé de manière notable ?

Exercice 4.8 ★★ — Taux de retour par mode de livraison (section 4.3.3)

En reliant `lignes_commande`, `commandes` et `retours`, calculez le **taux de retour** (part des lignes retournées) par **mode de livraison**. Comment l'expliquez-vous ? Vérifiez que les jointures n'ont pas modifié le nombre de lignes.

Exercice 4.9 ★★ — Large et long (section 4.3.4)

Construisez le tableau **catégorie × trimestre** du chiffre d'affaires de 2025 (format large), puis son format long, et calculez la **part de chaque trimestre** dans le chiffre d'affaires annuel de chaque catégorie. Quelle catégorie est la plus concentrée sur le quatrième trimestre ?

Exercice 4.10 ★★ — Moyenne glissante (section 4.3.5)

Dans `jours_exploitation.csv`, calculez une moyenne glissante du nombre de commandes sur **28 jours**. À quelle date est-elle maximale ? En quoi la moyenne glissante sur 7 jours donne-t-elle une date différente ?

Exercice 4.11 ★★★ — L'état caché (section 4.4)

Un notebook contient trois cellules : (1) $x = 10$, (2) $y = x * 2$, (3) $x = 100$ suivie de `print(y)`. Quelle valeur affiche la dernière cellule quand le notebook est exécuté **de haut en bas** ? Et si l'on exécute (1), (2), (3), puis qu'on **réexécute** (2) **seule** avant de relire l'écran ? Vérifiez avec `executer_notebook`.

Exercice 4.12 ★★★ — Année du calendrier contre année ISO (sections 4.3.5 et 4.3.6)

Combien de commandes ont une **année ISO différente** de leur année du calendrier ? Donnez les dates concernées et leur chiffre d'affaires. Que se passerait-il si l'on regroupait par (année du calendrier, semaine ISO) pour construire le tableau du lundi ?

Exercice 4.13 ★★ — polars (section 4.6)

Avec polars en mode paresseux, calculez, pour chaque canal, le **nombre de clients distincts** ayant commandé en 2025, et comparez avec pandas.

Corrigés

Corrigé 4.1

```
cl_texte = pd.read_csv("donnees/clients.csv")
cl_dates = pd.read_csv("donnees/clients.csv", parse_dates=["date_inscription"])
print(cl_texte["date_inscription"].dtype, cl_dates["date_inscription"].dtype)
print((cl_dates["date_inscription"].dt.year == 2024).sum(), cl_dates["annee_naissance"].median())
```

```
str datetime64[us]
666 1982.0
```

Sans `parse_dates`, `date_inscription` est du texte (`str`) ; avec, c'est une date (`datetime64`). 666 clients se sont inscrits en 2024 : ce comptage est direct avec une vraie date (`dt.year`). L'année de naissance médiane est 1982, soit un âge médian d'environ 43 ans en 2025.

Corrigé 4.2

```
cmd = pd.read_csv("donnees/commandes.csv")
print(len(cmd.loc[0:4]), len(cmd.iloc[0:4]))
print(cmd.loc[cmd["canal"] == "Site"].iloc[0]["id_commande"])
```

```
5 4
1
```

`loc[0:4]` renvoie 5 lignes (étiquettes 0 à 4, fin incluse), `iloc[0:4]` en renvoie 4 (positions 0 à 3, fin exclue). La première commande du Site est la numéro 1 : `.iloc[0]` compte à partir de la première ligne **du résultat filtré**, pas de la table d'origine.

Corrigé 4.3

```
cm = pd.read_csv("donnees/commandes.csv", parse_dates=["date_commande"])
c1 = cm[(cm["canal"] == "Site") & (cm["date_commande"].dt.year == 2024) & (cm["mode_livraison"] ==
↳ "Domicile") & cm["code_promo"].notna()]
c2 = cm.query("canal == 'Site' and date_commande.dt.year == 2024 and mode_livraison == 'Domicile' and
↳ code_promo.notna()")
print(len(c1), len(c2))
```

```
463 463
```

Les deux écritures donnent 463 commandes. La version booléenne est plus verbeuse mais fonctionne partout ; query est plus lisible quand les conditions sont nombreuses.

Corrigé 4.4

```
lg = pd.read_csv("donnees/lignes_commande.csv")
lg["remise_eur"] = lg["quantite"] * lg["prix_unitaire"] * lg["remise_pct"] / 100
print(round((lg["remise_eur"] > 0).mean() * 100, 1), round(lg["remise_eur"].sum(), 2))
```

```
15.8 78316.07
```

15,8 % des lignes ont une remise, pour un total de 78 316,07 € sur trois ans, soit un peu plus de 2 % du chiffre d'affaires (3 653 157 €, vu en 4.6.1).

Corrigé 4.5

```
codes = cm["code_promo"].fillna("aucun")
print((cm.assign(sans=cm["code_promo"].isna()).groupby("canal")["sans"].mean() *
↳ 100).round(1).to_dict())
print(codes.value_counts().sum() == len(cm))
```

```
{'Boutique': 84.4, 'Réseaux': 83.9, 'Site': 84.1}
True
```

La part de commandes sans code est de 84,4 % en boutique, 83,9 % sur les réseaux et 84,1 % sur le Site : elle ne dépend pas du canal. La somme des effectifs par code égale bien le nombre de commandes (True) : fillna n'a perdu aucune ligne.

Corrigé 4.6

```
cl <- read_csv("donnees/clients.csv", show_col_types = FALSE)
top_villes <- cl |> filter(date_inscription < as.Date("2023-01-01")) |> count(ville, sort = TRUE) |>
↳ head(3)
top_villes
```

```
# A tibble: 3 × 2
  ville      n
<chr> <int>
1 Ville A   579
2 Ville B   497
3 Ville C   417
```

```
cl_py = pd.read_csv("donnees/clients.csv", parse_dates=["date_inscription"])
print(cl_py[cl_py["date_inscription"] < "2023-01-01"]["ville"].value_counts().head(3).reset_index())
```

```

      ville  count
0  Ville A    579
1  Ville B    497
2  Ville C    417

```

identique au résultat de pandas : TRUE

Les trois villes sont la Ville A (579 clients), la Ville B (497) et la Ville C (417), en R comme en pandas ; seul le nom de la colonne de comptage diffère (n en R, count en pandas).

Corrigé 4.7

```

lg2 = pd.read_csv("donnees/lignes_commande.csv")
v = lg2.merge(cm, on="id_commande", validate="m:1").assign(annee=lambda d:
    ↪ d["date_commande"].dt.year)
t = v.groupby(["canal", "annee"]).agg(ca=("montant", "sum"), n=("id_commande", "nunique"))
panier = (t["ca"] / t["n"]).unstack().round(1)
print(panier)
print(((panier[2025] / panier[2023] - 1) * 100).round(1).to_dict())

```

```

annee      2023  2024  2025
canal
Boutique  100.3  99.4  103.1
Réseaux   99.9  99.0  102.4
Site      98.9  98.3  101.6
{'Boutique': 2.8, 'Réseaux': 2.5, 'Site': 2.7}

```

Le panier moyen est d'environ 98 à 100 € dans les trois canaux en 2023 et 2024, et de 101,6 à 103,1 € en 2025 : +2,8 % en boutique, +2,5 % sur les réseaux, +2,7 % sur le Site. La variation est modeste, et correspond à la hausse de prix de 3 % du 1er janvier 2025 (vérité programmée) : le panier n'a pas changé de nature, les prix ont monté.

Corrigé 4.8

```

rt = pd.read_csv("donnees/retours.csv")
w = lg2.merge(cm[["id_commande", "mode_livraison", "canal"]], on="id_commande",
    ↪ validate="m:1").assign(retourne=lambda d: d["id_ligne"].isin(rt["id_ligne"]))
print(len(lg2) == len(w))
print((w.groupby("mode_livraison")["retourne"].mean() * 100).round(1).to_dict())
print(pd.crosstab(w["mode_livraison"], w["canal"]).to_string())

```

```

True
{'Domicile': 8.5, 'Point relais': 8.7, 'Retrait magasin': 3.4}
canal      Boutique  Réseaux  Site
mode_livraison
Domicile           0      4949  19728
Point relais       0      3483  13249
Retrait magasin   39362       639   2495

```

Le nombre de lignes est inchangé (True). Le taux de retour est de 8,5 % pour la livraison à domicile, 8,7 % en point relais et 3,4 % en retrait magasin. Attention à ne pas conclure que le retrait magasin « évite » les retours : le retrait magasin est presque exclusivement un achat **en boutique** (39 362 lignes sur 42 496), et c'est le **canal** qui explique l'écart (3,1 % en boutique, voir 4.3.3). À canal égal, les modes de livraison se ressemblent. C'est un exemple d'effet de **composition**.

Corrigé 4.9

```
pr = pd.read_csv("donnees/produits.csv")
v25 = v[v["annee"] == 2025].merge(pr[["id_produit", "categorie"]], on="id_produit", validate="m:1")
v25 = v25.assign(trimestre=v25["date_commande"].dt.quarter)
large9 = v25.pivot_table(index="categorie", columns="trimestre", values="montant", aggfunc="sum")
part = (large9.div(large9.sum(axis=1), axis=0) * 100).round(1)
print(part)
print(part[4].idxmax(), part[4].max())
```

trimestre	1	2	3	4
categorie				
Bien-être	23.4	22.1	17.7	36.8
Cuisine	23.0	22.5	18.1	36.3
Décoration	21.3	19.7	17.0	42.0
Jardin	8.4	29.0	40.5	22.0
Maison	24.0	21.6	17.2	37.2
Papeterie	21.6	22.3	20.9	35.2
Décoration	42.0			

```
long9 = large9.reset_index().melt(id_vars="categorie", var_name="trimestre", value_name="ca")
print(long9.shape, long9.groupby("trimestre")["ca"].sum().round(0).to_dict())
```

```
(24, 3) {1: 251609.0, 2: 310783.0, 3: 314572.0, 4: 447800.0}
```

La décoration est la catégorie la plus concentrée sur le quatrième trimestre (42,0 % de son chiffre d'affaires annuel), contre 35 à 37 % pour les autres catégories hors jardin ; le jardin fait exception : 40,5 % au troisième trimestre et seulement 22,0 % au quatrième. Le format long compte 24 lignes (6 catégories × 4 trimestres) et les totaux par trimestre sont de 251 609 €, 310 783 €, 314 572 € et 447 800 €.

Corrigé 4.10

```
jr = pd.read_csv("donnees/jours_exploitation.csv", parse_dates=["date"])
jr["g28"] = jr["nb_commandes"].rolling(28).mean()
jr["g7"] = jr["nb_commandes"].rolling(7).mean()
print(jr.loc[jr["g28"].idxmax(), "date"].date(), round(jr["g28"].max(), 1))
print(jr.loc[jr["g7"].idxmax(), "date"].date(), round(jr["g7"].max(), 1))
```

```
2025-12-25 61.4
2025-12-04 64.4
```

La moyenne glissante sur 28 jours est maximale le 25 décembre 2025 (61,4 commandes par jour sur les 28 derniers jours), celle sur 7 jours le 4 décembre 2025 (64,4). Plus la fenêtre est courte, plus son maximum est sensible à un pic isolé ; la fenêtre longue mesure la tendance de fond.

Corrigé 4.11

```
A, B, C, D = ("code", "x = 10"), ("code", "y = x * 2"), ("code", "x = 100"), ("code", "print(y)")
haut_bas, _ = 0.executer_notebook([A, B, C, D], os.getcwd())
desordre, _ = 0.executer_notebook([A, B, C, B, D], os.getcwd())
print(0.sorties_texte(haut_bas)[-1], 0.sorties_texte(desordre)[-1])
```

```
20 200
```

De haut en bas, la dernière cellule affiche **20** : y a été calculé quand x valait 10, et ne se met pas à jour quand x change. Si l'on réexécute (2) après (3), y est recalculé avec x = 100 et la dernière

cellule affiche **200**. Le même document donne donc 20 ou 200 selon l'ordre dans lequel on a cliqué : c'est l'état caché.

Corrigé 4.12

```
cmd_iso = cm.assign(iso_an=cm["date_commande"].dt.isocalendar()["year"].astype(int),
↳ an=cm["date_commande"].dt.year)
ecart = cmd_iso[cmd_iso["iso_an"] != cmd_iso["an"]]
print(len(ecart), sorted(ecart["date_commande"].dt.date.astype(str).unique()))
ca_ecart = lg2[lg2["id_commande"].isin(ecart["id_commande"])]["montant"].sum()
print(round(ca_ecart, 2))
```

```
254 ['2023-01-01', '2024-12-30', '2024-12-31', '2025-12-29', '2025-12-30', '2025-12-31']
22186.07
```

254 commandes ont une année ISO différente de leur année du calendrier : celles du 1er janvier 2023 (un dimanche, qui appartient à la semaine 52 de 2022), des 30 et 31 décembre 2024 (semaine 1 de 2025) et des 29, 30 et 31 décembre 2025 (semaine 1 de 2026), pour 22 186,07 € de chiffre d'affaires. Si l'on regroupait par (année du calendrier, semaine ISO), les 30 et 31 décembre 2024 seraient rangés dans « 2024, semaine 1 » avec les premiers jours de janvier **2024** : un même groupe réunirait des jours distants d'un an, et le tableau du lundi de la semaine 1 serait faux.

Corrigé 4.13

```
res13 = (pl.scan_csv("donnees/commandes.csv", try_parse_dates=True)
        .filter(pl.col("date_commande").dt.year() == 2025)
        .group_by("canal").agg(clients=pl.col("id_client").n_unique()).sort("canal").collect())
pd13 = cm[cm["date_commande"].dt.year == 2025].groupby("canal")["id_client"].nunique().sort_index()
print(res13)
print((res13["clients"].to_numpy() == pd13.to_numpy()).all())
```

```
| canal | clients |
| --- | --- |
| str | u32 |
|-----|-----|
| Boutique | 2731 |
| Réseaux | 1139 |
| Site | 2844 |
True
```

En 2025, 2 731 clients distincts ont commandé en boutique, 1 139 sur les réseaux et 2 844 sur le Site ; polars et pandas coïncident (True). La somme (6 714) dépasse les 6 000 clients de la base : un même client peut commander dans plusieurs canaux, ce qui interdit d'additionner des clients distincts de canaux différents.

Chapitre 5 : Types de données, collecte et conception d'enquêtes — exercices et applications

Ce chapitre du cahier accompagne le chapitre 5 du livre. Il contient **huit applications guidées** (de petites études que vous refaites sur les données de la boutique) et **douze exercices** de difficulté croissante (★ à la main, ★★ calcul puis code, ★★★ étude plus ouverte), tous corrigés. Les applications 5.7 et 5.8 utilisent le **mini-serveur local** du livre (section 5.5) : aucun accès réseau externe.

```
import sys, io, time, warnings
warnings.filterwarnings("ignore")
sys.path.insert(0, "build")
import numpy as np, pandas as pd
import requests
import outils_ch05 as O

T = O.charger()
cli, cmd, lig, prod, ret, jr, enq = (T[k] for k in ["clients", "commandes", "lignes", "produits",
↪ "retours", "jours", "enquete"])
inv = O.invites(cmd, cli)
enq_u, ligne_droite = O.nettoyer_enquete(enq)
print(len(cli), len(cmd), len(lig), len(inv), len(enq), len(enq_u))
```

```
6000 36395 83905 3875 958 931
```

Applications

Application 5.1 — La fiche d'identité d'une table (sections 5.1 et 5.2)

Objectif. Écrire une fonction qui décrit n'importe quelle table : lignes, colonnes, type, nombre de valeurs distinctes, part de vides, minimum et maximum. C'est la première chose à faire avec un fichier inconnu.

Étape 1 — la fonction. Pour chaque colonne : type, nombre de valeurs distinctes et part de cases vides.

```
def profil(d):
    return pd.DataFrame({"type": d.dtypes.astype(str), "distinct": d.nunique(), "vide %": (100 *
↪ d.isna().mean()).round(1)})
print(profil(cmd).to_string())
```

	type	distinct	vide %
id_commande	int64	36395	0.0
date_commande	datetime64[us]	1096	0.0
heure	str	840	0.0
id_client	int64	4806	0.0
canal	str	3	0.0
mode_livraison	str	3	0.0
code_promo	str	3	84.2

Étape 2 — repérer le grain et la clé. Une colonne dont toutes les valeurs sont distinctes est une **clé candidate**. Vérifiez-le pour chaque table.

```
for nom, d in [("clients", cli), ("commandes", cmd), ("lignes", lig), ("retours", ret), ("produits",
↳ prod)]:
    cles = [c for c in d.columns if d[c].is_unique]
    print(f"{nom:10s} {len(d):>6d} lignes ; colonnes à valeurs uniques : {cles}")
```

```
clients      6000 lignes ; colonnes à valeurs uniques : ['id_client']
commandes    36395 lignes ; colonnes à valeurs uniques : ['id_commande']
lignes       83905 lignes ; colonnes à valeurs uniques : ['id_ligne']
retours      5002 lignes ; colonnes à valeurs uniques : ['id_retour', 'id_ligne']
produits     120 lignes ; colonnes à valeurs uniques : ['id_produit']
```

Étape 3 — candidates au type catégoriel. Les colonnes texte avec peu de modalités gagnent à être converties : on gagne de la mémoire et on évite les fautes de frappe.

```
for nom, d in [("commandes", cmd), ("clients", cli), ("enquête", enq)]:
    cand = [c for c in d.columns if str(d[c].dtype) in ("str", "object", "string") and d[c].nunique()
↳ <= 20]
    print(nom, "->", cand)
```

```
commandes -> ['canal', 'mode_livraison', 'code_promo']
clients -> ['ville', 'canal_acquisition']
enquête -> ['canal', 'tranche_age', 'commentaire']
```

Lecture. Les tables de la boutique ont une clé unique chacune (id_client, id_commande, id_ligne, id_retour, id_produit) ; la colonne code_promo est vide à 84 % (ce sont les commandes sans code, voir 5.1.3) et satisfaction_conseil à 59 % (non applicable hors boutique). Une fiche produite à la lecture, mise à jour après chaque nettoyage, vaut mieux que cent commentaires.

Pour aller plus loin. Ajoutez à la fiche le minimum et le maximum des colonnes numériques et des dates, et détectez les valeurs inattendues (une quantité négative, une date dans le futur).

Application 5.2 — Lire avec les bons types (section 5.1.3)

Objectif. Mesurer ce que coûte une lecture automatique : mémoire occupée et types statistiques erronés.

Étape 1 — lecture brute, puis lecture déclarée.

```
brut = pd.read_csv("donnees/commandes.csv")
net = pd.read_csv("donnees/commandes.csv", dtype={"canal": "category", "mode_livraison": "category",
↳ "code_promo": "category"}, parse_dates=["date_commande"])
print(brut.dtypes.astype(str).to_dict()); print(net.dtypes.astype(str).to_dict())
```

```
{'id_commande': 'int64', 'date_commande': 'str', 'heure': 'str', 'id_client': 'int64', 'canal':
↳ 'str', 'mode_livraison': 'str', 'code_promo': 'str'}
{'id_commande': 'int64', 'date_commande': 'datetime64[us]', 'heure': 'str', 'id_client': 'int64',
↳ 'canal': 'category', 'mode_livraison': 'category', 'code_promo': 'category'}
```

Étape 2 — mémoire occupée.

```
for nom, d in [("brut", brut), ("déclaré", net)]:
    print(nom, round(d.memory_usage(deep=True).sum() / 1e6, 2), "Mo")
```

```
brut 3.31 Mo
déclaré 1.46 Mo
```

Étape 3 — ce que permet une vraie date. On ne peut pas extraire le mois d'une chaîne ; avec une date, c'est immédiat. Comptez les commandes par mois en 2025.

```
print(net[net["date_commande"].dt.year ==
↪ 2025].groupby(net["date_commande"].dt.month).size().to_string())
```

```
date_commande
1      963
2      741
3      890
4      971
5     1021
6     1000
7      963
8      788
9     1097
10     1150
11     1509
12     1853
```

Étape 4 — un identifiant ne se calcule pas. Montrez qu'une moyenne d'identifiants « fonctionne » et ne signifie rien.

```
print("moyenne des id_client :", round(cli["id_client"].mean(), 1), "| milieu de la plage :",
↪ (cli["id_client"].min() + cli["id_client"].max()) / 2)
```

```
moyenne des id_client : 3000.5 | milieu de la plage : 3000.5
```

Lecture. La déclaration des types réduit la mémoire de 3,3 à 1,5 Mo, rend les dates utilisables (12 946 commandes en 2025, de 741 en février à 1 853 en décembre) et protège des opérations absurdes. Le gain de mémoire ne compte que pour de grandes tables ; le gain de **sens** compte toujours.

Application 5.3 — Le piège du grain (section 5.1.4)

Objectif. Calculer correctement le panier moyen, le nombre de commandes et le nombre de clients par canal, à partir d'une table jointe.

Étape 1 — la jointure naïve.

```
j = lig.merge(cmd[["id_commande", "id_client", "canal"]], on="id_commande")
print(len(j), "lignes ; commandes distinctes :", j["id_commande"].nunique(), "; clients distincts :",
↪ j["id_client"].nunique())
print("montant moyen par ligne :", round(j["montant"].mean(), 2))
```

```
83905 lignes ; commandes distinctes : 36395 ; clients distincts : 4806
montant moyen par ligne : 43.54
```

Étape 2 — remonter au bon grain avant de calculer. On agrège d'abord par commande, puis on calcule la moyenne.

```
paniers = j.groupby(["id_commande", "canal"], as_index=False)["montant"].sum()
print(paniers.groupby("canal")["montant"].agg(commandes="count",
↳ panier_moyen="mean").round(1).to_string())
```

	commandes	panier_moyen
canal		
Boutique	16975	100.9
Réseaux	3957	100.5
Site	15463	99.8

Étape 3 — le client est un autre grain. Le chiffre d'affaires moyen **par client** n'est ni le panier ni le montant d'une ligne : agrégez par client.

```
par_client = j.groupby("id_client")["montant"].sum()
print(round(par_client.mean(), 1), "€ par client (sur 3 ans) ; médiane :", round(par_client.median(),
↳ 1))
```

```
760.1 € par client (sur 3 ans) ; médiane : 474.0
```

Lecture. Le même fichier donne un « montant moyen » de 43,5 € (par ligne), un panier moyen de 100,4 € (par commande) et 760 € de chiffre d'affaires moyen par client sur trois ans : trois réponses justes à trois questions différentes. La moyenne est supérieure à la médiane (474 €) : quelques gros clients tirent la moyenne vers le haut, ce que le chapitre 1 explique.

Application 5.4 — Nettoyer l'enquête et calculer un NPS (sections 5.3.5 et 5.3.6)

Objectif. Appliquer des règles de nettoyage écrites à l'avance, puis calculer le NPS par canal avec son intervalle de confiance. Vérifier que la conclusion résiste au choix du seuil de rapidité.

Étape 1 — les trois filtres.

```
sl = (enq["satisfaction_globale"] == 5) & (enq["satisfaction_livraison"] == 5) &
↳ (enq["satisfaction_prix"] == 5)
doubles = enq.duplicated(subset=[c for c in enq.columns if c != "id_reponse"])
print("doublons :", int(doubles.sum()), "| trois notes de 5 :", int(sl.sum()), "| dont en moins de 25
↳ s :", int((sl & (enq["duree_reponse_s"] < 25)).sum()))
```

```
doublons : 27 | trois notes de 5 : 105 | dont en moins de 25 s : 35
```

Étape 2 — NPS par canal, avec intervalle.

```
propre = enq_u[~ligne_droite]
lignes_ = []
for c in ["Boutique", "Site", "Réseaux"]:
    d = propre[propre["canal"] == c]
    v, m = 0.nps(d["recommandation_0_10"])
    lignes_.append((c, len(d), round(v, 1), f"[{v - m:.1f} ; {v + m:.1f}]"))
print(pd.DataFrame(lignes_, columns=["canal", "n", "NPS", "IC 95 %"]).to_string(index=False))
```

canal	n	NPS	IC 95 %
Boutique	369	-0.5	[-9.0 ; 7.9]
Site	416	-38.7	[-46.2 ; -31.2]
Réseaux	112	-23.2	[-38.1 ; -8.4]

Étape 3 — sensibilité au seuil de rapidité. Faites varier le seuil (15, 25, 40 secondes) et regardez le nombre de réponses retirées et le NPS d'ensemble.

```
for seuil in (15, 25, 40):
    u, s = 0.nettoyer_enquete(enq, seuil_rapide=seuil)
    print(f"seuil {seuil:>2d} s : {int(s.sum()):>3d} retirées ; NPS
↳ {0.nps(u[~s]['recommandation_0_10'])[0]:.1f} ; satisfaction
↳ {u[~s]['satisfaction_globale'].mean():.3f}")
```

```
seuil 15 s : 19 retirées ; NPS -21.6 ; satisfaction 3.607
seuil 25 s : 34 retirées ; NPS -21.1 ; satisfaction 3.584
seuil 40 s : 43 retirées ; NPS -22.2 ; satisfaction 3.570
```

Lecture. Les trois seuils retirent 19, 34 et 43 réponses ; le NPS d'ensemble varie de $-22,2$ à $-21,1$, sans changer la conclusion. Le canal Boutique se distingue du canal Site par son NPS, alors que l'intervalle du canal Réseaux, fondé sur peu de réponses, est large. Quand un résultat ne dépend pas du seuil que l'on choisit, on peut l'écrire sans scrupule ; quand il en dépend, il faut le dire.

Application 5.5 — Répondants contre invités : pondérer (section 5.3.3 et 5.3.4)

Objectif. Comparer les répondants à la population invitée sur plusieurs variables, puis tester l'effet de **plusieurs** pondérations sur la satisfaction moyenne.

Étape 1 — comparaison sur ce que l'on sait des invités. On utilise les répondants identifiés.

```
rep = enq_u.dropna(subset=["id_client"]).astype({"id_client": int}).merge(inv[["id_client",
↳ "fidelite", "jours", "ville"]], on="id_client")
print("carte de fidélité : invités", round(100 * inv["fidelite"].mean(), 1), "% | répondants",
↳ round(100 * rep["fidelite"].mean(), 1), "%")
print("récents (≤ 60 j) : invités", round(100 * (inv["jours"] <= 60).mean(), 1), "% | répondants",
↳ round(100 * (rep["jours"] <= 60).mean(), 1), "%")
```

```
carte de fidélité : invités 35.6 % | répondants 39.8 %
récents (≤ 60 j) : invités 54.0 % | répondants 59.0 %
```

Étape 2 — pondération par canal et âge, puis par fidélité et récence. Pour la seconde, on se limite aux répondants identifiés.

```
def moyenne_ponderee(d, pop, colonnes, y="satisfaction_globale"):
    p = pop.groupby(colonnes).size() / len(pop)
    q = d.groupby(colonnes).size() / len(d)
    w = d.join((p / q).rename("w"), on=colonnes)["w"]
    return np.average(d[y], weights=w)
inv["recent"] = (inv["jours"] <= 60).astype(int); rep["recent"] = (rep["jours"] <= 60).astype(int)
print("moyenne brute : tous", round(enq_u["satisfaction_globale"].mean(), 3), "| identifiés",
↳ round(rep["satisfaction_globale"].mean(), 3))
print("pondérée canal × âge (tous) :", round(moyenne_ponderee(enq_u, inv, ["canal", "tranche_age"]),
↳ 3), "| fidélité × récence (identifiés) :", round(moyenne_ponderee(rep, inv, ["fidelite",
↳ "recent"]), 3))
```

```
moyenne brute : tous 3.636 | identifiés 3.622
pondérée canal × âge (tous) : 3.633 | fidélité × récence (identifiés) : 3.611
```

Étape 3 — la vérité programmée.

```
v = 0.verite_satisfaction(inv, repetitions=200)
print({k: round(x, 3) for k, x in v.items() if k.startswith("sat")})
```

```
{'sat_population': 3.607, 'sat_repondants': 3.615}
```

Lecture. Les deux pondérations déplacent la moyenne de très peu : de 3,636 à 3,633 pour canal × âge, de 3,622 à 3,611 pour fidélité × récence ; les deux restent proches de la vérité (3,607). La recette n'a aucun intérêt à produire des écarts spectaculaires : elle sert à **vérifier que la conclusion n'en dépend pas**.

Application 5.6 — Plans de sondage et taille d'échantillon (section 5.4)

Objectif. Mesurer comment la précision d'une estimation varie avec la taille de l'échantillon, et comparer les plans.

Étape 1 — la population de référence.

```
pop = 0.depense_2025(cmd, lig, cli)
print(len(pop), "clients actifs ; dépense moyenne 2025 :", round(pop["depense_2025"].mean(), 1), "€")
```

```
3875 clients actifs ; dépense moyenne 2025 : 341.9 €
```

Étape 2 — la précision selon la taille. On compare le tirage aléatoire simple pour trois tailles d'échantillon, 300 répétitions chacune.

```
res = {n: 0.simuler_plans(pop, n=n, repetitions=300, seed=2)["aleatoire_simple"].std() for n in (100,
↪ 300, 900)}
print({n: round(s, 1) for n, s in res.items()}, "| rapport 100 → 900 :", round(res[100] / res[900], 2))
```

```
{100: np.float64(33.4), 300: np.float64(18.6), 900: np.float64(10.6)} | rapport 100 → 900 : 3.14
```

Étape 3 — les quatre plans à taille égale.

```
sim = 0.simuler_plans(pop, n=300, repetitions=300, seed=2)
print(pd.DataFrame({"moyenne": sim.mean(), "écart-type": sim.std()}).round(1).to_string())
```

	moyenne	écart-type
aleatoire_simple	340.2	18.6
stratifiée	342.0	16.9
grappes	344.1	21.7
commodite	487.4	18.4

Lecture. Multiplier l'échantillon par neuf divise l'écart-type par 3,14, proche de $\sqrt{9} = 3$: la précision croît comme la **racine** de la taille, donc le **coût** de la précision croît comme son carré. Le tirage stratifié est le plus précis, la commodité est biaisée de 145 € : plus d'effectif ne la corrigerait pas.

Application 5.7 — Une API paginée et limitée en débit (section 5.5.2)

Objectif. Écrire un client qui récupère tout le catalogue malgré la pagination et la limite de débit, et le contrôle.

Étape 1 — démarrer le serveur local.

```
serveur = 0.MiniServeur(prod, 0.villes_ouvertes()).demarrer()
url, cle = serveur.url, {"X-API-Key": 0.CLE_API}
print(requests.get(url + "/api/v1/produits").status_code, requests.get(url +
↪ "/api/v1/produits?page=1", headers=cle).status_code)
```

```
401 200
```

Étape 2 — le client avec reprise après un 429. On attend le délai indiqué, puis on réessaie la même page, avec une limite de tentatives pour ne pas boucler indéfiniment.

```
def lire_api(par_page=20, max_essais=5):
    out, page, refus = [], 1, 0
    while page:
        for essai in range(max_essais):
            r = requests.get(url + "/api/v1/produits", headers=cle, params={"page": page, "per_page":
↪ par_page})
            if r.status_code != 429:
```

```

        break
    refus += 1; time.sleep(int(r.headers["Retry-After"]))
    r.raise_for_status()
    corps = r.json(); out += corps["data"]; page = page + 1 if corps["suivant"] else None
    return pd.DataFrame(out), refus
api, refus = lire_api()
print(len(api), "produits ; au moins un refus 429 rencontré :", refus > 0)

```

```
120 produits ; au moins un refus 429 rencontré : True
```

Étape 3 — utiliser et contrôler. Calculez le prix moyen par catégorie à partir de l'API et comparez-le au fichier.

```

a = api.groupby("categorie")["prix_vente"].mean().round(2); f =
↳ prod.groupby("categorie")["prix_vente"].mean().round(2)
print(pd.DataFrame({"api": a, "fichier": f}).to_string()); print("identiques :", bool((a == f).all()))
serveur.arreter()

```

```

      api  fichier
categorie
Bien-être  27.50   27.50
Cuisine    34.75   34.75
Décoration 38.05   38.05
Jardin     51.75   51.75
Maison     51.85   51.85
Papeterie  11.35   11.35
identiques : True

```

Lecture. Le client récupère 120 produits en rencontrant au moins un refus 429 (leur nombre exact dépend du moment de l'appel, car la limite se mesure en secondes) ; les prix moyens par catégorie sont identiques à ceux du fichier. Sans la boucle de reprise, le programme aurait renvoyé une erreur à la quatrième requête ; sans `raise_for_status`, il aurait traité en silence un message d'erreur comme des données.

Application 5.8 — Moissonner un catalogue poliment (section 5.5.3)

Objectif. Lire les pages du catalogue en respectant `robots.txt`, extraire les produits et construire un lecteur qui **résiste** à un changement de structure.

Étape 1 — les règles.

```

from urllib.robotparser import RobotFileParser
from bs4 import BeautifulSoup
serveur = 0.MiniServeur(prod, 0.villes_ouvertes()).demarrer(); url = serveur.url
rp = RobotFileParser(url + "/robots.txt"); rp.read()
print("catalogue :", rp.can_fetch("*", url + "/catalogue"), "| stock :", rp.can_fetch("*", url +
↳ "/prive/stock"), "| délai :", rp.crawl_delay("*"))

```

```
catalogue : True | stock : False | délai : 1
```

Étape 2 — un lecteur qui connaît deux structures. La version 1 utilise `div.produit` ; la version 2 utilise `article.item`.

```

def prix(txt):
    return float(txt.replace("EUR", "").replace("€", "").replace(",", ".").strip())
def lire(html):
    s = BeautifulSoup(html, "html.parser")
    v1 = [{"id_produit": int(c["data-id"]), "prix_vente": prix(c.select_one(".prix").text)} for c in
↳ s.select("div.produit")]
    v2 = [{"id_produit": int(c["id"][1:]), "prix_vente": prix(c.select_one(".tarif").text)} for c in
↳ s.select("article.item")]

```

```

    return v1 or v2
print(len(lire(requests.get(url + "/catalogue?page=1").text)), len(lire(requests.get(url +
↪ "/catalogue-v2?page=1").text)))

10 10

```

Étape 3 — trois pages de chaque version, avec le délai demandé, puis comparaison.

```

def pages(chemin, n=3):
    out = []
    for p in range(1, n + 1):
        out += lire(requests.get(url + chemin, params={"page": p}).text);
    ↪ time.sleep(rp.crawl_delay(""))
    return pd.DataFrame(out).sort_values("id_produit").reset_index(drop=True)
p1, p2 = pages("/catalogue"), pages("/catalogue-v2")
print(len(p1), len(p2), "| mêmes produits et prix dans les deux versions :", bool(p1.equals(p2)))
serveur.arreter()

30 30 | mêmes produits et prix dans les deux versions : True

```

Lecture. Le lecteur tolérant lit les deux structures et retrouve les mêmes 30 produits et les mêmes prix. Il reste fragile : une troisième structure renverrait zéro ligne. La bonne protection est un **contrôle d'effectif** (comparer au nombre attendu) qui arrête la collecte au lieu de laisser passer une table vide.

Exercices

Exercice 5.1 ★ — Classer des variables (section 5.1.1)

Pour chacune des variables suivantes, indiquez la **famille** (nominale, ordinale, quantitative discrète, quantitative continue, binaire, date/heure, texte, identifiant) et le **niveau de mesure** : (a) le numéro de commande ; (b) la température moyenne du jour ; (c) la note de satisfaction de 1 à 5 ; (d) la ville d'un client ; (e) le montant d'une ligne ; (f) l'heure de la commande ; (g) la présence d'une carte de fidélité ; (h) le nombre de lignes d'une commande ; (i) la tranche d'âge ; (j) le commentaire libre.

Exercice 5.2 ★ — Quel résumé pour quelle variable ? (section 5.1.2)

- (1) Dans le groupe A, les dix réponses à une échelle de 1 à 5 sont toutes 3 ; dans le groupe B, il y a cinq « 1 » et cinq « 5 ». Calculez la moyenne et la médiane de chaque groupe et dites ce que chaque résumé cache. (2) Sur `enquete.satisfaction_globale`, calculez la moyenne, la médiane et la part de « 4 ou 5 » **par canal**, et expliquez quel résumé vous présenteriez à la gérante.

Exercice 5.3 ★ — Lire sans rien perdre (section 5.1.3)

Le texte "ref;prix;date;ok\nA007;1 299,90;04/11/2025;vrai\nB012;45,00;05/11/2025;faux\n" est lu sans option. Prévoyez à la main les types obtenus, puis écrivez la lecture qui conserve ref en texte, lit le prix en nombre décimal, la date comme une date jour/mois/année et ok comme un booléen.

Exercice 5.4 ★★ — Combien de commandes par client ? (section 5.1.4)

- (1) À partir de la table jointe `lignes_commande × commandes`, calculez de deux façons le nombre moyen de commandes par client ayant commandé : en comptant les lignes (faux) et en comptant les commandes distinctes (juste). (2) Calculez le panier moyen par canal avec la bonne méthode et expliquez l'écart avec la moyenne par ligne.

Exercice 5.5 ★ — Recouper deux sources (section 5.2.2)

La table `jours_exploitation` donne, pour chaque jour, le nombre de commandes et le chiffre d'affaires. Vérifiez qu'elle est **cohérente** avec les tables `commandes` et `lignes_commande` : même nombre total de commandes, même chiffre d'affaires par jour, une ligne par jour de la période sans trou ni doublon.

Exercice 5.6 ★★ — Rendre un fichier plus anonyme (section 5.2.3)

Dans la table des clients, la part de clients **uniques** sur (`ville`, `annee_naissance`, `canal_acquisition`, `fidelite`) est élevée. Regroupez l'année de naissance par **décennie**, recalculez la part de clients uniques et la taille du plus petit groupe, puis proposez un regroupement supplémentaire qui amène la taille minimale à au moins 5 (le **5-anonymat**). Que perd-on en précision ?

Exercice 5.7 ★ — Pondérer à la main puis par code (section 5.3.4)

Parmi les invités, 70 % sont des clients avec carte et 30 % sans carte ; parmi les répondants, 80 % avec carte et 20 % sans carte. Les clients avec carte répondent en moyenne 3,9 et les autres 3,2. (1) Calculez les poids, la moyenne brute et la moyenne pondérée. (2) Refaites le calcul par code sur la variable `fidelite` des répondants identifiés de la boutique, en rétablissant la composition de la population invitée.

Exercice 5.8 ★★ — Les bornes sans hypothèse, par canal (section 5.3.4)

Pour chaque canal, calculez le taux de réponse (réponses distinctes sur invités du canal), la satisfaction moyenne des répondants et les **bornes** de la satisfaction de la population obtenues sans aucune hypothèse sur les non-répondants. Quel canal a les bornes les plus étroites, et pourquoi ?

Exercice 5.9 ★★ — Un NPS à la main, puis sur le fichier (section 5.3.6)

- (1) Sur 80 réponses, 28 promoteurs, 20 détracteurs et 32 passifs : calculez le NPS, son erreur-type et son intervalle à 95 %. (2) Combien de réponses faudrait-il, avec ces mêmes proportions, pour que la demi-largeur de l'intervalle soit inférieure à 5 points ? (3) Sur le fichier nettoyé, calculez le NPS des clients de 55 à 64 ans et dites si l'on peut le distinguer de celui des moins de 25 ans.

Exercice 5.10 ★★ — Combien de répondants pour voir une formulation ? (section 5.4.1)

Simulez un split-ballot dans lequel la formulation B augmente la note moyenne de 0,2 point (notes arrondies de 1 à 5, écart-type de 1). Pour des effectifs de 100, 200, 400 et 800 par version, estimez par 500 répétitions la **puissance** : la part des expériences dans lesquelles l'intervalle de confiance de l'écart exclut zéro.

Exercice 5.11 ★ — Dimensionner une enquête (section 5.4.3)

- (1) Quel nombre de réponses faut-il pour estimer une proportion à ± 4 points près (cas prudent $p = 0,5$) ? (2) Et si l'on attend $p \approx 0,2$? (3) Corrigez pour une population de 3 875 clients. (4) Avec un taux de réponse de 24 %, combien d'invitations ?

Exercice 5.12 ★★★ — Un client robuste pour une collecte hebdomadaire (section 5.5)

Écrivez une fonction `collecter()` qui lit le catalogue par l'API, **vérifie** le résultat (nombre de produits, identifiants uniques, prix strictement positifs, accord avec le fichier `produits.csv`) et lève une erreur explicite en cas de problème. Testez-la sur le serveur normal, puis sur une version **dégradée** du serveur (que vous fabriquez en lui donnant un catalogue tronqué à 100 produits) : la fonction doit alors signaler l'écart.

Corrigés**Corrigé 5.1**

Variable	Famille	Niveau
(a) numéro de commande	identifiant	nominal
(b) température moyenne du jour	quantitative continue	intervalle
(c) note de 1 à 5	qualitative ordinale	ordinal
(d) ville	qualitative nominale	nominal
(e) montant d'une ligne	quantitative continue	rapport
(f) heure de la commande	heure	intervalle
(g) carte de fidélité	binaire	nominal
(h) nombre de lignes d'une commande	quantitative discrète	rapport
(i) tranche d'âge	qualitative ordinale	ordinal
(j) commentaire libre	texte	—

Les trois erreurs fréquentes : classer (a) en « quantitative » parce que ce sont des chiffres ; classer (c) en « quantitative continue » ; croire que (b) est un rapport (le zéro Celsius est conventionnel).

Corrigé 5.2

- (1) Groupe A : moyenne 3, médiane 3. Groupe B : moyenne $(5 \times 1 + 5 \times 5) / 10 = 3$, médiane 3 (la médiane de dix valeurs, cinq 1 puis cinq 5, est la moyenne des 5^e et 6^e valeurs, soit 3). Les deux résumés sont identiques et **cachent tout** : A est un groupe indifférent, B un groupe polarisé. Seule la **distribution** distingue les groupes.

- (2) Par canal :

```
g = enq_u.groupby("canal")["satisfaction_globale"]
print(pd.DataFrame({"moyenne": g.mean(), "médiane": g.median(), "part 4-5": g.apply(lambda s: (s >=
↵ 4).mean())}).round(3).to_string())
```

```

           moyenne  médiane  part 4-5
canal
```

Boutique	3.951	4.0	0.681
Réseaux	3.517	4.0	0.509
Site	3.386	3.0	0.447

La Boutique est la mieux placée sur les trois résumés (3,95 de moyenne, 68 % de réponses favorables). La **part de 4 ou 5** est le résumé le plus parlant pour la gérante (« 68 % des clients de la boutique sont satisfaits ») et ne suppose rien sur l'écart entre les modalités. On y ajoute la distribution complète en annexe.

Corrigé 5.3

Lue sans option, la table donne : ref en texte (« A007 », « B012 » : pas de piège ici, mais un code 007 serait devenu 7), prix en **texte** (à cause de l'espace et de la virgule), date en texte, ok en texte.

```
t = "ref;prix;date;ok\nA007;1 299,90;04/11/2025;vrai\nB012;45,00;05/11/2025;faux\n"
d = pd.read_csv(io.StringIO(t), sep=";", dtype={"ref": "string"}, decimal=".", thousands=" ",
    ↳ parse_dates=["date"], date_format="%d/%m/%Y", true_values=["vrai"], false_values=["faux"])
print(d.dtypes.astype(str).to_dict()); print([str(x) for x in d.iloc[0]])
```

```
{'ref': 'string', 'prix': 'float64', 'date': 'datetime64[us]', 'ok': 'bool'}
['A007', '1299.9', '2025-11-04 00:00:00', 'True']
```

Corrigé 5.4

```
mm = lig.merge(cmd[["id_commande", "id_client", "canal"]], on="id_commande")
faux = len(mm) / mm["id_client"].nunique()
juste = mm["id_commande"].nunique() / mm["id_client"].nunique()
par_ligne = mm["montant"].mean()
panier = mm.groupby(["id_commande", "canal"])["montant"].sum().groupby("canal").mean()
print(round(faux, 2), round(juste, 2), round(par_ligne, 1)); print(panier.round(1).to_dict())
```

```
17.46 7.57 43.5
{'Boutique': 100.9, 'Réseaux': 100.5, 'Site': 99.8}
```

Compter les lignes donne 17,5 « commandes » par client au lieu de 7,6. Le panier moyen par canal est de 100,9 € (Boutique), 99,8 € (Site) et 100,5 € (Réseaux) ; la moyenne par ligne (43,5 €) est celle d'un objet vendu, pas d'un achat : une commande compte en moyenne 2,31 lignes.

Corrigé 5.5

```
m5 = lig.merge(cmd[["id_commande", "date_commande"]], on="id_commande")
ca = m5.groupby("date_commande")["montant"].sum().reindex(jr["date"], fill_value=0).round(2)
nb = cmd.groupby("date_commande").size().reindex(jr["date"], fill_value=0)
jours_ok = pd.date_range(jr["date"].min(), jr["date"].max())
print("total des commandes :", jr["nb_commandes"].sum() == len(cmd), "| CA par jour égal :",
    ↳ bool(np.allclose(ca.values, jr["chiffre_affaires"].values)))
print("nombre par jour égal :", bool((nb.values == jr["nb_commandes"].values).all()), "| une ligne
    ↳ par jour :", jr["date"].is_unique and len(jr) == len(jours_ok))
```

```
total des commandes : True | CA par jour égal : True
nombre par jour égal : True | une ligne par jour : True
```

Les quatre contrôles sont vrais : la table de synthèse est **cohérente** avec le détail. En pratique, une table de synthèse maintenue à la main s'écarte du détail ; le contrôle croisé est le moyen le plus rapide de le voir.

Corrigé 5.6

```
cli["decennie"] = cli["annee_naissance"] // 10 * 10
cli["tranche"] = np.where(cli["annee_naissance"] < 1960, 1950, cli["decennie"]) # tous les nés
↳ avant 1960 ensemble
for nom, cols in (("année exacte, avec la ville", ["ville", "annee_naissance", "canal_acquisition",
↳ "fidelite"]), ("décennie, avec la ville", ["ville", "decennie", "canal_acquisition", "fidelite"]),
↳ ("décennie, sans la ville", ["decennie", "canal_acquisition", "fidelite"]),
↳ ("tranches élargies, sans la ville", ["tranche", "canal_acquisition", "fidelite"])):
    t = cli.groupby(cols)["id_client"].transform("size")
    print(f"{nom:34s}: {100 * (t == 1).mean():4.1f} % uniques ; plus petit groupe : {int(t.min())}")
```

```
année exacte, avec la ville      : 26.4 % uniques ; plus petit groupe : 1
décennie, avec la ville         :  2.1 % uniques ; plus petit groupe : 1
décennie, sans la ville         :  0.0 % uniques ; plus petit groupe : 3
tranches élargies, sans la ville :  0.0 % uniques ; plus petit groupe : 16
```

Avec l'année exacte, 26,4 % des clients sont uniques ; avec la **décennie**, 2,1 % ; en retirant aussi la ville, plus personne n'est unique mais le plus petit groupe compte encore 3 personnes seulement (les nés avant 1960 sont rares). En **regroupant tous les nés avant 1960**, le plus petit groupe passe à 16 personnes : le seuil de 5 est atteint. On a réglé l'anonymat en **perdant la ville** et en grossissant l'âge : une analyse géographique n'est plus possible sur le fichier partagé. Le compromis entre utilité et protection est un choix à documenter.

Corrigé 5.7

- (1) Poids : avec carte $0,70/0,80 = 0,875$; sans carte $0,30/0,20 = 1,5$. Moyenne brute : $0,8 \times 3,9 + 0,2 \times 3,2 = 3,76$; pondérée : $0,7 \times 3,9 + 0,3 \times 3,2 = 3,69$. La pondération retire les 0,07 point dus à la sur-représentation des clients avec carte.

- (2) Sur les données :

```
rep7 = enq_u.dropna(subset=["id_client"]).astype({"id_client": int}).merge(inv[["id_client",
↳ "fidelite"]], on="id_client")
pi = inv["fidelite"].value_counts(normalize=True); pr = rep7["fidelite"].value_counts(normalize=True)
w = rep7["fidelite"].map(pi / pr)
print("part avec carte : invités", round(pi[1], 3), "| répondants", round(pr[1], 3))
print("moyenne brute", round(rep7["satisfaction_globale"].mean(), 3), "| pondérée",
↳ round(np.average(rep7["satisfaction_globale"], weights=w), 3))
```

```
part avec carte : invités 0.356 | répondants 0.398
moyenne brute 3.622 | pondérée 3.616
```

La pondération déplace la moyenne de 3,622 à 3,616 : la fidélité ne distingue pas assez les répondants pour changer la conclusion.

Corrigé 5.8

```
inv_c = inv.groupby("canal").size(); rep_c =
↳ enq_u.groupby("canal")["satisfaction_globale"].agg(["size", "mean"])
tab = pd.DataFrame({"invités": inv_c, "réponses": rep_c["size"], "taux": rep_c["size"] / inv_c,
↳ "moyenne": rep_c["mean"]})
tab["borne basse"] = tab["taux"] * tab["moyenne"] + (1 - tab["taux"]) * 1
tab["borne haute"] = tab["taux"] * tab["moyenne"] + (1 - tab["taux"]) * 5
print(tab.round(3).to_string())
```

canal	invités	réponses	taux	moyenne	borne basse	borne haute
Boutique	1591	385	0.242	3.951	1.714	4.746

Réseaux	453	116	0.256	3.517	1.645	4.620
Site	1831	430	0.235	3.386	1.560	4.621

Le canal Site a le taux de réponse le plus élevé (23,5 %) et donc les bornes les plus étroites, mais elles couvrent encore 3,1 points sur une échelle qui en compte 4 : aucune conclusion sans hypothèse. Plus le taux de réponse est faible, plus l'incertitude est grande : la largeur des bornes est $4(1 - r)$.

Corrigé 5.9

(1) $p = 28/80 = 0,35$, $d = 20/80 = 0,25$; NPS = +10 points. Variance d'une réponse : $0,35 + 0,25 - 0,10^2 = 0,59$; erreur-type $\sqrt{0,59/80} \approx 0,0859$, soit 8,6 points ; intervalle à 95 % : $10 \pm 16,8$, donc de $-6,8$ à $+26,8$ points : on ne sait pas si le NPS est positif.

(2) Demi-largeur de 5 points : $1,96\sqrt{0,59/n} \leq 0,05$, soit $n \geq 0,59 \times (1,96/0,05)^2 \approx 907$ réponses.

(3) Sur le fichier :

```
nett = enq_u[~ligne_droite]
for t in ["55-64 ans", "moins de 25 ans"]:
    d = nett[nett["tranche_age"] == t]["recommandation_0_10"]; v_, m_ = 0.nps(d)
    print(f"{t:16s} n = {len(d):>3d} NPS {v_:6.1f} IC95 [{v_ - m_:1f} ; {v_ + m_:1f}]")
```

```
55-64 ans      n = 119 NPS -31.9 IC95 [-46.5 ; -17.4]
moins de 25 ans n = 87  NPS -11.5 IC95 [-28.8 ; 5.8]
```

Les deux intervalles sont très larges (peu de réponses par tranche) et se chevauchent : **on ne peut pas distinguer** les deux tranches. Voilà ce que coûte un découpage trop fin.

Corrigé 5.10

```
rng = np.random.default_rng(10)
def puissance(n, effet=0.2, rep=500):
    ok = 0
    for _ in range(rep):
        a = np.clip(np.round(rng.normal(3.5, 1, n)), 1, 5); b = np.clip(np.round(rng.normal(3.5 +
↪ effet, 1, n)), 1, 5)
        ok += abs(b.mean() - a.mean()) > 1.96 * np.sqrt(a.var(ddof=1) / n + b.var(ddof=1) / n)
    return ok / rep
pw = {n: puissance(n) for n in (100, 200, 400, 800)}
print(pw)
```

```
{100: np.float64(0.268), 200: np.float64(0.514), 400: np.float64(0.748), 800: np.float64(0.964)}
```

La puissance passe de 27 % (100 par version) à 96 % (800 par version) : pour détecter un effet de 0,2 point avec une chance sur deux, il faut environ 200 répondants par version ; pour être presque sûr, plus de 800. Un petit pilote ne peut pas conclure sur une formulation légèrement orientée.

Corrigé 5.11

(1) $n_0 = 1,96^2 \times 0,25/0,04^2 = 600,25$, soit **601** réponses. (2) $n_0 = 1,96^2 \times 0,2 \times 0,8/0,04^2 = 384,16$, soit **385** : une proportion éloignée de 0,5 demande moins de réponses. (3) Correction de population finie : $601/(1 + 600/3\,875) \approx 520$ et $385/(1 + 384/3\,875) \approx 350$. (4) Avec un taux de réponse de 24 % : $520/0,24 \approx 2\,167$ invitations pour le cas prudent.

```
for p in (0.5, 0.2):
    n0 = 0.n_corr(0.04, p=p); n1 = 0.n_corr(0.04, N=3875, p=p)
    print(f"p = {p} : {n0:.0f} réponses ; population finie : {n1:.0f} ; invitations à 24 % : {n1 /
    ↪ 0.24:.0f}")
```

```
p = 0.5 : 600 réponses ; population finie : 520 ; invitations à 24 % : 2166
p = 0.2 : 384 réponses ; population finie : 350 ; invitations à 24 % : 1457
```

Corrigé 5.12

```
def collecter(url, cle, attendu=120):
    out, page = [], 1
    while page:
        r = requests.get(url + "/api/v1/produits", headers=cle, params={"page": page, "per_page": 50})
        if r.status_code == 429:
            time.sleep(int(r.headers["Retry-After"])); continue
        r.raise_for_status(); c = r.json(); out += c["data"]; page = page + 1 if c["suivant"] else None
    d = pd.DataFrame(out)
    if len(d) != attendu or not d["id_produit"].is_unique or not (d["prix_vente"] > 0).all():
        raise ValueError(f"collecte douteuse : {len(d)} produits (attendu {attendu})")
    ecart = d.merge(prod[["id_produit", "prix_vente"]], on="id_produit", suffixes=("", "_ref"))
    if not (ecart["prix_vente"] == ecart["prix_vente_ref"]).all():
        raise ValueError("prix différents du fichier de référence")
    return d
cle = {"X-API-Key": 0.CLE_API}
s1 = 0.MiniServeur(prod, 0.villes_ouvertes()).demarrer()
print("serveur normal :", len(collecter(s1.url, cle)), "produits"); s1.arreter()
s2 = 0.MiniServeur(prod.head(100), 0.villes_ouvertes()).demarrer()
try:
    collecter(s2.url, cle)
except ValueError as e:
    print("serveur dégradé :", e)
s2.arreter()
```

```
serveur normal : 120 produits
serveur dégradé : collecte douteuse : 100 produits (attendu 120)
```

La fonction signale l'écart au lieu de livrer une table incomplète. Retenez la structure : **collecter, contrôler, ne rendre la main qu'après les contrôles**. La même fonction, lancée chaque semaine, devient un **test de la source** : si un jour le catalogue change de forme, c'est elle qui sonne l'alarme avant les analyses.

Projet du volume et auto-évaluation

— exercices et applications

Ce chapitre du cahier clôt le volume I. Il contient le **projet du volume** : une **première analyse complète**, du fichier brut au tableau de synthèse, que vous remettrez à la gérante ; puis l'**auto-évaluation** (quarante questions). Il ne demande aucune notion nouvelle : chaque étape s'appuie sur un chapitre du livre, et chaque chiffre est **recoupé par au moins deux outils** (SQL, Python, R, tableur).

Projet du volume

P.1 Le cahier des charges

La gérante vous écrit : « Pour la réunion de janvier, j'aimerais une page qui compare le dernier trimestre 2025 au même trimestre de 2024, par canal : ce que l'on a vendu, combien de commandes, le panier moyen, ce que l'on rembourse en retours et ce que l'on gagne vraiment. Et je voudrais être sûre que les chiffres sont bons : le logiciel de caisse et le site ne donnent pas toujours les mêmes totaux. »

Vous traduisez en **cinq exigences** :

1. **Un tableau de synthèse** par canal (Boutique, Site, Réseaux) : chiffre d'affaires, commandes, panier moyen, taux de retour, marge brute, avec l'évolution en pourcentage.
2. **Des chiffres vérifiés** : chaque total obtenu par au moins deux outils différents, et un écart expliqué ou nul.
3. **Un fichier brut maîtrisé** : l'export de la caisse de la boutique (une semaine) est lu, nettoyé, puis **réconcilié** avec la base de données.
4. **Un classeur lisible** que la gérante peut ouvrir et dont les formules sont visibles.
5. **Un message** de cinq lignes, un graphique, et la liste de ce que l'analyse **ne dit pas**.

La méthode suit neuf étapes, chacune appuyée sur un chapitre du livre :

Étape	Question	Chapitre du livre
P.2 Comprendre	Que contient chaque table ? Quel est son grain ?	5.1, 3.1
P.3 Lire le fichier brut	Comment passer d'un export désordonné à un tableau propre ?	4.3, 2.3
P.4 Réconcilier	La caisse et la base disent-elles la même chose ?	3.2, 4.1

Étape	Question	Chapitre du livre
P.5 Calculer la synthèse	Quel est le chiffre d'affaires par canal et par trimestre ?	3.1, 3.4, 4.1, 4.2
P.6 Les indicateurs	Panier moyen, retours, marge : comment les calculer sans se tromper ?	1.5, 1.1
P.7 Le classeur	Comment livrer un tableau à formules vérifiables ?	2.1, 2.4
P.8 Le graphique	Que montrer ?	1.1
P.9 Le message	Qu'est-ce qu'on dit, et qu'est-ce qu'on ne peut pas dire ?	1.4, 5.3

Les données. `donnees/boutique.db` (base SQLite), `donnees/export_caisse_brut.csv` (une semaine de caisse de la boutique, désordonnée) et les fichiers CSV associés. Elles sont **simulées** : la vérité est programmée (docstring de `build/donnees_a1.py`). Les taux et les montants sont fictifs ; la TVA est fixée à 20 % **pour l'illustration**.

P.2 Étape 1 : comprendre les données

Avant tout calcul, on vérifie le **grain** de chaque table (qu'est-ce qu'une ligne ?) et les clés qui les relient (livre, 3.1 et 5.1). Un comptage simple suffit.

```
import sqlite3, io
import numpy as np, pandas as pd

con = sqlite3.connect("donnees/boutique.db")
for t in ["clients", "produits", "commandes", "lignes_commande", "retours", "jours_exploitation"]:
    n = con.execute(f"select count(*) from {t}").fetchone()[0]
    print(f"{t:20s} {n:>7,d} lignes".replace(",", " "))
print("commandes distinctes dans les lignes :", con.execute("select count(distinct id_commande) from
↳ lignes_commande").fetchone()[0])
print("lignes par commande en moyenne :", round(con.execute("select count(*)*1.0/count(distinct
↳ id_commande) from lignes_commande").fetchone()[0], 2))
```

```
clients          6 000 lignes
produits         120 lignes
commandes        36 395 lignes
lignes_commande  83 905 lignes
retours          5 002 lignes
jours_exploitation 1 096 lignes
commandes distinctes dans les lignes : 36395
lignes par commande en moyenne : 2.31
```

Le **grain** est la ligne de commande : une commande contient plusieurs lignes, et un retour porte sur une ligne. Retenez-le : tout calcul de « nombre de commandes » doit compter des commandes **distinctes**, pas des lignes.

P.3 Étape 2 : lire le fichier brut de la caisse

L'export de la caisse est un fichier à **la française** : encodage cp1252, séparateur ;, virgule décimale, dates jj/mm/aaaa, trois lignes de titre, un en-tête répété à chaque « page », une ligne de total, des cellules vides. On l'ouvre sans rien deviner (livre, 4.3 et 2.3).

```
with open("donnees/export_caisse_brut.csv", encoding="cp1252") as f:
    lignes = f.read().splitlines()
```

```
print("lignes du fichier :", len(lignes))
print(*lignes[:5], sep="\n")
print("...")
print(lignes[-1])
```

```
lignes du fichier : 289
Export caisse - Boutique;;;;;;
Période du 03/11/2025 au 09/11/2025;;;;;;
;;;;;;
N° ticket;Date;Heure;Article;Catégorie;Qté;Prix unitaire;Montant
T33133;03/11/2025;09:00;BOÎTE RUSTIQUE;Maison;2;52,43;104,86
...
;;;;;;Total;11561,47
```

On saute les trois lignes de titre, on lit tout en **texte**, on retire la ligne de total (que l'on garde de côté pour le recouplement) et les en-têtes répétés, puis on convertit les types explicitement.

```
total_affiche = float(lignes[-1].split(";")[-1].replace(",", "."))
corps = "\n".join(lignes[3:-1])
brut = pd.read_csv(io.StringIO(corps), sep=";", dtype=str)
brut = brut[brut["N° ticket"] != "N° ticket"].copy()
brut.columns = ["ticket", "date", "heure", "article", "categorie", "qte", "prix_unitaire", "montant"]
brut["date"] = pd.to_datetime(brut["date"], format="%d/%m/%Y")
brut["qte"] = brut["qte"].astype(int)
for c in ["prix_unitaire", "montant"]:
    brut[c] = brut[c].str.replace(",", ".").astype(float)
brut["article"] = brut["article"].str.capitalize()
brut["categorie"] = brut["categorie"].str.capitalize()
print(brut.shape, "| montants manquants :", int(brut["montant"].isna().sum()))
print("somme des montants présents :", round(brut["montant"].sum(), 2), "| total affiché par la  
↪ caisse :", total_affiche)
```

```
(280, 8) | montants manquants : 8
somme des montants présents : 11250.01 | total affiché par la caisse : 11561.47
```

Lecture. Le fichier compte 280 lignes d'achats, dont 8 sans montant. La somme des montants présents est de 11 250,01 €, alors que la caisse affiche 11 561,47 € : il manque **311,46 €**, soit 2,7 % du total.

La somme des montants lus **ne retombe pas** sur le total affiché : l'écart vient des cellules vides. On ne comble pas à l'aveugle : on cherche la cause, puis on **réconcilie** avec une source fiable.

P.4 Étape 3 : réconcilier la caisse et la base

La même semaine existe dans la base de données (canal « Boutique », commandes du 3 au 9 novembre 2025). On recalcule le total par SQL, ligne par ligne, et on compare (livre, 3.2 et 3.4).

```
sql_semaine = """
select c.id_commande, c.date_commande, l.id_ligne, p.nom_produit, p.categorie, l.quantite,
↪ l.prix_unitaire, l.remise_pct, l.montant
from commandes c join lignes_commande l using (id_commande) join produits p using (id_produit)
where c.canal = 'Boutique' and c.date_commande between '2025-11-03' and '2025-11-09'
"""
base = pd.read_sql(sql_semaine, con)
print("lignes dans la base :", len(base), "| lignes dans l'export :", len(brut))
print("CA de la base :", round(base["montant"].sum(), 2), "| total affiché par la caisse :",
↪ total_affiche)
brut["id_commande"] = brut["ticket"].str[1:].astype(int)
manquants = brut[brut["montant"].isna()]
print("tickets concernés par un montant manquant :", manquants["id_commande"].nunique())
```

```
lignes dans la base : 280 | lignes dans l'export : 280
CA de la base : 11561.47 | total affiché par la caisse : 11561.47
tickets concernés par un montant manquant : 8
```

Lecture. La base contient exactement les mêmes 280 lignes que l'export, et son chiffre d'affaires (11 561,47 €) est **égal au total affiché par la caisse**. Les 8 montants vides concernent 8 tickets différents.

Le total affiché par la caisse est **identique** à celui de la base : la caisse avait raison, c'est l'**export** qui a perdu des montants. On peut alors reconstituer les montants manquants à partir de la base et vérifier que la somme complète retombe exactement.

```
cle = ["id_commande", "nom_produit", "quantite", "prix_unitaire"]
brut2 = brut.rename(columns={"article": "nom_produit", "qte": "quantite"}).copy()
b = base.copy()
for d in (brut2, b):
    d["nom_produit"] = d["nom_produit"].str.capitalize()
    d["rang"] = d.groupby(cle).cumcount()          # distingue deux lignes identiques d'un même ticket
fusion = brut2.merge(b[cle + ["rang", "montant"]], on=cle + ["rang"], how="left", suffixes=("",
↳ "_base"))
print("lignes de l'export sans correspondance dans la base :",
↳ int(fusion["montant_base"].isna().sum()), "| lignes après fusion :", len(fusion))
fusion["montant_complet"] = fusion["montant"].fillna(fusion["montant_base"])
print("somme complète :", round(fusion["montant_complet"].sum(), 2), "| total de la caisse :",
↳ total_affiche)
print("écart :", round(fusion["montant_complet"].sum() - total_affiche, 2))
```

```
lignes de l'export sans correspondance dans la base : 0 | lignes après fusion : 280
somme complète : 11561.47 | total de la caisse : 11561.47
écart : 0.0
```

Lecture. Aucune ligne de l'export n'est sans correspondance, et la somme complète retombe **exactement** sur le total de la caisse (écart nul). La clé de rapprochement comprend un **rang** parce que deux lignes d'un même ticket peuvent être identiques (même article, même quantité, même prix) : sans lui, la fusion dupliquerait des lignes et le total serait faux.

À retenir. Une réconciliation se fait en trois temps : **compter** (mêmes lignes ?), **sommer** (même total ?), **expliquer** (d'où vient l'écart ?). Ici l'écart venait de cellules vides dans l'export ; la base, source de vérité, a permis de les reconstituer.

P.5 Étape 4 : la synthèse du dernier trimestre, par trois outils

On calcule le chiffre d'affaires et le nombre de commandes du **quatrième trimestre** (octobre à décembre) de 2024 et de 2025, par canal. La même question, posée à **SQL**, à **pandas** et à **R**, doit donner les mêmes chiffres (livre, 3.4, 4.1 et 4.2).

```
sql_t4 = """
select strftime('%Y', c.date_commande) as annee, c.canal, count(distinct c.id_commande) as commandes,
↳ round(sum(l.montant), 2) as ca
from commandes c join lignes_commande l using (id_commande)
where strftime('%m', c.date_commande) in ('10', '11', '12') and strftime('%Y', c.date_commande) in
↳ ('2024', '2025')
group by annee, c.canal order by c.canal, annee
"""
t4_sql = pd.read_sql(sql_t4, con)
print(t4_sql.to_string(index=False))
```

annee	canal	commandes	ca
2024	Boutique	1834	175280.54
2025	Boutique	1846	185292.97
2024	Réseaux	433	39807.02
2025	Réseaux	511	51073.62
2024	Site	1846	175635.46
2025	Site	2155	211433.79

```
cmd = pd.read_sql("select * from commandes", con, parse_dates=["date_commande"])
lig = pd.read_sql("select * from lignes_commande", con)
x = lig.merge(cmd[["id_commande", "date_commande", "canal", "id_client", "code_promo"]],
↳ on="id_commande")
x["annee"], x["mois"] = x["date_commande"].dt.year, x["date_commande"].dt.month
t4 = x[(x["mois"] >= 10) & x["annee"].isin([2024, 2025])]
t4_pd = t4.groupby(["canal", "annee"]).agg(commandes="id_commande", "nunique"), ca=("montant",
↳ "sum")).round(2).reset_index()
print("pandas = SQL :", np.allclose(t4_pd["ca"].values, t4_sql["ca"].values) and
↳ (t4_pd["commandes"].values == t4_sql["commandes"].values).all())
```

pandas = SQL : True

Lecture. SQL et pandas donnent les mêmes six lignes : au quatrième trimestre 2025, le chiffre d'affaires est de 185 293 € pour la Boutique, 51 074 € pour les Réseaux et 211 434 € pour le Site.

```
library(dplyr, warn.conflicts = FALSE)
dossier <- Sys.getenv("DONNEES")
lig <- read.csv(file.path(dossier, "lignes_commande.csv")); cmd <- read.csv(file.path(dossier,
↳ "commandes.csv"))
x <- lig |> inner_join(cmd, by = "id_commande") |>
  mutate(annee = as.integer(substr(date_commande, 1, 4)), mois = as.integer(substr(date_commande, 6,
↳ 7))) |>
  filter(mois >= 10, annee %in% c(2024, 2025))
r <- x |> group_by(canal, annee) |> summarise(commandes = n_distinct(id_commande), ca =
↳ round(sum(montant), 2), .groups = "drop") |> arrange(canal, annee)
print(as.data.frame(r))
```

	canal	annee	commandes	ca
1	Boutique	2024	1834	175280.54
2	Boutique	2025	1846	185292.97
3	Réseaux	2024	433	39807.02
4	Réseaux	2025	511	51073.62
5	Site	2024	1846	175635.46
6	Site	2025	2155	211433.79

Lecture. R retrouve les mêmes totaux (le nom des colonnes et l'ordre des lignes différent à peine). Trois langages, un seul résultat : on peut passer à la suite.

Les trois outils donnent les mêmes totaux. Ce n'est pas un détail : si deux outils divergent, **l'un des deux calcule autre chose** (un filtre de dates différent, une jointure qui duplique des lignes, un arrondi).

P.6 Étape 5 : les indicateurs de la synthèse

On complète avec le **panier moyen** (CA divisé par le nombre de commandes), le **taux de retour** (lignes retournées sur lignes vendues), la **marge brute** et l'évolution en pourcentage. Deux pièges : le panier moyen d'un total n'est pas la moyenne des paniers moyens des canaux, et une marge se calcule **hors taxe** (livre, 1.1 et 1.5).

```
TVA = 0.20      # taux de TVA fictif, pour l'illustration
prod = pd.read_sql("select id_produit, cout_achat from produits", con)
ret = pd.read_sql("select id_ligne, montant_rembourse from retours", con)
t4 = t4.merge(prod, on="id_produit").merge(ret, on="id_ligne", how="left")
t4["retourne"] = t4["montant_rembourse"].notna()
t4["marge_ht"] = t4["montant"] / (1 + TVA) - t4["quantite"] * t4["cout_achat"]
syn = t4.groupby(["canal", "annee"]).agg(ca=("montant", "sum"), commandes=("id_commande", "nunique"),
    ↪ lignes=("id_ligne", "size"),
    ↪ retours=("retourne", "sum"), marge=("marge_ht",
    ↪ "sum")).reset_index()
syn["panier_moyen"] = syn["ca"] / syn["commandes"]
syn["taux_retour"] = syn["retours"] / syn["lignes"]
syn["taux_marge"] = syn["marge"] / (syn["ca"] / (1 + TVA))
print(syn.round(3).to_string(index=False))
```

canal	annee	ca	commandes	lignes	retours	marge	panier_moyen	taux_retour	taux_marge
Boutique	2024	175280.54	1834	4154	107	53517.907	95.573	0.026	0.366
Boutique	2025	185292.97	1846	4266	150	58902.348	100.375	0.035	0.381
Réseaux	2024	39807.02	433	1003	81	11998.227	91.933	0.081	0.362
Réseaux	2025	51073.62	511	1185	90	16315.470	99.948	0.076	0.383
Site	2024	175635.46	1846	4261	371	53514.863	95.144	0.087	0.366
Site	2025	211433.79	2155	4911	419	67718.105	98.113	0.085	0.384

Lecture. Le Site a le taux de retour le plus élevé (8,5 % des lignes en 2025, contre 3,5 % pour la Boutique et 7,6 % pour les Réseaux). Le taux de marge brute, hors taxe, est voisin de 38 % pour les trois canaux en 2025, et de 36–37 % en 2024.

```
tot = t4.groupby("annee").agg(ca=("montant", "sum"), commandes=("id_commande", "nunique"),
    ↪ retours=("retourne", "sum"), lignes=("id_ligne", "size"))
tot["panier_moyen"] = tot["ca"] / tot["commandes"]
moyenne_des_moyennes = syn[syn["annee"] == 2025]["panier_moyen"].mean()
print("panier moyen 2025 (total) :", round(tot.loc[2025, "panier_moyen"], 2), "| moyenne des paniers
    ↪ moyens des canaux :", round(moyenne_des_moyennes, 2))
evo = syn.pivot(index="canal", columns="annee", values="ca")
print("évolution du CA 2025 / 2024 :", ((evo[2025] / evo[2024] - 1) * 100).round(1).to_dict(), "|
    ↪ total :", round((tot.loc[2025, "ca"] / tot.loc[2024, "ca"] - 1) * 100, 1))
```

```
panier moyen 2025 (total) : 99.25 | moyenne des paniers moyens des canaux : 99.48
évolution du CA 2025 / 2024 : {'Boutique': 5.7, 'Réseaux': 28.3, 'Site': 20.4} | total : 14.6
```

Lecture. Le panier moyen du total (99,25 €) diffère de la moyenne des paniers moyens des canaux (99,48 €) : c'est le **poids des commandes** de chaque canal qui explique l'écart, d'où la règle de calculer le panier moyen sur les totaux. Le chiffre d'affaires du trimestre progresse de 14,6 % ; la progression est de 5,7 % pour la Boutique, 20,4 % pour le Site et 28,3 % pour les Réseaux. En euros, le Site contribue le plus (+35 798 €).

P.7 Étape 6 : le classeur de synthèse

La gérante veut un **classeur**, avec des formules qu'elle peut inspecter. On y met les données agrégées (canal × année) sur une feuille et la synthèse sur une autre, avec de vraies formules ; on les **fait recalculer** par LibreOffice pour vérifier que le classeur affiche les mêmes chiffres que Python (livre, 2.1 et 2.4). Excel lui-même n'est pas installé : la vérification est faite avec LibreOffice Calc.

```
import sys, os, tempfile
sys.path.insert(0, "build")
import outils_xl as X
dossier = tempfile.mkdtemp(dir=os.environ.get("TMPDIR"))
donnees = [{"canal", "annee", "ca", "commandes", "retours", "lignes"}] + [
    [r.canal, int(r.annee), round(float(r.ca), 2), int(r.commandes), int(r.retours), int(r.lignes)]
    for r in syn.itertuples()]
synthese = [{"Canal", "CA 2024", "CA 2025", "Évolution", "Panier moyen 2025", "Taux de retour 2025"}]
for i, canal in enumerate(["Boutique", "Réseaux", "Site"], start=2):
    synthese.append([canal, f'=SUMIFS(Donnees!C:C,Donnees!A:A,A{i},Donnees!B:B,2024)',
        f'=SUMIFS(Donnees!C:C,Donnees!A:A,A{i},Donnees!B:B,2025)',
        f'=C{i}/B{i}-1', f'=C{i}/SUMIFS(Donnees!D:D,Donnees!A:A,A{i},Donnees!B:B,2025)',
        f'=SUMIFS(Donnees!E:E,Donnees!A:A,A{i})',
        f'=SUMIFS(Donnees!B:B,2025)/SUMIFS(Donnees!F:F,Donnees!A:A,A{i},Donnees!B:B,2025)'])
synthese.append(["Total", "=SUM(B2:B4)", "=SUM(C2:C4)", "=C5/B5-1", None, None])
chemin = X.classeur(os.path.join(dossier, "synthese_T4.xlsx"), {"Synthese": synthese, "Donnees":
    donnees})
val = X.valeurs(X.recalculer(chemin), "Synthese")
for ligne in val:
    print([round(v, 3) if isinstance(v, float) else v for v in ligne])
```

```
['Canal', 'CA 2024', 'CA 2025', 'Évolution', 'Panier moyen 2025', 'Taux de retour 2025']
['Boutique', 175280.54, 185292.97, 0.057, 100.375, 0.035]
['Réseaux', 39807.02, 51073.62, 0.283, 99.948, 0.076]
['Site', 175635.46, 211433.79, 0.204, 98.113, 0.085]
['Total', 390723.02, 447800.38, 0.146, None, None]
```

```
ca25 = syn[syn["annee"] == 2025].set_index("canal")["ca"]
ok = all(abs(val[i][2] - ca25[val[i][0]]) < 0.01 for i in range(1, 4))
print("formules du classeur = résultats de pandas :", ok, "| formule de l'évolution (affichage
    français) :", X.en_fr("=C2/B2-1"))
print("formule du CA 2025 (affichage français) :", X.en_fr(synthese[1][2]))
```

```
formules du classeur = résultats de pandas : True | formule de l'évolution (affichage français) :
    =C2/B2-1
formule du CA 2025 (affichage français) : =SOMME.SI.ENS(Donnees!C:C;Donnees!A:A;A2;Donnees!B:B;2025)
```

Lecture. Les formules du classeur donnent les mêmes valeurs que pandas. La seconde ligne montre la formule telle qu'elle s'afficherait dans un Excel en français : `SOMME.SI.ENS` avec le séparateur ;.

Attention. Un classeur n'est fiable que si les **entrées** sont séparées des **calculs** et si le résultat est recoupé. Ici, les chiffres affichés par les formules sont comparés à ceux de pandas : sans cette comparaison, une erreur de plage ou un 2024 en dur passe inaperçu.

C2	fx	=SOMME.SI.ENS(Donnees!C:C;Donnees!A:A;A2;Donnees!B:B;2025)				
	A	B	C	D	E	F
1	Canal	CA 2024	CA 2025	Évolution	Panier moyen 2025	Taux de retour 2025
2	Boutique	175 280,54	185 292,97	5,7 %	100,38	3,5 %
3	Réseaux	39 807,02	51 073,62	28,3 %	99,95	7,6 %
4	Site	175 635,46	211 433,79	20,4 %	98,11	8,5 %
5	Total	390 723,02	447 800,38	14,6 %		
Synthese		Donnees				

Figure 2: Maquette du classeur de synthèse : la cellule active contient la formule du chiffre d'affaires 2025 de la Boutique. Maquette dessinée avec matplotlib à partir des valeurs recalculées par LibreOffice, pas une capture d'Excel.

P.8 Étape 7 : le graphique

Un seul graphique, qui répond à la question de la gérante : **qu'est-ce qui a changé, et où ?** On montre le chiffre d'affaires par canal pour les deux trimestres, avec l'évolution écrite sur les barres (livre, 1.1).

```
import matplotlib.pyplot as plt
from style import setup, BLEU, ORANGE, MUET
setup()
canaux = ["Boutique", "Réseaux", "Site"]
fig, ax = plt.subplots(figsize=(6.4, 3.4))
l = np.arange(3); w = 0.36
a24 = [evo.loc[c, 2024] / 1000 for c in canaux]; a25 = [evo.loc[c, 2025] / 1000 for c in canaux]
ax.bar(l - w / 2, a24, w, color=MUET, label="T4 2024"); ax.bar(l + w / 2, a25, w, color=BLEU,
    label="T4 2025")
for i, c in enumerate(canaux):
    ax.text(i + w / 2, a25[i] + 3, f"{{(a25[i] / a24[i] - 1) * 100:+.0f}} %".replace(".", ","),
        ha="center", fontsize=9, color=BLEU)
ax.set_xticks(l); ax.set_xticklabels(canaux); ax.set_ylabel("Chiffre d'affaires (k€)");
    ax.set_ylim(0, 250); ax.legend(frameon=False, loc="upper center", ncol=2)
ax.set_title("Quatrième trimestre : 2025 contre 2024, par canal", loc="left")
fig.savefig("figures/ch10-synthese-t4.png", dpi=200, bbox_inches="tight"); plt.close(fig)
print("figure écrite")
```

figure écrite

P.9 Étape 8 : le message à la gérante

Le message tient en cinq lignes, chacune avec **un chiffre et sa base de comparaison**, plus ce que l'analyse ne dit pas. Voici la trame.

Objet : synthèse du quatrième trimestre 2025.

1. Le chiffre d'affaires du trimestre est de X €, en hausse de a % sur le même trimestre de 2024 ; la plus forte hausse en euros vient du canal Y.
2. Le nombre de commandes progresse de b % ; le panier moyen est de Z € (calculé sur le total, pas sur la moyenne des canaux).
3. Le taux de retour est de r % ; il est plus élevé sur le Site.
4. La marge brute (hors taxe) représente m % du chiffre d'affaires hors taxe.
5. Les totaux ont été recoupés par SQL, pandas et R, et le classeur par LibreOffice : aucun écart.

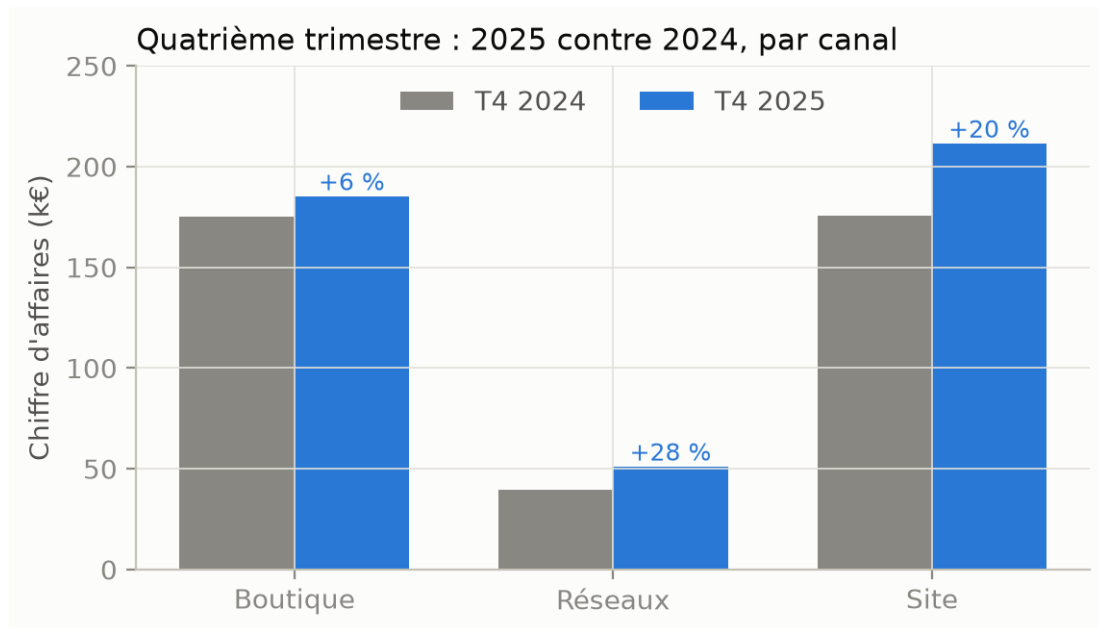


Figure 3: Chiffre d'affaires du quatrième trimestre par canal, 2024 et 2025, avec l'évolution en pourcentage.

Ce que cette analyse ne dit pas : *pourquoi* les ventes ont évolué (promotions, saison, publicité : voir le volume III) ; si les clients reviendront ; la marge après frais de personnel et de livraison.

On remplit la trame **par le calcul**, pas à la main, pour éviter toute faute de recopie.

```
t25, t24 = tot.loc[2025], tot.loc[2024]
site = syn[(syn["annee"] == 2025)].set_index("canal")
fr = lambda x, nd=1: f"{x:,.{nd}f}".replace(",", " ").replace(".", ",")
gain = (evo[2025] - evo[2024])
print(f"1) CA : {fr(t25['ca'], 0)} € ({fr((t25['ca'] / t24['ca'] - 1) * 100)} %) ; plus forte hausse
↳ en euros : {gain.idxmax()} ({fr(gain.max(), 0)} €)")
print(f"2) Commandes : {fr((t25['commandes'] / t24['commandes'] - 1) * 100)} % ; panier moyen :
↳ {fr(t25['panier_moyen'], 2)} €")
print(f"3) Taux de retour : {fr(t25['retours'] / t25['lignes'] * 100)} % ; le plus élevé :
↳ {site['taux_retour'].idxmax()} ({fr(site['taux_retour'].max() * 100)} %)")
print(f"4) Taux de marge brute (HT) : {fr(syn[syn['annee'] == 2025]['marge'].sum() / (t25['ca'] / (1 +
↳ TVA)) * 100)} %")
```

1) CA : 447 800 € (14,6 %) ; plus forte hausse en euros : Site (+35 798 €)
 2) Commandes : 9,7 % ; panier moyen : 99,25 €
 3) Taux de retour : 6,4 % ; le plus élevé : Site (8,5 %)
 4) Taux de marge brute (HT) : 38,3 %

P.10 Les limites de l'étude

- **Une description, pas une explication.** Le tableau dit *ce qui s'est passé*, pas *pourquoi* ; la promotion, la saison et la publicité jouent ensemble (volume III).
- **Deux trimestres seulement.** Une évolution de quelques pour cent sur un trimestre peut être de la variation ordinaire ; on ne la compare pas à une tendance sur plusieurs années.
- **Hypothèses de marge.** La TVA est fictive et le coût d'achat est supposé constant ; les frais de livraison et de personnel ne sont pas comptés.
- **Taux de retour mesuré sur les lignes vendues** dans le trimestre, retours **comptés à la date de vente** ; les retours de décembre arrivent en janvier et sont **sous-estimés** pour les der-

nières semaines.

- **Données simulées.** La réalité apporte des cas que ce jeu ne contient pas (remboursements partiels, annulations, doublons de commandes).

P.11 Variante : l'enquête de satisfaction

La même démarche s'applique à `donnees/enquete_satisfaction.csv` : **lire**, **nettoyer** (doublons, réponses trop rapides), **calculer** une satisfaction moyenne et un indicateur de recommandation (*Net Promoter Score*), puis **douter** (qui a répondu ?). Voici la version courte.

```
enq = pd.read_csv("donnees/enquete_satisfaction.csv")
n0 = len(enq)
enq = enq.drop_duplicates(subset=[c for c in enq.columns if c != "id_reponse"])
n1 = len(enq)
enq = enq[enq["duree_reponse_s"] >= 20]
print("réponses brutes :", n0, "| après doublons :", n1, "| après réponses trop rapides :", len(enq))
reco = enq["recommandation_0_10"]
nps = ((reco >= 9).mean() - (reco <= 6).mean()) * 100
print("satisfaction moyenne :", round(enq["satisfaction_globale"].mean(), 2), "| NPS :", round(nps,
    ↵ 1))
inv = pd.read_sql("select distinct id_client from commandes where date_commande >= '2025-01-01'", con)
print("clients invités :", len(inv), "| taux de réponse :", round(len(enq) / len(inv) * 100, 1), "%")
```

```
réponses brutes : 958 | après doublons : 931 | après réponses trop rapides : 897
satisfaction moyenne : 3.58 | NPS : -21.1
clients invités : 3875 | taux de réponse : 23.1 %
```

Lecture. Sur 958 lignes brutes, on retire 27 doublons puis 34 réponses remplies en moins de 20 secondes : il reste 897 réponses, pour 3 875 clients invités, soit un taux de réponse de 23,1 %. La satisfaction moyenne est de 3,58 sur 5 et le NPS de −21 points. **Attention** : ces chiffres décrivent les **répondants**, et rien ne garantit qu'ils ressemblent aux clients (chapitre 5, section 5.3) ; on ne les annonce pas comme « la satisfaction des clients ».

Auto-évaluation

Mode d'emploi. Répondez à voix haute ou par écrit **avant** de lire le corrigé, en une ou deux phrases. Les sections à relire sont indiquées dans le corrigé. Trente-cinq bonnes réponses sur quarante signalent un volume bien assimilé ; les questions des sections facultatives (⊕ 1.5, 2.5, 2.6, 3.5, 3.6, 4.5, 4.6, 5.4, 5.5) comptent si vous les avez lues.

Les essentiels de la statistique (chapitre 1)

1. Neuf commandes valent 20, 25, 30, 35, 40, 45, 50, 60 et 260 €. Calculez la moyenne et la médiane. Laquelle annoncez-vous à la gérante pour décrire « une commande typique », et pourquoi ?
2. Le premier quartile des paniers est de 20 € et le troisième de 40 €. Quels sont les seuils de la règle des 1,5 écart interquartile ? Une commande de 95 € est-elle « aberrante » ? Doit-on la supprimer ?
3. Le panier moyen vaut 100 € avec un écart-type de 40 €. Quel est le score z d'une commande de 180 € ? Si les paniers suivaient une loi normale, quelle proportion dépasserait 180 € ? Pourquoi cette réponse est-elle probablement fausse pour nos paniers ?
4. Le taux de retour est de 6 %. Sur 200 lignes vendues, combien de retours attend-on, avec quel écart-type ?

5. Un échantillon de 400 commandes donne un panier moyen de 100,4 € et un écart-type de 85 €. Donnez l'erreur type et un intervalle de confiance à 95 % de la moyenne.
6. Combien de personnes faut-il interroger pour estimer une proportion à plus ou moins 3 points, avec 95 % de confiance, dans le pire cas ?
7. La dépense publicitaire et le chiffre d'affaires sont corrélés (0,42), mais la corrélation tombe à presque rien à mois égal. Que s'est-il passé, et que conclure ?
8. Un prix augmente de 20 % puis baisse de 20 %. Le résultat est-il le prix initial ? Un taux de retour passe de 6 % à 8 % : de combien en points, et en pourcentage ? Quelle croissance annuelle moyenne mène de 100 à 121 en deux ans ?

Excel (chapitre 2)

9. En C2, la formule =B2*\$F\$1 est copiée vers le bas jusqu'en C3. Que devient-elle, et pourquoi le \$?
10. Écrivez la formule qui donne le chiffre d'affaires du canal « Site » depuis le 1er octobre 2025 à partir de la feuille Lignes (canal en colonne E, date en C, montant en M).
11. Citez deux pièges de RECHERCHEV et la fonction qui les évite.
12. La colonne A contient des numéros de ticket comme T33133. Comment extraire le nombre 33133 (en nombre, pas en texte) ?
13. Que renvoie =FIN.MOIS(DATE(2025;11;15);0) ? Et =DATE(2025;11;15)+30 ?
14. Un tableau croisé dynamique n'affiche pas les ventes d'hier. Quelles sont les deux causes les plus fréquentes ?
15. Quelle structure de classeur sépare données, calculs et présentation ? Pourquoi éviter les cellules fusionnées dans une table de données ?
16. Qu'apportent les « étapes appliquées » de Power Query par rapport à un nettoyage fait à la main dans une feuille ?

SQL (chapitre 3)

17. Pourquoi ne peut-on pas écrire WHERE SUM(montant) > 1000 ? Quelle clause utiliser ?
18. Une colonne contient cinq valeurs : 10, 20, NULL, 30, NULL. Que valent COUNT(*), COUNT(colonne) et AVG(colonne) ?
19. Combien de clients de la base n'ont **jamais** commandé ? Écrivez la requête.
20. On joint la table des jours (une ligne par jour, avec la dépense publicitaire) à la table des commandes (plusieurs par jour) puis on somme la dépense. Quel est le problème, et quelle est l'ampleur de l'erreur sur 2025 ?
21. Quatre produits ont des ventes de 90, 80, 80 et 70. Donnez ROW_NUMBER, RANK et DENSE_RANK.
22. Le chiffre d'affaires de trois mois vaut 100, 110 et 99. Quelle fonction fenêtre donne la variation par rapport au mois précédent ? Quelles valeurs ?
23. À quoi sert une CTE (WITH), et pourquoi vérifie-t-on un résultat SQL par un second outil ?
24. Pourquoi ne faut-il jamais construire une requête en collant du texte saisi par un utilisateur, et que faire à la place ?

Python et R (chapitre 4)

25. Quelle est la différence entre loc et iloc ? Pourquoi les conditions combinées s'écrivent-elles (a) & (b) entre parenthèses ?
26. Quels arguments de lecture faut-il pour un fichier CSV français (point-virgule, virgule décimale, accents Windows) ?
27. En 2025, combien y a-t-il de lignes de commande et combien de commandes distinctes ? Quelle fonction pandas compte les commandes distinctes ?

28. Comment détecter, au moment de la jointure de deux tables avec pandas, qu'une clé est en double dans la table que l'on croit sans doublon ?
29. Traduisez en tidyverse : « garder les commandes du Site, ajouter le montant hors taxe, grouper par mois, sommer ».
30. Quand préfère-t-on le format large et quand le format long ? Quelles fonctions passent de l'un à l'autre en pandas et en R ?
31. Pourquoi un notebook qui « marche chez moi » peut-il ne pas se rejouer chez un collègue, et quelle habitude évite le problème ?
32. Pourquoi la vectorisation (NumPy, pandas, polars) est-elle préférable à une boucle Python sur les lignes ?

Types de données, collecte et enquêtes (chapitre 5)

33. Classez par niveau de mesure : la note de satisfaction de 1 à 5, la ville, la température en °C, le chiffre d'affaires. Quelles opérations a-t-on le droit de faire sur chacune ?
34. Quel est le grain de la table commandes ? De la table lignes_commande ? Quel risque court-on en joignant les deux et en sommant une colonne de la première ?
35. Pourquoi lire un identifiant comme 007421 en texte ?
36. Seuls 24 % des clients invités ont répondu, avec une moyenne de 3,64. Entre quelles valeurs la moyenne de **tous** les clients peut-elle se situer, sans autre hypothèse ?
37. Sur 50 réponses de recommandation (0 à 10), 20 sont des 9 ou 10, 15 des 7 ou 8, et 15 des 0 à 6. Quel est le NPS ?
38. Combien de répondants pour une marge d'erreur de 3 points, et pour 2 points ?
39. Citez quatre biais d'enquête et un remède pour chacun.
40. Une API répond « 429 » : que signifie ce code et comment se comporter ? Pourquoi consulter robots.txt avant de collecter une page ?

Corrigés des questions

Pour les questions numériques, **calculons** plutôt que de nous fier à la mémoire. Les formules Excel sont vérifiées avec LibreOffice (Excel n'est pas installé) et recoupées par pandas.

```
import numpy as np, pandas as pd, sqlite3, datetime as dt
from scipy.stats import norm

paniers = [20, 25, 30, 35, 40, 45, 50, 60, 260]
print("Q1 moyenne :", round(np.mean(paniers), 1), "| médiane :", np.median(paniers))
print("Q2 seuils :", 20 - 1.5 * 20, "et", 40 + 1.5 * 20)
print("Q3 z =", (180 - 100) / 40, "| P(> 180) si normal :", round(1 - norm.cdf(2), 4))
print("Q4 retours attendus :", 200 * 0.06, "| écart-type :", round(np.sqrt(200 * 0.06 * 0.94), 2))
se = 85 / np.sqrt(400)
print("Q5 erreur type :", round(se, 2), "| IC 95 % :", round(100.4 - 1.96 * se, 1), "à", round(100.4
    + 1.96 * se, 1))
print("Q6 n =", round(1.96 ** 2 * 0.25 / 0.03 ** 2))
print("Q8 1,2 x 0,8 =", round(1.2 * 0.8, 2), "| 6 % -> 8 % :", 2, "points", round((8 / 6 - 1) * 100,
    1), "% | croissance annuelle :", round((1.21 ** 0.5 - 1) * 100, 1), "%")
print("Q36 bornes :", round(0.24 * 3.64 + 0.76 * 1, 2), "à", round(0.24 * 3.64 + 0.76 * 5, 2))
print("Q37 NPS :", (20 - 15) / 50 * 100, "points | Q38 n pour 3 points :", round(1.96 ** 2 * 0.25 /
    0.03 ** 2), "pour 2 points :", round(1.96 ** 2 * 0.25 / 0.02 ** 2))
```

```
Q1 moyenne : 62.8 | médiane : 40.0
Q2 seuils : -10.0 et 70.0
Q3 z = 2.0 | P(> 180) si normal : 0.0228
Q4 retours attendus : 12.0 | écart-type : 3.36
Q5 erreur type : 4.25 | IC 95 % : 92.1 à 108.7
```

Q6 n = 1067
 Q8 $1,2 \times 0,8 = 0.96$ | 6 % -> 8 % : 2 points, 33.3 % | croissance annuelle : 10.0 %
 Q36 bornes : 1.63 à 4.67
 Q37 NPS : 10.0 points | Q38 n pour 3 points : 1067 , pour 2 points : 2401

```
con = sqlite3.connect("donnees/boutique.db")
print("Q18 COUNT(*), COUNT(col), AVG :", con.execute("with t(v) as (values
↳ (10),(20),(NULL),(30),(NULL)) select count(*), count(v), avg(v) from t").fetchone())
print("Q19 clients sans commande :", con.execute("select count(*) from clients c left join commandes o
↳ on o.id_client = c.id_client where o.id_commande is null").fetchone()[0])
juste = con.execute("select round(sum(depense_pub)) from jours_exploitation where date >=
↳ '2025-01-01').fetchone()[0]
faux = con.execute("select round(sum(j.depense_pub)) from jours_exploitation j join commandes c on
↳ c.date_commande = j.date where j.date >= '2025-01-01').fetchone()[0]
print("Q20 dépense 2025 :", juste, "€ | après jointure :", faux, "€ | rapport :", round(faux / juste,
↳ 1))
print("Q21", con.execute("with t(p, v) as (values ('a',90),('b',80),('c',80),('d',70)) select p,
↳ row_number() over (order by v desc), rank() over (order by v desc), dense_rank() over (order by v
↳ desc) from t").fetchall())
print("Q22", con.execute("with t(m, v) as (values (1,100.0),(2,110.0),(3,99.0)) select m, round(v /
↳ lag(v) over (order by m) - 1, 3) from t").fetchall())
print("Q27 lignes, commandes distinctes (2025) :", con.execute("select count(*), count(distinct
↳ c.id_commande) from commandes c join lignes_commande l using (id_commande) where c.date_commande
↳ >= '2025-01-01').fetchone())
```

Q18 COUNT(*), COUNT(col), AVG : (5, 3, 20.0)
 Q19 clients sans commande : 1194
 Q20 dépense 2025 : 75995.0 € | après jointure : 2991731.0 € | rapport : 39.4
 Q21 [('a', 1, 1, 1), ('b', 2, 2, 2), ('c', 3, 2, 2), ('d', 4, 4, 3)]
 Q22 [(1, None), (2, 0.1), (3, -0.1)]
 Q27 lignes, commandes distinctes (2025) : (29827, 12946)

```
import sys, os, tempfile
sys.path.insert(0, "build")
import outils_xl as X
lignes = pd.read_excel("donnees/ventes_2025.xlsx", sheet_name="Lignes")
rows = [list(lignes.columns)] + [(v.date() if isinstance(v, pd.Timestamp) else (None if pd.isna(v)
↳ else (v.item() if hasattr(v, "item") else v))) for v in r] for r in
↳ lignes.itertuples(index=False)]
f10 = '=SUMIFS(Lignes!M:M,Lignes!E:E,"Site",Lignes!C:C,">="&DATE(2025,10,1))'
calc = [[f10, "=EOMONTH(DATE(2025,11,15),0)", "=DATE(2025,11,15)+30", '=VALUE(MID("T33133",2,10))']]
dossier = tempfile.mkdtemp(dir=os.environ.get("TMPDIR"))
res = X.valeurs(X.recalculer(X.classeur(os.path.join(dossier, "q.xlsx"), {"Calcul": calc, "Lignes":
↳ rows})), "Calcul")[0]
attendu = lignes[lignes["canal"] == "Site"] & (lignes["date_commande"] >=
↳ "2025-10-01")["montant"].sum()
print("Q10 formule :", X.en_fr(f10))
print("Q10 LibreOffice :", round(res[0], 2), "| pandas :", round(attendu, 2))
serie = lambda n: n.date() if hasattr(n, 'date') else dt.date(1899, 12, 30) +
↳ dt.timedelta(days=int(n)) # numéro de série Excel -> date
print("Q12 STXT(...) :", res[3], "| Q13 FIN.MOIS :", serie(res[1]), "| +30 jours :", serie(res[2]))
```

Q10 formule : =SOMME.SI.ENS(Lignes!M:M;Lignes!E:E;"Site";Lignes!C:C;">="&DATE(2025;10;1))
 Q10 LibreOffice : 211433.79 | pandas : 211433.79
 Q12 STXT(...) : 33133 | Q13 FIN.MOIS : 2025-11-30 | +30 jours : 2025-12-15

1. Moyenne 62,8 €, médiane **40 €** (code ci-dessus). On annonce la **médiane** : la commande de 260 € tire la moyenne vers le haut, alors que huit commandes sur neuf sont de 60 € ou moins. Une distribution asymétrique se décrit par la médiane, accompagnée des quartiles (1.1.1, 1.1.4).

2. Écart interquartile = $40 - 20 = 20$; seuils $20 - 1,5 \times 20 = -10$ et $40 + 1,5 \times 20 = 70$. La commande de 95 € dépasse 70 € : elle est **signalée**, pas supprimée. On cherche d'abord si c'est une erreur de

saisie ou un vrai gros achat ; une vraie valeur reste dans l'analyse (et se discute à part) (1.1.3).

3. $z = (180 - 100)/40 = 2$: la commande est à deux écarts-types au-dessus de la moyenne. Pour une loi normale, la proportion au-dessus de $z = 2$ vaut environ 2,3 %. Mais les paniers sont **asymétriques**, pas normaux : la règle ne s'applique pas, et il faut lire la distribution réelle (1.2.3, 1.2.4).

4. On attend $200 \times 0,06 = 12$ retours, avec un écart-type de $\sqrt{200 \times 0,06 \times 0,94} \approx 3,4$: trois ou quatre retours de plus ou de moins ne sont pas surprenants (1.2.2).

5. Erreur type = $85/\sqrt{400} = 4,25$ € ; intervalle à 95 % $\approx 100,4 \pm 1,96 \times 4,25$, soit de **92,1 à 108,7** € (1.3.3).

6. $n = 1,96^2 \times 0,25/0,03^2 \approx 1\,067$ répondants (le pire cas est $p = 0,5$) (1.3.5).

7. La dépense publicitaire et les ventes montent **ensemble en novembre-décembre** : la **saison** est une variable de confusion. À mois égal, la corrélation devient presque nulle : le chiffre brut ne prouve pas que la publicité augmente les ventes, et il ne prouve pas non plus qu'elle n'a aucun effet. Pour le savoir, il faut une comparaison contrôlée ou une expérience (1.4.3, 1.4.6).

8. $1,2 \times 0,8 = 0,96$: on perd 4 %, le prix initial n'est pas retrouvé. Un taux de 6 % à 8 % est une hausse de **2 points**, soit **33 %** en valeur relative. De 100 à 121 en deux ans, le taux annuel moyen est de $\sqrt{1,21} - 1 = 10$ % (1.5.1, 1.5.2).

9. Elle devient =B3*\$F\$1 : la référence B2 est **relative** (elle suit la copie), \$F\$1 est **absolue** (le taux reste ancré sur la même cellule) (2.1.2).

10. En français : =SOMME.SI.ENS(Lignes!M:M;Lignes!E:E;"Site";Lignes!C:C;">="&DATE(2025;10;1)). Résultat recalculé par LibreOffice : 211 433,79 €, identique à pandas (code ci-dessus) (2.1.4).

11. (1) La colonne cherchée doit être la **première à gauche** de la table ; (2) le numéro de colonne est écrit en dur et **casse** quand on insère une colonne ; (3) si l'on oublie le dernier argument, la correspondance est **approchée** et le résultat peut être faux sans erreur. RECHERCHEX (ou INDEX + EQUIV) évite ces pièges (2.1.6).

12. =CNUM(STXT(A2;2;10)) : STXT extrait le texte à partir du deuxième caractère, CNUM le convertit en nombre. Résultat : 33133 (code ci-dessus) (2.1.7).

13. FIN.MOIS renvoie le **30/11/2025** ; ajouter 30 à une date donne le **15/12/2025** : une date est un nombre de jours (2.1.8).

14. (1) Le tableau croisé **n'a pas été actualisé** (il garde une copie des données) ; (2) sa **source** est une plage fixe qui n'inclut pas les nouvelles lignes, alors qu'un tableau structuré s'étendrait seul (2.2.5).

15. Trois étages : **données brutes, calculs, présentation**. Les cellules fusionnées cassent le tri, les filtres, les tableaux croisés et la copie de formules : dans une table de données, une ligne est une observation et une colonne une variable (2.4.1, 2.4.2).

16. Les étapes sont **enregistrées et rejouables** : on peut les relancer sur le fichier du mois suivant, les relire, les corriger et les expliquer. Un nettoyage manuel dans une feuille n'est ni reproductible ni traçable (2.3.1).

17. WHERE s'applique **avant** l'agrégation, à des lignes ; la somme n'existe pas encore. On filtre un agrégat avec **HAVING** (3.1.7).

18. COUNT(*) = 5, COUNT(colonne) = 3 (les NULL ne sont pas comptés) et AVG(colonne) = 20 (moyenne de 10, 20 et 30 ; les NULL sont ignorés, pas traités comme des zéros) (3.1.8).

19. 1 194 clients. `SELECT COUNT(*) FROM clients c LEFT JOIN commandes o ON o.id_client = c.id_client WHERE o.id_commande IS NULL` : le `LEFT JOIN` garde tous les clients, le `IS NULL` retient ceux qui n'ont aucune commande (3.2.3).

20. La jointure de « un jour » avec « plusieurs commandes » **répète** la ligne du jour autant de fois qu'il y a de commandes ce jour-là ; la somme de la dépense est donc multipliée. Sur 2025 : 75 995 € devient 2 991 731 €, soit près de **39 fois** trop. On agrège **avant** de joindre, ou l'on joint des tables de même grain (3.2.6).

21. `ROW_NUMBER` : 1, 2, 3, 4 ; `RANK` : 1, 2, 2, 4 ; `DENSE_RANK` : 1, 2, 2, 3 (3.3.3).

22. `LAG` : $v / \text{LAG}(v) - 1$ donne **+10 %** pour le deuxième mois et **-10 %** pour le troisième (code ci-dessus) (3.3.4).

23. Une CTE **nomme une étape** : la requête se lit de haut en bas, chaque étape se vérifie seule. On recoupe par un second outil parce qu'une jointure qui duplique, un filtre de dates décalé ou un arrondi fausse un total **sans erreur visible** (3.4.1, 3.4.2).

24. Du texte saisi peut contenir du SQL (**injection**) et modifier la requête, voire détruire des données. On utilise des **requêtes paramétrées** : la requête et les valeurs sont transmises séparément (3.5.7).

25. `loc` sélectionne par **étiquettes** (bornes incluses), `iloc` par **positions**. Les opérateurs `&`, `|` ont une priorité supérieure aux comparaisons : sans parenthèses, $a > 1 \ \& \ b < 2$ n'a pas le sens voulu, et `and/or` ne fonctionnent pas sur des colonnes entières (4.1.4, 4.1.5).

26. `sep=";"`, `decimal=","` et `encoding="cp1252"`, avec éventuellement `dtype` (identifiants en texte) et `parse_dates` (ou une conversion explicite avec le format `jj/mm/aaaa`) (4.1.3, 4.3.1).

27. 29 827 lignes de commande pour 12 946 commandes distinctes (code ci-dessus). En pandas : `nunique()` (ou `.drop_duplicates()` puis `len`) (4.3.2).

28. Avec `merge(..., validate="m:1")` : pandas lève une erreur si la clé de droite n'est pas unique. On peut aussi contrôler avant : `table["cle"].is_unique` ou `table.duplicated("cle").sum()` (4.3.3).

29. `cmd |> filter(canal == "Site") |> mutate(ca_ht = montant / 1.2) |> group_by(mois) |> summarise(ca = sum(ca_ht))` (4.2.3).

30. Le format **large** se lit bien (une colonne par mois) et convient aux tableaux de présentation ; le format **long** (une ligne par observation) convient au calcul, au filtrage et aux graphiques. En pandas : `melt` (large vers long), `pivot` ou `pivot_table` (long vers large) ; en R : `pivot_longer`, `pivot_wider` (4.3.4).

31. Un notebook garde un **état caché** : on peut exécuter les cellules dans le désordre, ou supprimer une cellule dont dépendent d'autres. L'habitude qui protège : **redémarrer et tout exécuter** de haut en bas avant de partager, avec des chemins relatifs et des données en entrée (4.4.2, 4.4.3).

32. La vectorisation applique l'opération à **toute la colonne** dans du code compilé ; une boucle Python répète l'interprétation ligne par ligne, ce qui est beaucoup plus lent sur des dizaines de milliers de lignes (4.6.1).

33. **Satisfaction de 1 à 5** : ordinal (compter, ordonner, médiane ; la moyenne est discutable). **Ville** : nominal (compter, mode). **Température en °C** : intervalle (différences et moyenne, mais 20 °C n'est pas « deux fois plus chaud » que 10 °C). **Chiffre d'affaires** : rapport (toutes les opérations, y compris les ratios) (5.1.2).

34. Le grain de commandes est **une commande** ; celui de `lignes_commande`, **une ligne de commande**. Joindre les deux puis sommer une colonne de commandes la **multiplie** par le nombre de

lignes de chaque commande : c'est le double comptage (5.1.4, 3.2.6).

35. Un identifiant n'est pas une quantité : lu comme un nombre, 007421 devient 7421 et les zéros de tête sont perdus (et deux identifiants distincts peuvent se confondre). On lit les identifiants en **texte** (5.1.3).

36. Sans aucune hypothèse, la moyenne de tous les clients est entre $0,24 \times 3,64 + 0,76 \times 1 \approx 1,63$ (si tous les non-répondants étaient au plus bas) et $0,24 \times 3,64 + 0,76 \times 5 \approx 4,67$: l'intervalle est très large, et c'est pourquoi on étudie **qui** a répondu (5.3.4).

37. Promoteurs (9–10) : $20/50 = 40\%$; détracteurs (0–6) : $15/50 = 30\%$; **NPS** = +10 **points** (les 7–8 sont « passifs » et ne comptent pas) (5.3.6).

38. 1 067 répondants pour ± 3 points et 2 401 pour ± 2 points, dans le pire cas : diviser la marge d'erreur par 1,5 coûte 2,25 fois plus de réponses (5.4.3).

39. Par exemple : **biais de sélection** (tirer au hasard dans la population) ; **non-réponse** (relancer, comparer répondants et invités, pondérer) ; **désirabilité sociale** (anonymat, formulation neutre) ; **formulation et ordre des questions** (test pilote, rotation de l'ordre). Aucune de ces corrections n'est parfaite (5.4.4).

40. Le code 429 signifie « trop de requêtes » : on a dépassé la **limite de débit**. On attend (en suivant l'en-tête Retry-After s'il existe), on espace les appels, et l'on réessaie avec un délai croissant. robots.txt indique ce que l'éditeur autorise aux programmes ; le consulter est une règle de **politesse** et de prudence juridique, à compléter par les conditions d'utilisation du site (5.5.2 à 5.5.4).

Votre grille d'auto-évaluation

Pour chaque ligne : **je sais l'expliquer** / **je sais le faire** / **à revoir**.

Compétence	Où la retravailler
Résumer une variable : centre, dispersion, forme, valeurs aberrantes	1.1
Choisir une loi simple et lire une courbe normale	1.2
Mesurer l'erreur d'échantillonnage, calculer un intervalle de confiance	1.3
Lire une corrélation et la distinguer d'une causalité	1.4
Calculer pourcentages, croissances, moyennes pondérées, marges	1.5
Écrire des formules Excel (conditionnelles, recherche, texte, dates)	2.1
Construire un tableau croisé dynamique	2.2
Importer et transformer avec Power Query	2.3
Organiser un classeur fiable et le contrôler	2.4
Interroger une base : SELECT, WHERE, GROUP BY, HAVING	3.1
Joindre des tables sans doubler les lignes	3.2
Utiliser les fonctions fenêtres	3.3
Structurer une requête avec des CTE et la vérifier	3.4
Manipuler un DataFrame pandas	4.1
Faire la même chose avec le tidyverse	4.2
Lire, regrouper, joindre, restructurer des données	4.3

Compétence	Où la retravailler
Livrer un notebook reproductible	4.4
Reconnaître les types et les niveaux de mesure, le grain d'une table	5.1
Choisir et évaluer une source de données	5.2
Concevoir une enquête et en corriger les biais	5.3, 5.4
Mener une première analyse de bout en bout	Projet du volume